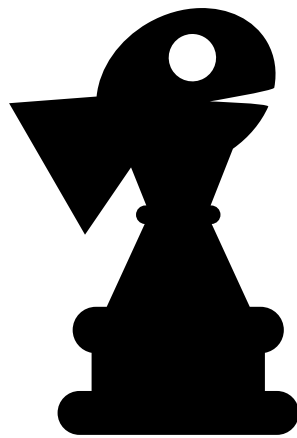


# **Analyse und Optimierung einer Schach-Engine**



## **Studienarbeit**

**Bachelor of Science**

des Studiengangs Informatik  
an der Dualen Hochschule Baden-Württemberg Stuttgart

von

**Emil Schläger & Lukas Piorek**

bei

**Prof. Dr. Janko Dietzsch**

11. Juni 2026

## **Ehrenwörtliche Erklärung**

Ich versichere hiermit, dass ich die vorliegende Arbeit selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe und diese Arbeit bei keiner anderen Prüfung mit gleichem oder vergleichbarem Inhalt vorgelegt habe und diese bislang nicht veröffentlicht wurde. Ich habe bei der Erstellung der Arbeit KI-Werkzeuge eingesetzt. Dies habe ich an den entsprechenden Stellen der Arbeit kenntlich gemacht.

Emil Sillöy

Lukas Piorek

## Zusammenfassung

Diese Studienarbeit befasst sich mit der Konzeption, Implementierung und systematischen Optimierung einer rein algorithmischen Schach-Engine namens „Thunfisch“. Da der Zustandsraum des Schachspiels gigantisch ist und dies eine einfache Brute-Force-Suche ausschließt, liegt der Fokus der Arbeit auf der Entwicklung hocheffizienter Datenstrukturen und Suchalgorithmen. Als Programmiersprache dient Rust, da diese maximale Ausführungsgeschwindigkeit mit Speichersicherheit vereint. Die Architektur der Engine stützt sich auf drei zentrale Säulen:

- **Zuggenerierung:** Das Spielbrett wird hardwarenah über Bitboards abgebildet. Mithilfe von Masken und Magic Bitboards ist die Engine in der Lage, Millionen von Zügen pro Sekunde zu generieren.
- **Suchalgorithmus:** Der Kern der Engine basiert auf dem Minimax-Verfahren mit Alpha-Beta-Pruning. Um die exponentiell wachsende Suchkomplexität zu bewältigen, wird diese Basis um diverse Heuristiken erweitert.
- **Positionsevaluation:** Die statische Stellungsbewertung nutzt zwischen Mittel- und Endspiel interpolierende Piece Square Tables sowie spezifische Terme für Bauernstrukturen und Königssicherheit. Die Gewichtungen dieser Parameter werden automatisiert durch ein Texel-Tuning-Verfahren trainiert.

Die Leistungssteigerung durch die einzelnen implementierten Features wird während der Entwicklung kontinuierlich durch Sequential Probability Ratio Tests (SPRT) empirisch validiert. In einem abschließenden Turnier erreicht Thunfisch eine kalibrierte Spielstärke von 2559 Elo. Damit wird das zu Beginn definierte Ziel von 2500 Elo nicht nur erreicht, sondern übertroffen. Zusätzlich werden die Auswirkungen der einzelnen Funktionen auf die Spielstärke analysiert.

Die Arbeit demonstriert somit erfolgreich, dass die Entwicklung einer Schach-Engine auf Großmeister-Niveau nicht zwingend maschinelles Lernen erfordert, sondern allein durch die tiefgehende Optimierung klassischer Alpha-Beta-Suchen in einer performanten Sprache wie Rust realisierbar ist.

# Abstract

This work addresses the design, implementation, and systematic optimization of „Thunfisch“, a purely algorithmic chess engine. Due to the vast state space of chess, which precludes a naive brute-force search, this work focuses on developing highly efficient data structures and search algorithms. Rust is chosen as the programming language to combine maximum execution speed with memory safety. The engine’s architecture rests on three core pillars:

- **Move Generation:** The chessboard is represented at a low-level using bitboards. Utilizing bitmasks and magic bitboards, the engine is capable of generating millions of moves per second.
- **Search Algorithm:** The core of the engine is based on the minimax algorithm with alpha-beta pruning. To mitigate exponential search complexity, this foundation is enhanced by various search heuristics.
- **Evaluation:** The static position evaluation employs piece-square tables that interpolate between middlegame and endgame, alongside specific terms for pawn structures and king safety. The weights of these parameters are optimized automatically using a Texel tuning method.

The performance gains of individual features are empirically validated during development using Sequential Probability Ratio Tests (SPRT). In a concluding tournament, Thunfisch reaches a calibrated playing strength of 2559 Elo, thereby not only meeting but exceeding the initial target of 2500 Elo. Additionally, the impact of each feature on the overall playing strength is analyzed.

Consequently, this thesis successfully demonstrates that developing a grandmaster-level chess engine does not strictly require machine learning. Instead, it can be fully realized through the deep optimization of traditional alpha-beta search techniques within a high-performance language like Rust.

# Inhaltsverzeichnis

|                                                        |      |
|--------------------------------------------------------|------|
| Abkürzungsverzeichnis .....                            | VIII |
| Abbildungsverzeichnis .....                            | IX   |
| Tabellenverzeichnis .....                              | XI   |
| Formelverzeichnis .....                                | XII  |
| Quellcodeverzeichnis .....                             | XIII |
| 1 Einleitung .....                                     | 1    |
| 1.1 Motivation und Relevanz von Computerschach .....   | 2    |
| 1.2 Problemstellung und Abgrenzung .....               | 3    |
| 1.3 Zielsetzung der Arbeit .....                       | 4    |
| 1.4 Struktur der Arbeit .....                          | 4    |
| 2 Theoretische Grundlagen .....                        | 6    |
| 2.1 Kommunikation im Schach .....                      | 6    |
| 2.1.1 Algebraische Notation .....                      | 6    |
| 2.1.2 Forsyth Edwards Notation .....                   | 8    |
| 2.1.3 Universal Chess Interface .....                  | 8    |
| 2.2 Elo-Wertung .....                                  | 10   |
| 2.3 Rust im Kontext rechenintensiver Anwendungen ..... | 11   |
| 2.3.1 Zero-Cost Abstraction .....                      | 12   |
| 2.3.2 Speichersicherheit ohne Garbage Collector .....  | 13   |
| 2.3.3 Branchless Programming .....                     | 15   |
| 2.4 Constant Evaluation .....                          | 16   |
| 2.5 Formale Repräsentation des Spiels .....            | 16   |
| 2.5.1 Bitboards .....                                  | 18   |
| 2.5.2 Zobrist Hashing .....                            | 21   |
| 3 Systemarchitektur der Schach-Engine .....            | 24   |
| 3.1 Komponenten .....                                  | 25   |
| 3.2 Ablauf der Suche .....                             | 26   |
| 4 Zuggenerierung .....                                 | 27   |
| 4.1 Ablauf .....                                       | 28   |
| 4.2 Berechnung der Schachmaske .....                   | 29   |
| 4.3 Berechnung der Fesselmaske .....                   | 32   |
| 4.4 Zuggenerierung für statische Figuren .....         | 34   |

|       |                                             |    |
|-------|---------------------------------------------|----|
| 4.5   | Zuggenerierung für Gleitfiguren .....       | 37 |
| 4.5.1 | Magic-Bitboards .....                       | 39 |
| 4.5.2 | Reduktion auf legale Züge .....             | 41 |
| 5     | Suchalgorithmen .....                       | 44 |
| 5.1   | Minimax .....                               | 44 |
| 5.2   | Alpha-Beta-Pruning .....                    | 46 |
| 5.3   | Quiescence Search .....                     | 48 |
| 5.4   | Principal Variation Search .....            | 50 |
| 5.5   | Iterative Deepening .....                   | 51 |
| 5.6   | Transpositionstabelle .....                 | 51 |
| 5.6.1 | Optimierte Datenstruktur der Einträge ..... | 51 |
| 5.6.2 | Speichern von Einträgen .....               | 53 |
| 5.6.3 | Abfragen von Einträgen .....                | 55 |
| 5.6.4 | Einsatzzweck in der Suche .....             | 55 |
| 5.7   | Zugsortierung .....                         | 56 |
| 5.7.1 | MVV-LVA .....                               | 57 |
| 5.7.2 | Killer Heuristics .....                     | 57 |
| 5.7.3 | History Heuristics .....                    | 58 |
| 5.7.4 | Move Picker .....                           | 60 |
| 5.8   | Forward Pruning .....                       | 62 |
| 5.9   | Late Move Reductions .....                  | 64 |
| 5.9.1 | Festlegen der Reduzierung .....             | 64 |
| 5.9.2 | LMR in Kombination mit PVS .....            | 65 |
| 6     | Positionsevaluation .....                   | 66 |
| 6.1   | Material .....                              | 67 |
| 6.2   | Piece Square Tables .....                   | 67 |
| 6.2.1 | Theoretischer Hintergrund .....             | 67 |
| 6.2.2 | Umsetzung .....                             | 68 |
| 6.3   | Weitere Terme der Evaluation .....          | 70 |
| 6.3.1 | Freibauern .....                            | 70 |
| 6.3.2 | Doppelbauern .....                          | 71 |
| 6.3.3 | Königssicherheit .....                      | 72 |
| 6.4   | Texel Tuning .....                          | 73 |
| 7     | Evaluation und Ergebnisse .....             | 76 |
| 7.1   | Testmethodik .....                          | 76 |

|       |                                                         |     |
|-------|---------------------------------------------------------|-----|
| 7.1.1 | Sequential Probability Ratio Test .....                 | 76  |
| 7.1.2 | Anwendung in der Engine-Entwicklung .....               | 77  |
| 7.1.3 | Methodik während der Entwicklung .....                  | 78  |
| 7.2   | Verbesserungen in Spielstärke einzelner Features .....  | 80  |
| 7.3   | Verbesserungen in Suchtiefe .....                       | 82  |
| 7.4   | Bestimmung der Elo-Wertung .....                        | 84  |
| 8     | Zusammenfassung und Ausblick .....                      | 86  |
| 8.1   | Limitationen .....                                      | 87  |
| 8.2   | Ausblick .....                                          | 88  |
|       | Literatur .....                                         | XIV |
|       | Anmerkung zur Nutzung von Künstlicher Intelligenz ..... | XX  |

## Abkürzungsverzeichnis

|         |                                                 |
|---------|-------------------------------------------------|
| ADAM    | Adaptive Moment Estimation                      |
| CCRL    | Computer Chess Rating Lists                     |
| CPU     | Central Processing Unit                         |
| FEN     | Forsyth Edwards Notation                        |
| LMR     | Late Move Reductions                            |
| LSB     | Least Significant Bit                           |
| MCTS    | Monte Carlo Tree Search                         |
| MVV-LVA | Most Valuable Victim - Least Valuable Aggressor |
| NMP     | Null Move Pruning                               |
| PSQT    | Piece Square Table                              |
| PV      | Principal Variation                             |
| PVS     | Principal Variation Search                      |
| QS      | Quiescence Search                               |
| RFP     | Reverse Futility Pruning                        |
| SPRT    | Sequential Probability Ratio Test               |
| TT      | Transpositionstabelle                           |
| UCI     | Universal Chess Interface                       |

# Abbildungsverzeichnis

|              |                                                                                                                                                  |    |
|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Abbildung 1  | Visualisierung der algebraischen Notation .....                                                                                                  | 7  |
| Abbildung 2  | Sequenzdiagramm des Kommunikationsablaufs in UCI .....                                                                                           | 9  |
| Abbildung 3  | Indizierung des Bitboards .....                                                                                                                  | 20 |
| Abbildung 4  | Systemarchitektur der Schach-Engine inklusive Datenfluss zwischen<br>den Komponenten .....                                                       | 24 |
| Abbildung 5  | Visualisierung einer Schachmaske - gesetzte Felder sind grün<br>markiert .....                                                                   | 31 |
| Abbildung 6  | Visualisierung einer Fesselmaske - gesetzte Felder sind grün<br>markiert .....                                                                   | 33 |
| Abbildung 7  | Reduktion der potenziellen Ziele eines Springers (a) auf pseudo-legale<br>Ziele (c) mithilfe der Maske aus freien und gegnerischen Feldern (b) . | 35 |
| Abbildung 8  | Reduktion der pseudo-legalen Ziele (a) eines Springers mithilfe einer<br>Schachmaske (b) auf legale Ziele (c) .....                              | 36 |
| Abbildung 9  | Visualisierung der Blocker Maske (c) durch UND-Verknüpfung der<br>aktuell belegten Felder (a) und der Magic-Maske (b) .....                      | 39 |
| Abbildung 10 | Reduktion der potenziellen Ziele eines Turms (a) auf pseudo-legale<br>Ziele (c) mithilfe der Maske aus freien und gegnerischen Feldern (b) .     | 41 |
| Abbildung 11 | Reduktion der pseudo-legalen Ziele (a) eines Turms mithilfe einer<br>Fesselmaske (b) auf legale Ziele (c) .....                                  | 42 |
| Abbildung 12 | Exemplarischer Minimax-Baum mit Blattbewertungen .....                                                                                           | 45 |
| Abbildung 13 | Minimax-Baum mit propagierten Bewertungen und markierter<br>Principal Variation (PV) .....                                                       | 46 |
| Abbildung 14 | Minimax-Baum mit abgeschnittenen Pfaden .....                                                                                                    | 48 |
| Abbildung 15 | Beispielposition für Horizon Effect, Weiß am Zug .....                                                                                           | 49 |
| Abbildung 16 | Beispielposition für Horizon Effect, nach 3 Ply .....                                                                                            | 49 |
| Abbildung 17 | Beispielposition für Killer moves, Weiß am Zug .....                                                                                             | 57 |
| Abbildung 18 | Zustandsdiagramm des Move Pickers .....                                                                                                          | 61 |
| Abbildung 19 | Visuelle Darstellung von Gleichung 7 .....                                                                                                       | 65 |
| Abbildung 20 | Beispiel für eine simple PSQT für Bauern .....                                                                                                   | 68 |
| Abbildung 21 | Interpolation zwischen Mittel- und Endspielmasken .....                                                                                          | 69 |

|              |                                                         |    |
|--------------|---------------------------------------------------------|----|
| Abbildung 22 | Exemplarische Masken zur Erkennung von Freibauern ..... | 71 |
| Abbildung 23 | Masken für Königssicherheit .....                       | 72 |
| Abbildung 24 | Elo-Gewinn jedes Features .....                         | 81 |
| Abbildung 25 | Auswirkung von Features auf die Suchtiefe .....         | 83 |

*Hinweis: Sofern bei einzelnen Abbildungen, Tabellen, Formeln oder Listings keine Quellen  
angeführt werden, wurden sie vom Verfasser erstellt.*

## Tabellenverzeichnis

|           |                                                                                 |    |
|-----------|---------------------------------------------------------------------------------|----|
| Tabelle 1 | Wert von Schachfiguren relativ zueinander .....                                 | 67 |
| Tabelle 2 | Elo-Gewinn mit Schätzungsfehler jedes Features .....                            | 82 |
| Tabelle 3 | Elo-Ergebnisse des Gauntlet-Turniers aus Sicht der Stash-Referenz-Engines ..... | 84 |

*Hinweis: Sofern bei einzelnen Abbildungen, Tabellen, Formeln oder Listings keine Quellen angeführt werden, wurden sie vom Verfasser erstellt.*

## Formelverzeichnis

|              |                                                                                                    |    |
|--------------|----------------------------------------------------------------------------------------------------|----|
| Gleichung 1  | Berechnung der Siegeswahrscheinlichkeit gegeben die Elo-Differenz<br>$\Delta$ .....                | 11 |
| Gleichung 2  | Berechnung der Differenz in Elo-Wertung zweier Engines .....                                       | 11 |
| Gleichung 3  | Speicherbedarf einer Lookup-Tabelle für alle Angriffs-Bitboards einer<br>Gleitfigur .....          | 38 |
| Gleichung 4  | Index-Berechnung in der Lookup-Tabelle für Gleitfiguren mit Magic<br>Bitboards .....               | 39 |
| Gleichung 5  | Berechnung der Priorität beim Speichern eines neuen Eintrags in der<br>Transpositionstabelle ..... | 54 |
| Gleichung 6  | Gravity Formula zur Aktualisierung von History-Werten .....                                        | 59 |
| Gleichung 7  | Funktion der Basisreduktion eines Zuges in LMR .....                                               | 64 |
| Gleichung 8  | Sigmoid zum Normieren der Evaluation .....                                                         | 73 |
| Gleichung 9  | Differenzenquotient zur numerischen Gradientenberechnung einer Loss-<br>Funktion .....             | 74 |
| Gleichung 10 | Hochrechnung zur Trainingsdauer einer Epoche .....                                                 | 74 |

*Hinweis: Sofern bei einzelnen Abbildungen, Tabellen, Formeln oder Listings keine Quellen  
angeführt werden, wurden sie vom Verfasser erstellt.*

## Quellcodeverzeichnis

|           |                                                                                                                           |    |
|-----------|---------------------------------------------------------------------------------------------------------------------------|----|
| Listing 1 | Repräsentation einer Schachfigur als Enum in Rust .....                                                                   | 13 |
| Listing 2 | Beispiel zum Ownership-Prinzip in Rust .....                                                                              | 14 |
| Listing 3 | Anwendung von Constant Evaluation in Rust für eine Lookup-Tabelle . .                                                     | 16 |
| Listing 4 | Spielbrett Struktur in Rust .....                                                                                         | 18 |
| Listing 5 | Vergleich zwischen Rust und Assembly Quellcode bei der Verwendung des<br>Newtype-Patterns .....                           | 21 |
| Listing 6 | Implementierung eines effizienten Iterators für ein Bitboard .....                                                        | 34 |
| Listing 7 | Definition der Datenstrukturen für komprimierte und dekomprimierte<br>Tabelleneinträge in der Transpositionstabelle ..... | 52 |
| Listing 8 | Schrittweiser Insertion Sort zum Identifizieren des nächstbesten Zuges . .                                                | 62 |
| Listing 9 | Ausschnitt einer CI/CD Pipeline zum automatischen Durchführen eines<br>SPRTs auf Pull Requests .....                      | 79 |

*Hinweis: Sofern bei einzelnen Abbildungen, Tabellen, Formeln oder Listings keine Quellen  
angeführt werden, wurden sie vom Verfasser erstellt.*

# 1 Einleitung

Das Schachspiel fasziniert die Menschheit seit Jahrhunderten und gilt traditionell als Sinnbild für die Komplexität menschlicher Entscheidungsfindung und strategischer Weitsicht. Mit dem Aufkommen der modernen Informatik Mitte des 20. Jahrhunderts rückte auch Schach schnell in den Fokus der Forschung und etablierte sich als eine spezielle Herausforderung für die Entwicklung von Künstlicher Intelligenz und algorithmischer Optimierung. Die vorliegende Studienarbeit widmet sich der Konzeption, Analyse und Optimierung einer Schach-Engine namens *Thunfisch*. Dieser Name wurde in humorvoller Anlehnung und als Hommage an die bekannte C++-Engine Stockfish gewählt, die aktuell als das stärkste Schachprogramm der Welt gilt. Im weiteren Verlauf dieser Studienarbeit wird die entwickelte Software häufig als Thunfisch referenziert.

Unter einer Schach-Engine versteht man ein Programm, welches autonom Schach spielen kann. Dabei kann sich das Programm sowohl gegen Menschen als auch gegen andere Schach-Engines messen. Die Kernaufgabe der Engine besteht darin, durch die sogenannte Zuggenerierung alle legalen Züge in einer aktuellen Brettstellung zu berechnen. Darauf aufbauend simuliert die Engine den Spielbaum viele Züge in die Zukunft. Die sich daraus ergebenden finalen Stellungspositionen werden anschließend durch eine Evaluationsfunktion numerisch bewertet. Das Ziel ist es, mit diesen Informationen für eine gegebene Brettstellung den besten möglichen Zug zu finden.

Die Entwicklung Thunfischs nahm ihren Anfang ursprünglich als Einzelprojekt im Rahmen des Wahlmoduls zur Programmiersprache Rust an der Hochschule. Aufgrund der rasant steigenden architektonischen Komplexität, des akademischen Anspruchs und der inhärenten Faszination der Thematik wuchs das Vorhaben jedoch schnell deutlich über seine anfänglichen Grenzen hinaus. Aus diesem Grund wird das Projekt nun im Rahmen dieser Studienarbeit weitaus tiefer bearbeitet.

In diesem einführenden Kapitel werden die grundlegende Motivation der Arbeit erläutert, die technischen Problemstellungen präzise abgegrenzt und die angestrebten Entwicklungsziele definiert.

## 1.1 Motivation und Relevanz von Computerschach

Die primäre Motivation für die Entwicklung von Schachprogrammen lag und liegt nicht in der Automatisierung des Spiels an sich, sondern in der Beschaffenheit des Problems: Schach ist ein Nullsummenspiel mit perfekter Information. Es gibt keinen Zufall und keine verborgenen Variablen. Die Regeln sind deterministisch und streng limitiert, doch der resultierende Zustandsraum ist gigantisch. Der US-amerikanische Mathematiker Claude Shannon berechnete 1950 die sogenannte Shannon-Zahl, die Untergrenze für die Anzahl der möglichen Schachpartien, auf etwa  $10^{120}$  [1]. Diese immense Komplexität schließt eine vollständige Lösung durch reines Durchrechnen aller Varianten (Brute-Force) selbst für modernste Supercomputer aus. Folglich zwingt das Schachspiel Informatiker dazu, effiziente Suchalgorithmen, Heuristiken und Architekturen zu entwickeln, die unter begrenzten Ressourcen möglichst optimale Entscheidungen treffen können. Die Relevanz des Computerschachs erstreckt sich daher weit über das Spielbrett hinaus. Die hier entwickelten Konzepte, wie Suchbäume, Pruning-Techniken und Evaluierungsfunktionen, finden auch außerhalb des Schachspiels weitreichende Anwendung in Entscheidungssystemen.

Der Wunsch, eine schachspielende Maschine zu erschaffen, existierte weit vor der Erfindung des Computers, wie der berühmte „Schachtürke“ von Wolfgang von Kempelen im 18. Jahrhundert zeigte [2]. Dabei handelte es sich um eine scheinbar komplexe und automatische Maschine mit vielen Zahnrädern und Hebeln. Sie erweckte den Eindruck, eigenständig Schach spielen zu können, jedoch hat in der Realität ein Mensch im Inneren der Maschine die Figuren manuell über Magnete bewegt [2].

Die wissenschaftliche Fundierung des Computerschachs begann im Jahr 1950 mit Claude Shannons wegweisendem Paper „Programming a Computer for Playing Chess“ [1]. Shannon definierte darin die Grundlagen für Schachalgorithmen, die bis heute Bestand haben. Er unterschied zwischen Typ-A-Strategien, die auf einer breiten, flachen Suche basieren (Brute-Force), und Typ-B-Strategien, die versuchen, ähnlich wie Menschen, nur vielversprechende Züge zu berechnen [1]. Parallel dazu entwarf Alan Turing mit „Turochamp“ den ersten Schachalgorithmus, der mangels ausreichender Hardware zunächst nur auf dem Papier existierte und von Turing selbst manuell „ausgeführt“ wurde. Damit wurde der Grundstein für die Entwicklung von Schach-Engines gelegt.

In den folgenden Jahrzehnten wurden Computer immer leistungsfähiger und Schachprogramme immer besser. Die Forschung konzentrierte sich auf die Optimierung der Baumsuche, wie z. B. durch Alpha-Beta-Pruning.

1995 stellte IBM seine Chess-Engine Deep Blue vor. Sie sorgte für einen historischen Meilenstein: Nachdem Deep Blue 1996 gegen den amtierenden Schachweltmeister Garry Kasparov noch knapp verlor, besiegte sie den Großmeister im Folgejahr [3]. Dieser Sieg markierte das Ende der menschlichen Überlegenheit. Seitdem ist die Kluft zwischen den besten Menschen und Programmen nur noch größer geworden.

Den bisher letzten großen Paradigmenwechsel erlebte das Computerschach im Jahr 2017 durch die Vorstellung von AlphaZero von Googles Tochterunternehmen DeepMind [4]. AlphaZero nutzte keine menschengemachte Evaluation mehr. Ausgestattet lediglich mit den Grundregeln des Spiels, brachte sich das System das Schachspielen durch Reinforcement Learning gegen sich selbst bei. Die Kombination aus Monte Carlo Tree Search (MCTS) und tiefen Neuronalen Netzen zur Stellungsbewertung schlug die bis dahin beste Schach-Engine Stockfish deutlich [4]. Dies bewies die Überlegenheit von Machine-Learning-Ansätzen für die Evaluation. In der Folge wurden beide Ansätze kombiniert. Moderne Engines integrieren heute Neuronale Netze, um die tiefe Suche der klassischen Algorithmen mit der Bewertung von Neuronalen Netzen zu vereinen [4].

## 1.2 Problemstellung und Abgrenzung

Die zentrale Problemstellung bei der Entwicklung einer Schach-Engine liegt in der immensen Komplexität des Zustandsraums des Spiels. Minimax, ein naives Brute-Force-Verfahren (s. Abschnitt 5.1), stößt aufgrund des exponentiellen Wachstums des Suchbaums extrem schnell an seine Grenzen.

Um diesen großen Suchraum zu bewältigen, basiert die Architektur dieser Chess-Engine auf drei Säulen:

- **Zuggenerierung:** Es bedarf einer schnellen Zuggenerierung, welche für jede beliebige Brettstellung in Bruchteilen von Sekunden fehlerfrei alle legalen Folgezüge berechnet
- **Suchalgorithmus:** Darauf aufbauend ist eine effiziente Suche unerlässlich, die nicht blind jeden möglichen Ast des Spielbaums durchsucht, sondern die schiere

Komplexität durch den Einsatz intelligenter Heuristiken drastisch reduziert und irrelevante Pfade frühzeitig verwirft.

- **Statische Evaluation:** Abschließend wird eine statische Evaluationsfunktion benötigt, die in der Lage ist, die letztendlich erreichten Endpositionen zu bewerten, um der Suche überhaupt erst eine verlässliche Entscheidungsgrundlage zu bieten.

Allgemein konzentriert sich die vorliegende Studienarbeit explizit und ausschließlich auf die rein algorithmische Schachprogrammierung. Das bedeutet eine bewusste, methodische Abgrenzung von Ansätzen, die auf maschinellem Lernen oder stochastischer Baumsuche (wie z.B. MCTS) basieren. Diese Entscheidung beruht auf persönlicher Präferenz der Autoren. Der Fokus soll vollständig auf den fundamentalen Algorithmen, Datenstrukturen und hardwarenahen Optimierungen liegen. Die direkte Auseinandersetzung mit der reinen Logik des Spiels und der Architektur des Suchbaums wird hier als relevanter erachtet. Zwar dominieren Ansätze mit maschinellem Lernen für die Evaluation heute unbestreitbar, jedoch verlagert sich der Entwicklungsaufwand bei diesen Architekturen im späteren Projektverlauf zwangsläufig auf das Sammeln von Daten, langem Modelltraining, iterativem Hyperparameter-Tuning und ständiges Retraining. Da dieser iterative und repetitive Prozess den Fokus von der eigentlichen Entwicklung verschiebt, wurde bewusst auf sie verzichtet. Stattdessen soll die Spielstärke durch klassische Heuristiken und Algorithmen maximiert werden.

### 1.3 Zielsetzung der Arbeit

Das übergeordnete Ziel dieser Studienarbeit ist die vollständige Implementierung, Analyse und Optimierung einer rein algorithmischen Schach-Engine. Als Programmiersprache wird die emergente, für u.a. ihre Geschwindigkeit bekannte Sprache Rust gewählt. Das Ziel der Engine ist es, die Wertung von 2500 Elo zu erreichen. Diese Wertung wird gewählt, da sie auf menschlicher Ebene eine der Voraussetzungen dafür ist, den Titel des Großmeisters, des höchsten offiziell erreichbaren Titels für einen Schachspieler, zu erlangen [5].

### 1.4 Struktur der Arbeit

Die Arbeit gliedert sich entlang der zentralen Komponenten einer Schach-Engine. Zunächst werden in Abschnitt 2 die theoretischen Grundlagen gelegt: Notationen, das

Elo-System, die für diese Arbeit relevante Rust-Programmierung sowie die formale Repräsentation des Spiels durch Bitboards und Zobrist-Hashing. Darauf aufbauend zeigt Abschnitt 4 die Implementierung eines legalen Zuggenerators mittels Bitboards, Masken und Magic Bitboards.

Abschnitt 5 behandelt die Suchalgorithmen der Engine, ausgehend vom naiven Minimax-Verfahren über Alpha-Beta-Pruning bis hin zu den heuristischen Erweiterungen wie der Principal Variation Search, Transpositionstabelle, Zugsortierung, Forward Pruning und Late Move Reductions. Abschnitt 6 beschreibt die Evaluationsfunktion mit Piece Square Tables, weiteren strategischen Termen und dem Texel-Tuning-Verfahren zur automatischen Optimierung der Gewichte.

Die Ergebnisse der implementierten Optimierungen werden in Abschnitt 7 systematisch erfasst: Für jedes Feature wird der Elo-Gewinn mittels sequentieller Hypothesentests gemessen, und die absolute Spielstärke der Engine wird durch ein Turnier gegen die Referenz-Engine Stash bestimmt. Abschließend fasst Abschnitt 8 die Arbeit zusammen, diskutiert Limitationen und gibt einen Ausblick auf mögliche Weiterentwicklungen.

## 2 Theoretische Grundlagen

Dieses Kapitel legt die theoretischen Grundlagen für die Entwicklung einer Schach-Engine in Rust. Es behandelt zunächst die Kommunikationsprotokolle und Notationen, die den Austausch zwischen Mensch und Maschine ermöglichen, sowie das Elo-Bewertungssystem als Maß für Spielstärke. Anschließend werden die relevanten Eigenschaften von Rust bei der Entwicklung einer Schach-Engine dargestellt. Zuletzt wird die formale Repräsentation des Schachspiels behandelt, die das Fundament des gesamten Projekts bildet. Damit eine Schach-Engine Züge effizient berechnen kann, genügt eine naive Modellierung nicht. Das Spiel muss im Quellcode so abgebildet werden, dass die zentralen Operationen der Zuggenerierung, Stellungsbewertung und Suche mit maximaler Performanz ausgeführt werden können. Die dafür entwickelten Datenstrukturen wie Bitboards und Zobrist-Hashing sind keine Detailfragen der Implementierung, sondern die Grundlage, auf der die gesamte Engine erst effizient arbeiten kann.

### 2.1 Kommunikation im Schach

Damit die Engine den Zustand eines Schachspiels und ausgeführte Züge verarbeiten kann, braucht es Kommunikationsmethoden zwischen Mensch und Schach-Engine, sowie zwischen zwei Engines. Während menschliche Spieler meist auf kontextbasierte Formate, wie die kurze algebraische Notation, zurückgreifen, benötigen Computer eindeutige Notationen. Im Folgenden werden die wichtigsten Kommunikationsmethoden und deren Bedeutung bei der Entwicklung einer Schach-Engine erläutert.

#### 2.1.1 Algebraische Notation

Die kurze algebraische Notation ist das weltweit am häufigsten genutzte Format zur Kommunikation von Zügen. Sie basiert auf der Interpretation des Schachbretts als ein Koordinatensystem: Ausgehend vom Feld unten links werden die Linien mit a bis h und die Reihen mit 1 bis 8 gekennzeichnet. Das Feld unten links wird somit mit a1 bezeichnet und das Feld oben rechts mit h8. Eine visuelle Darstellung dieses Systems ist in Abbildung 1 abgebildet.

|   | a  | b  | c  | d  | e  | f  | g  | h  |   |
|---|----|----|----|----|----|----|----|----|---|
| 8 | a8 | b8 | c8 | d8 | e8 | f8 | g8 | h8 | 8 |
| 7 | a7 | b7 | c7 | d7 | e7 | f7 | g7 | h7 | 7 |
| 6 | a6 | b6 | c6 | d6 | e6 | f6 | g6 | h6 | 6 |
| 5 | a5 | b5 | c5 | d5 | e5 | f5 | g5 | h5 | 5 |
| 4 | a4 | b4 | c4 | d4 | e4 | f4 | g4 | h4 | 4 |
| 3 | a3 | b3 | c3 | d3 | e3 | f3 | g3 | h3 | 3 |
| 2 | a2 | b2 | c2 | d2 | e2 | f2 | g2 | h2 | 2 |
| 1 | a1 | b1 | c1 | d1 | e1 | f1 | g1 | h1 | 1 |
|   | a  | b  | c  | d  | e  | f  | g  | h  |   |

Abbildung 1: Visualisierung der algebraischen Notation

Ein Zug wird in der Regel durch den Großbuchstaben der ziehenden Figur (K für König, Q für Dame, R für Turm, B für Läufer und N für Springer) und dem Zielfeld beschrieben. Bei Bauernzügen entfällt der Buchstabe der Figur. Schlagende Züge erhalten zusätzlich ein „x“ (z.B. Nxf3). Zusätzlich definiert „+“ eine Schachsituation, „#“ ein Schachmatt und „O-O“ die kurze, sowie „O-O-O“ die lange Rochade.

Während die kurze algebraische Notation für Menschen sehr intuitiv ist, stellt sie für Schach-Engines eine Herausforderung dar. Es wird nämlich nicht definiert, welche der Figuren sich genau zum Zielfeld bewegt. So muss immer erst berechnet werden, welcher der beiden Springer sich zum Beispiel wirklich auf das Spielfeld bewegen kann. Dafür muss der Brettzustand bekannt sein und Teile der Zuggenerierung müssen ausgeführt werden, um diese Mehrdeutigkeiten aufzulösen.

Deswegen verwenden die meisten Engines eine Variante der langen algebraischen Notation. In dieser Notation werden die Start- und Zielfelder angegeben (z. B. e2e4). Dadurch entstehen keine Mehrdeutigkeiten. Bei der Rochade wird nur die Positionsänderung des Königs angegeben. Dadurch, dass sich dieser dann aber mehr als ein Feld bewegt hat, kann die Rochade eindeutig identifiziert werden (z.B. e1g1). Bei der Bauernumwandlung wird zusätzlich noch die Zielfigur als Kleinbuchstabe angehängt (z.B. a7a8q).

### 2.1.2 Forsyth Edwards Notation

Während die algebraische Notation isolierte Züge beschreibt, dient die Forsyth Edwards Notation (FEN) dazu, einen statischen Brettzustand in Textform abzubilden. Die FEN ist essenziell, um direkt eine bestimmte Spielsituation in die Engine zu laden. Besonders für das Testen vieler Spielsituationen ist die Unterstützung dieser Notation unerlässlich.

Eine FEN-Zeichenkette besteht aus sechs durch Leerzeichen getrennten Feldern, die alle notwendigen Statusinformationen enthalten:

- **Figurenaufstellung:** Beschreibt das Brett zeilenweise von Reihe 8 bis Reihe 1. Großbuchstaben stehen für weiße Figuren, Kleinbuchstaben für schwarze. Zahlen von 1 bis 8 definieren die Anzahl aufeinanderfolgender leerer Felder. Die einzelnen Reihen werden durch einen Schrägstrich („/“) getrennt.
- **Aktiver Spieler:** Ein „w“ für Weiß oder ein „b“ für Schwarz, je nachdem, wer am Zug ist.
- **Rochade:** Zeigt an, ob die kurze (K/k) oder lange (Q/q) Rochade für Weiß bzw. Schwarz theoretisch noch möglich ist. Ist keine Rochade mehr erlaubt, wird dies mit einem Bindestrich („-“) notiert.
- **En-Passant:** Gibt das Feld an, auf dem im aktuell folgenden Zug ein En-Passant-Schlag möglich wäre. Andernfalls steht hier ein „-“.
- **Halbzug-Uhr:** Zählt die Halbzüge seit dem letzten Bauernzug oder Schlagzug. Dieser Wert ist kritisch für die korrekte Implementierung der 50-Züge-Remisregel.
- **Zugnummer:** Zählt die bisherigen Spielzüge der Partie. Der Zähler startet bei 1 und wird nach jedem abgeschlossenen Zug von Schwarz um eins inkrementiert.

So wird zum Beispiel die Startposition im Schach mit folgender Zeichenkette beschrieben: `rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1`

### 2.1.3 Universal Chess Interface

Das Universal Chess Interface (UCI) ist ein weit verbreitetes Protokoll für die Kommunikation zwischen Engines und Presentation Layer Programmen wie grafische

Oberflächen oder online Plattformen wie Lichess<sup>1</sup>. UCI ist der de facto Standard für diesen Nutzungszweck, besitzt aber keine formale Spezifikation - Von der Community wird ein von Stefan Meyer-Kahlen als Teil seines Programms Shredder Chess verfügbares Dokument [6] als Standardbeschreibung akzeptiert.

UCI nutzt den Standard I/O des Betriebssystems, um mit einem laufenden Prozess einer Engine zu kommunizieren. Abbildung 2 zeigt den simplifizierten Standardablauf der UCI-Kommunikation:

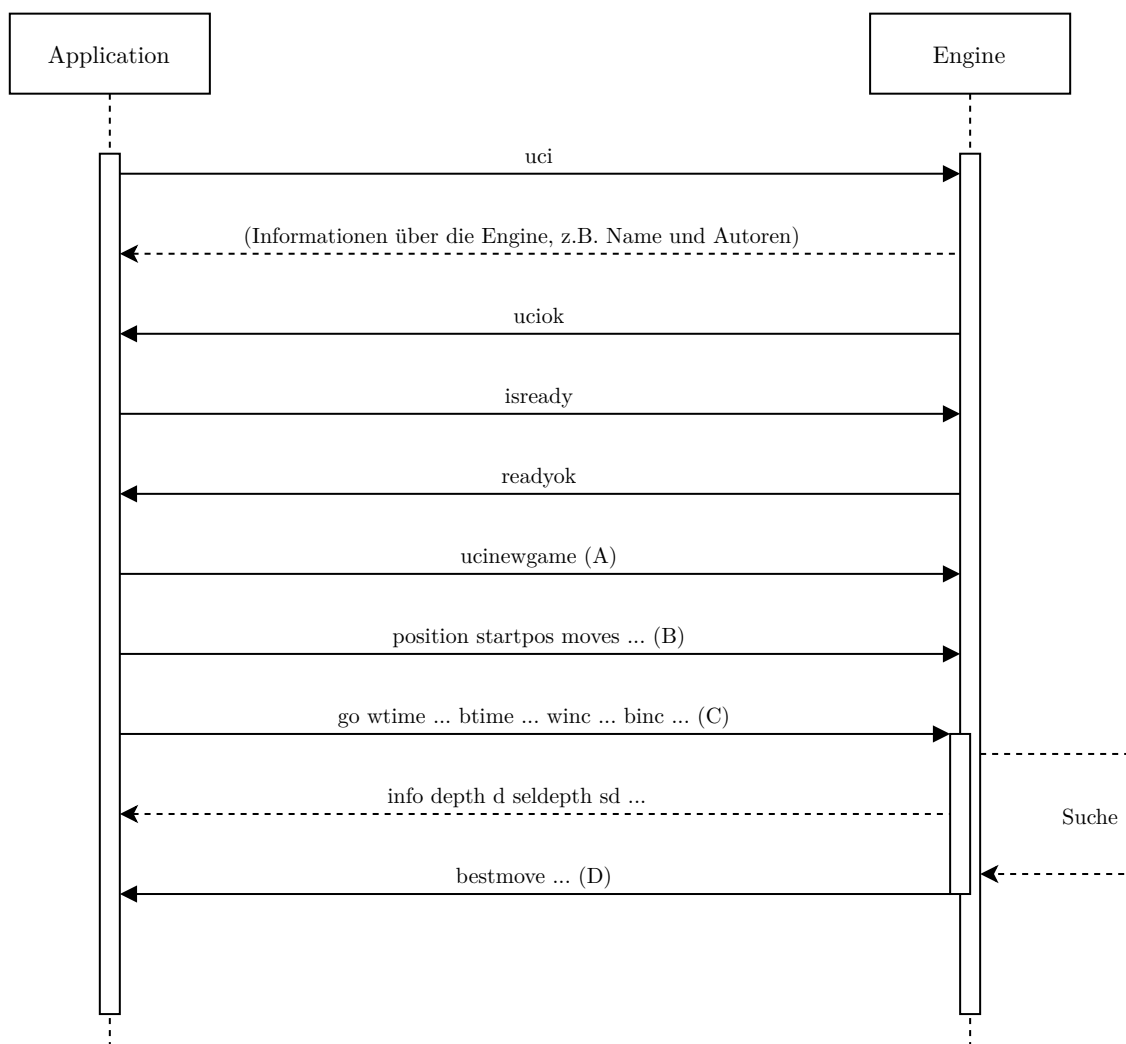


Abbildung 2: Sequenzdiagramm des Kommunikationsablaufs in UCI

Die wichtigsten Momente hierbei sind wie folgt:

<sup>1</sup><https://lichess.org> (30.05.2026)

- (A) - `ucinewgame` teilt der Engine mit, dass ein neues Spiel begonnen wurde. Dies gibt der Engine die Möglichkeit, gewisse Zustände (wie beispielsweise die Transpositionstabelle) zurückzusetzen.
- (B) - Das Kommando `position` teilt der Engine mit, welche Position sich aktuell auf dem Feld befindet. Für die Standard-Schachvariante folgt dies immer dem Format `position startpos moves`, gefolgt von einer Kette an Zügen im Format der langen algebraischen Notation, getrennt durch Leerzeichen
- (C) Das Kommando `go` gibt der Engine die Freigabe, ihre Suche zu starten. An den Befehl können verschiedene Parameter angehängt werden; in einem Spiel auf Zeit sind diese `wtime` und `btime` sowie `winc` und `binc`. Diese geben an, wie viel Zeit beide Spieler jeweils haben, sowie den Inkrement, den jeder Spieler nach einem Zug bekommt. Die Angabe erfolgt jeweils in Millisekunden.
- (D) Nachdem die Engine ihre Suche beendet hat, gibt sie den besten gefundenen Zug im Format der langen algebraischen Notation zurück.

Während der Suche kann eine Engine unter dem Prefix `info` Informationen über die laufende Suche ausgeben. Auch bei diesen Informationen gibt es einige Werte, die von [6] erwähnt werden und von grafischen Programmen unterstützt werden. Diese enthalten unter anderem:

- `depth`: Die Tiefe der Hauptsuche in Ply
- `seldepth`: Die Tiefe der gesamten Suche, inklusive Quiescence Search (QS), in Ply
- `score`: Die gesuchte Evaluation der Ausgangsposition
- `pv`: Die gefundene Principal Variation (PV)

Es ist üblich, eine Info-Ausgabe pro gesuchter Tiefe (vgl. Abschnitt 5.5) auszugeben.

## 2.2 Elo-Wertung

Die Elo-Wertung ist eine statistisch fundierte Quantifizierung der Spielstärken von Schachspielern. Die Wertung ist benannt nach ihrem Entwickler Arpad Elo, welcher das System 1978 in [7] formal veröffentlicht hat. Elo geht davon aus, dass das Spielverhalten eines Spielers mit  $\mathcal{N}(\mu, \sigma)$  normalverteilt ist, und es somit auch für einen schwächeren Spieler „an einem guten Tag“ möglich ist, einen stärkeren Spieler zu schlagen. Da ein Schachspiel nur drei mögliche Ausgänge hat (Sieg Weiß / Unentschieden / Sieg

Schwarz), wird die Elo-Wertung eines Spielers durch viele Spiele nach dem Gesetz der großen Zahlen gebildet, um sich an  $\mu$  anzunähern [7, ch. 1.3].

Im Kern des Elo-Systems steht die Berechnung einer Wahrscheinlichkeit, dass ein Spieler  $A$  gegen einen anderen Spieler  $B$  in einem Spiel gewinnen kann, basierend auf der Differenz ihrer Elo-Wertungen  $\Delta = \text{Elo}_B - \text{Elo}_A$ . Diese Wahrscheinlichkeit gibt Elo durch einen Sigmoid an [7, eq. (46)]:

$$P_w = \frac{1}{1+10^{-\frac{\Delta}{C}}}$$

Gleichung 1: Berechnung der Siegeswahrscheinlichkeit gegeben die Elo-Differenz  $\Delta$

$C$  dient hierbei als normalisierende Konstante [7, ch. 8.32]. In Fastchess [8], einem populären Programm zum Durchführen von Turnieren und Vergleichen verschiedener Schach-Engines, gilt  $C = 400$ . Im Kontext der Engine-Entwicklung ist hauptsächlich eine Umformung dieser Formel relevant. Hier ist in der Regel das Ziel, herauszufinden, wie viel Elo-Punkte eine Engine  $A$  stärker ist als eine Engine  $B$ . Die Siegeswahrscheinlichkeit  $P_w$  kann hierbei empirisch ermittelt werden, indem viele Spiele der beiden Engines durchgeführt werden. Durch triviale Umformungen ergibt sich die Differenz in Elo zwischen den beiden Engines wie folgt:

$$\Delta = -C \cdot \log_{10} \left( \frac{1}{P_w} - 1 \right)$$

Gleichung 2: Berechnung der Differenz in Elo-Wertung zweier Engines

Im Laufe der Arbeit wird die Elo-Wertung, bzw. die Differenz an Elo-Wertung, verwendet, um die Spielstärke verschiedener Engines, oder verschiedenen Versionen derselben Engine, zu repräsentieren.

## 2.3 Rust im Kontext rechenintensiver Anwendungen

Rust ist eine kompilierte Programmiersprache, die auf hohe Performanz, Speichersicherheit und Parallelität ausgelegt ist. Sie basiert auf einem Projekt von Graydon Hoare. Er wollte eine schnelle Programmiersprache entwerfen, welche aber trotzdem eine sichere Speicherverwaltung besitzt. Die Sprache wurde 2010 offiziell angekündigt und die Version 1.0 erschien im Jahr 2015 [9]. Mittlerweile unterliegt die Sprache der Rust Foundation, welche eigens für dessen Verwaltung gegründet wurde.

Sie ist eine der am schnellsten wachsenden und beliebtesten Programmiersprachen laut der jährlichen Stackoverflow Survey [10].

Historisch gesehen war die Schachprogrammierung stets ein Rennen um maximale Ausnutzung der Hardware. Da eine Erhöhung der Schnelligkeit die Spielstärke unmittelbar erhöht, dominierten lange Zeit Programmiersprachen, die eine absolute Kontrolle über den Prozessor erlaubten. Deswegen wurden die ersten Engines in Assembly geschrieben. Ein bekanntes Beispiel ist die Engine Sargon, welche 1978 das Turnier bei der „The Second West Coast Computer Faire“ gewann [11]. Mit der steigenden Komplexität der Engines hat sich C++ zum neuen Standard für Engines entwickelt. Die bekannteste C++ Engine ist Stockfish [12], welches aktuell die stärkste Schach-Engine der Welt ist [13]. In jüngster Zeit gewinnt Rust jedoch massiv an Bedeutung. Bekannte Beispiele umfassen die Engines Reckless [14] und Viridithas [15], welche aktuell beide zu den 10 stärksten Engines der Welt gehören. Dieser Aufstieg hat Rust seiner Geschwindigkeit zu verdanken, welche sich auf Augenhöhe mit C++ befindet, ohne jedoch die gleichen Risiken in der Speicherverwaltung zu haben.

### 2.3.1 Zero-Cost Abstraction

Das Prinzip der „Zero-Cost Abstraction“ ist ein essenzieller Teil von Rust. Es ermöglicht die Verwendung von Elementen aus höheren Programmiersprachen, LaufzeitPerformanz einzubüßen. Der einzige Geschwindigkeitsverlust entsteht während des Kompilierens [16]. Für die Entwicklung einer Schach-Engine ist dies ein großer Vorteil, weil abstraktere Repräsentationen aus höheren Programmiersprachen die Lesbarkeit und Flexibilität des Programmcodes erheblich erhöhen [17, S. 88].

Listing 1 verdeutlicht die Abstraktion in Rust mithilfe eines Enums. Während der Entwickler im Quellcode mit aussagekräftigen Wörtern wie `Piece::Knight` arbeitet, reduziert der Compiler diese Information auf einen einfachen numerischen Wert. Durch `#[repr(usize)]` kann sogar der darunter liegende Datentyp explizit festgelegt werden. In diesem Fall wird ein `Piece` also als 64-Bit unsigned Integer gespeichert. Dadurch ist es möglich ein Array durch einen Enum zu indizieren, welches bei Schach-Engines besonders in der Bewertung relevant ist [17, S. 88].

```

1  #[repr(usize)]
2  pub enum Piece {
3      Pawn = 0,
4      Knight = 1,
5      Bishop = 2,
6      Rook = 3,
7      Queen = 4,
8      King = 5,
9      Empty = 6,
10 }

```

 Rust

Listing 1: Repräsentation einer Schachfigur als Enum in Rust

### 2.3.2 Speichersicherheit ohne Garbage Collector

Während Sprachen wie C++ zwar auf einen Garbage Collector verzichten, jedoch die Speichersicherheit in die Verantwortung des Entwicklers legen, erkaufen Sprachen wie Python oder Java Sicherheit durch Speicherbereinigung auf Kosten der Performanz. Rust hingegen löst dieses Dilemma, indem es Speichersicherheit und maximale Performanz durch statische Analyse bereits zur Kompilierzeit vereint. Speziell handelt es sich um das Ownership-Prinzip, welches das Alleinstellungsmerkmal von Rust ist [16, S. 77]

Es basiert auf drei Grundregeln [16, S. 79]:

- Jeder Wert hat einen Owner (Besitzer)
- Es kann zu jeder Zeit nur einen Besitzer geben
- Wenn der Owner sein Scope (Gültigkeitsbereich) verlässt, wird der Wert gelöscht

Anhand von Listing 2 lässt sich die Funktionsweise des Ownership-Modells und dessen Auswirkungen auf die Speichersicherheit detailliert aufzeigen.

```
1 fn main() {  
2     let str_1 = "Schach";  
3     let str_2 = str_1;  
4     println!(str_1);  
5  
6     let string_1 = String::from("Schach");  
7     let string_2 = string_1;  
8     println!(string_1);  
9 }
```

 Rust

Listing 2: Beispiel zum Ownership-Prinzip in Rust

Zunächst wird die Variable `str_1` definiert, welche ein String-Literal (`&str`) speichert. Da es sich hierbei um einen Fat Pointer handelt, der lediglich aus einer Adresse und einer Länge besteht, wird dieser auf dem Stack verwaltet. Solche einfachen Typen implementieren in Rust den Copy-Trait. Bei der Zuweisung von `str_1` auf `str_2` wird der Wert bitweise auf dem Stack kopiert. Beide Variablen bleiben danach gültig, da die Operation extrem kostengünstig ist und keinen zusätzlichen Speicher auf dem Heap beansprucht. Das erste Print-Statement ist somit erfolgreich, da `str_1` seinen Besitz nicht verloren hat [16, S. 77 - 88].

Im Gegensatz dazu steht die Variable `string_1`, die ein Objekt des Typs `String` hält. Dieser Datentyp verwaltet seinen Inhalt auf dem Heap, um Größenänderungen zur Laufzeit zu ermöglichen. Rust verfolgt hier das Prinzip der Besitzübertragung. Bei der Zuweisung wird nicht der gesamte Inhalt des Heaps kopiert, was die Performanz beeinträchtigen würde, sondern nur das Ownership übertragen. Nach dieser Zeile ist `string_1` für den Compiler ungültig. Der Versuch, `string_1` in der folgenden Zeile auszugeben, führt zu einem Kompilierfehler. In C würde kein Fehler auftreten, aber es würden nun zwei Pointer auf den gleichen Speicher im Heap zeigen. Nun ist unklar welcher Pointer für das Freigeben des Speichers verantwortlich ist [16, S. 77 - 88].

Dieses Verhalten stellt sicher, dass niemals zwei Variablen für denselben Wert verantwortlich sind. Da dieses Verhalten bereits zur Kompilierzeit analysiert wird, benötigt Rust keinen Garbage Collector, um während der Laufzeit nach verwaisten Objekten zu suchen. Für die Entwicklung der Schach-Engine ist dieses Vorgehen von

entscheidender Bedeutung. Die Rechenleistung kann vollständig für die Spielbaumsuche und die Evaluation genutzt werden, ohne dass unvorhersehbare Pausen durch eine Speicherbereinigung die Effizienz mindern. Trotzdem muss der Speicher aber nicht, wie in anderen Systemsprachen, manuell bereinigt werden [16, S. 77 - 88].

Obwohl die Speichersicherheit besonders in sicherheitskritischen Anwendungen relevant ist, bieten sich auch Vorteile für Programme wie Schach-Engines. Beispielsweise ist das Ownership Prinzip sehr gut darin, Race Conditions im Threading zu verhindern. Auch einfachere Fehler wie Out-of-Bounds reads werden durch Rusts Compiler verhindert. Solche Fehler könnten bei Schach-Engines u.a. dazu führen, dass das Programm während eines Spiels abstürzt. Rust bietet der Engine dadurch eine Stabilitätsgarantie.

### **2.3.3 Branchless Programming**

Im Kontext von rechenintensiver Anwendungen, insbesondere bei der Entwicklung von Chess-Engines, nimmt das Branchless Programming (sprungfreies Programmieren) eine Schlüsselrolle ein. Dabei handelt es sich um eine Programmiertechnik, die darauf abzielt, bedingte Verzweigungen im Kontrollfluss durch mathematische oder bitweise Operationen zu ersetzen. In Rust sind davon vor allem `if` Abfragen oder auch `match` Abfragen betroffen. Das Ziel ist es, den Code so zu gestalten, dass der Prozessor einen linearen Pfad von Instruktionen ausführen kann, ohne an Entscheidungspunkten innehalten zu müssen [18].

Um zu verstehen, warum dieser Ansatz so effektiv ist, muss man die Arbeitsweise moderner Central Processing Units (CPUs) betrachten. Prozessoren nutzen Pipelines, um Befehle in mehreren Bereichen gleichzeitig zu verarbeiten. Während ein Befehl noch berechnet wird, wird der nächste bereits geladen. Trifft der Prozessor auf eine Verzweigung (engl. Branch), muss er vorab raten, welcher Pfad eingeschlagen wird (engl. Branch Prediction), um die Pipeline weiterhin effizient zu füllen [18].

Liegt die CPU mit ihrer Vorhersage richtig, läuft das Programm mit maximaler Geschwindigkeit weiter. Schlägt die Vorhersage jedoch fehl (engl. Branch Misprediction), muss die gesamte Pipeline geleert und der korrekte Pfad neu geladen werden. Dies führt zu Rechenpausen (engl. Stalls), die im Vergleich zu einfachen Rechenoperationen extrem kostspielig sind und die Performanz massiv einbrechen lassen können [18].

## 2.4 Constant Evaluation

Die Constant Evaluation ist ein Feature in Rust, das es ermöglicht, deterministischen Programmcode zur Kompilierzeit anstatt zur Laufzeit zu berechnen. Sie ermöglicht es, komplexe Datenstrukturen vorab zu berechnen, um Rechenlast während der Laufzeit zu sparen. In Rust wird diese Funktionalität durch den Const-Kontext ermöglicht [19]. Listing 3 zeigt, wie eine Lookup-Tabelle für alle möglichen Zielpositionen eines Springers bereits während des Kompilierens berechnet wird. Dort ist es möglich, Hilfsvariablen (z.B. `table`) zu verwenden, welche nicht konstant sind, aber dabei helfen, die finale Konstante zu berechnen. In diesem Fall wird auch `static` anstatt `const` für die Tabelle gewählt, damit diese nur einmal im Programmspeicher existiert. Sie wird also nicht in jeder Funktion einzeln eingefügt (engl. `inlining`), sondern zentral an einem Ort im Speicher abgelegt. Dadurch wird die Cache-Lokalität verbessert [19].

```
1  pub static KNIGHT_TARGETS: [Bitboard; 64] = {  
2      let mut table = [Bitboard(0); 64];  
3      let offsets = [(-2, -1), (-2, 1), (-1, -2), (-1, 2), (1, -2), (1, 2),  
4          (2, -1), (2, 1)];  
5      // [...] Logik zum Füllen der Tabelle  
6      table
```



Listing 3: Anwendung von Constant Evaluation in Rust für eine Lookup-Tabelle

## 2.5 Formale Repräsentation des Spiels

Listing 4 zeigt das Zentrum des gesamten Programms. Hier werden alle benötigten Informationen über den aktuellen Stand des Spiels gespeichert. Die ersten 4 Attribute speichern die aktuelle Position der Figuren. Abschnitt 2.5.1 geht näher auf diesen Teil ein.

Die nächsten 4 Booleans speichern, ob die schwarze oder weiße Seite Castling (dt. Rochade) durchführen darf. Die kleine Rochade wird dabei als „Kingside castling“ und die große Rochade als „Queenside castling“ bezeichnet. Diese sind notwendig, da die Rochade (engl. Castling) im Schach nur zulässig ist, wenn weder der König noch

der jeweilige Turm zuvor bewegt wurden. Da diese Information nicht allein aus der aktuellen Figurenaufstellung abgeleitet werden kann (ein König könnte sich weg- und wieder zurückbewegt haben), muss dieser Zustand explizit in der Spielbrett-Struktur mitgeführt werden [20, S. 6-7].

Das Attribut `ep_target` speichert ein mögliches Zielfeld für einen En-Passant-Schlag. Dieser Zustand lässt sich, genau wie die Castling-Rechte, nicht aus den Positionen der Figuren allein ableiten, da dieser nur nach einem Doppelschritt eines gegnerischen Bauern möglich ist. Deswegen muss auch diese Option explizit gespeichert werden [20, S. 5].

Die verbleibenden Felder verwalten den administrativen Spielzustand:

- `current_color`: Zeigt, welche Seite gerade am Zug ist
- `halfmove_clock`: Zählt die Züge für die 50-Züge-Regel. Diese besagt, dass eine Partie als Remis gilt, wenn in den letzten 50 vollen Zügen (100 Halbzüge (Ply)), weder eine Figur geschlagen, noch ein Bauer gezogen wurde [20, S. 13].
- `total_halfmove_counter`: Zählt die Anzahl der bisher ausgeführten Halbzüge.
- `unmake_info_stack`: Relevant für das Rückgängigmachen von Zügen in der Zuggenerierung. In Abschnitt 4 wird darauf näher eingegangen.
- `repetition_stack`: Dient zu Erkennung von Remis durch dreimalige Stellungswiederholung [20, S. 9].
- `hash`: Stellt einen Zobrist-Hash (s. Abschnitt 2.5.2) für den aktuellen Zustand des Spielbretts bereit, welcher in der Transpositionstabelle (s. Abschnitt 5.6) für Caching verwendet wird.

```

1  pub struct Board {
2      figure_bbs: [Bitboard; 13],
3      color_bbs: [Bitboard; 2],
4      occupied: Bitboard,
5      figures: [Figure; 64],
6
7      black_king_castle: bool,
8      black_queen_castle: bool,
9      white_queen_castle: bool,
10     white_king_castle: bool,
11
12     ep_target: Option<Bit>,
13
14     current_color: Color,
15     halfmove_clock: usize,
16     total_halfmove_counter: usize,
17     unmake_info_stack: Vec<UnmakeInfo>,
18     repetition_stack: Vec<u64>,
19     hash: u64,
20 }

```

 Rust

Listing 4: Spielbrett Struktur in Rust

### 2.5.1 Bitboards

Die wichtigsten Attribute in Listing 4 sind die **Bitboards**. Bei ihnen handelt es sich um eine Datenstruktur im Computerschach, welche das Spielbrett mittels eines einzigen unsigned 64-Bit-Integers (in Rust **u64**) abbildet. Jedes Bit dieser Zahl repräsentiert ein Feld auf dem Schachbrett. Ist dieses Bit auf 1 gesetzt, ist das Feld besetzt und ist das Bit auf 0 gesetzt, ist das Feld leer. Dieser Ansatz wurde 1952 von Mikhail R. Shura-Bura vorgestellt [21]. Abbildung 3 zeigt die Zuordnung zwischen Bitindex des Integers und Position auf dem Schachbrett. Genannt wird diese Zuordnung „Little-Endian Rank-File Mapping“ [22]. Es ist ein glücklicher Zufall, dass ein Schachbrett genau 64 Felder hat und sich dadurch perfekt in einen 64-Bit Integer abbilden lässt. Der Vorteil von Bitboards liegt in der Möglichkeit, bitweise Operationen auf dem Schachbrett durchzuführen. Diese

Operationen sind auf einem Computer stark optimiert, und dadurch extrem schnell [21]. Dennoch haben Bitboards eine große Limitierung: Für jedes Feld lässt sich nur genau eine Information speichern. Allerdings ist ein Feld eines echten Schachspiels nicht binär: Ein Feld kann leer sein, oder durch eine von 6 Figuren in einer von zwei Farben besetzt sein. Deswegen wird in dieser Schach-Engine ein Bitboard pro Figurenart und Farbe verwendet. Insgesamt entstehen so 6 Bitboards für die weißen Figuren (weiße Bauern, weiße Springer, weiße Läufer, weiße Türme, weiße Dame und weißer König) und 6 Bitboards für die schwarzen Figuren. Zusätzlich speichert ein Bitboard alle leeren Felder [21]. So entstehen insgesamt 13 Bitboards, welche als `figure_bbs` in der Spielbrett-Struktur aus Listing 4 gespeichert werden. Das `color_bbs` Array stellt zusätzlich noch ein Bitboard pro Seite bereit, welches alle Felder der jeweiligen Seite gesetzt hat. Dieses wird nach jedem Zug durch die logische Veroderung der 6 Figuren-Bitboards erreicht. Das `occupied` Bitboard wird dann wiederum durch die Veroderung der beiden `color_bbs` erreicht.

Als Beispiel für ein Bitboard lässt sich ein König betrachten. Er befindet sich aktuell auf Position A1 (Index 0 in der Abbildung 3). Dadurch wird er als Integer betrachtet durch die Zahl 1 repräsentiert. Eine Bewegung nach rechts sorgt für eine Verschiebung nach B1 (Index 1 in der Abbildung 3). Diese Bewegung lässt sich durch eine bitweise Linksverschiebung um 1 realisieren. Dadurch wird aus der Zahl 1 die Zahl 2. Analog lässt sich eine Bewegung nach oben durch eine Linksverschiebung um 8 realisieren. Dadurch wird aus der Zahl 1 die Zahl 256. Die Bewegungen nach links und unten lassen sich durch Rechtsverschiebungen, nach dem gleichem Prinzip, realisieren.

|   | a  | b  | c  | d  | e  | f  | g  | h  |   |
|---|----|----|----|----|----|----|----|----|---|
| 8 | 56 | 57 | 58 | 59 | 60 | 61 | 62 | 63 | 8 |
| 7 | 48 | 49 | 50 | 51 | 52 | 53 | 54 | 55 | 7 |
| 6 | 40 | 41 | 42 | 43 | 44 | 45 | 46 | 47 | 6 |
| 5 | 32 | 33 | 34 | 35 | 36 | 37 | 38 | 39 | 5 |
| 4 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 | 4 |
| 3 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 3 |
| 2 | 8  | 9  | 10 | 11 | 12 | 13 | 14 | 15 | 2 |
| 1 | 0  | 1  | 2  | 3  | 4  | 5  | 6  | 7  | 1 |
|   | a  | b  | c  | d  | e  | f  | g  | h  |   |

Abbildung 3: Indizierung des Bitboards

In diesem Projekt wird das Newtype-Pattern von Rust benutzt, um dem Bitboard seinen eigenen Datentypen zu geben. Diese Abstraktion dient der Typsicherheit während der Kompilierung. Nach der Übersetzung in Maschinencode wird nach dem Prinzip der Zero-Cost Abstraction jedoch weiterhin mit einem `u64` Integer gearbeitet, wodurch die Performanz zur Laufzeit nicht beeinträchtigt wird [16, S. 587]. Diese Behauptung wird in Listing 5 bewiesen. Auf der linken Seite ist der Rust-Quellcode sichtbar. Zuerst wird eine `pub struct BB(pub u64)` Struktur erstellt. Diese Zeile ist für die Erstellung des Newtypes verantwortlich, welcher intern als `u64` repräsentiert wird. Für diesen neuen Typen wird dann mithilfe von `impl std::ops::BitAnd` die Rechenoperation für das bitweise UND implementiert. Dadurch kann man nun diese Operation auf die zwei Bitboards (`a` und `b`) in der `main()` Funktion anwenden. Die Blackbox-Funktion wird hierbei benutzt, damit der Compiler dieses triviale Beispielprogramm nicht schon während des Kompilierens berechnet. Auf der rechten Seite ist dann das entsprechende `x86_64` Assembly sichtbar. Dort wurde die Struktur Bitboard vollständig weg optimiert, und es werden nur noch die beiden Bitboards in die Register geladen und das Ergebnis mithilfe der Instruktion `and` berechnet. Dieses Beispiel zeigt die Stärken des Newtype-Pattern, welches Abstraktion während der Entwicklung mit maximaler Performanz zur Laufzeit ermöglicht [16, S. 587].

|                                                                                                                                                                                                                                                                                                                                                                                                      |                                                                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> 1  use std::hint::black_box; 2  pub struct BB(pub u64); 3  impl std::ops::BitAnd for BB { 4      type Output = Self; 5      fn bitand(self, rhs: Self) -&gt; 6          Self { 7          BB(self.0 &amp; rhs.0) 8      } 9  } 10 fn main() { 11     let a = black_box(BB(0b0001)); 12     let b = black_box(BB(0b0011)); 13     let result = a &amp; b; 14     black_box(result); 15 } </pre> | <pre> 1 2  mov qword ptr [rsp - 8], 1 3  lea rax, [rsp - 8] 4  mov rax, qword ptr [rsp - 8] 5  mov qword ptr [rsp - 8], 3 6  lea rcx, [rsp - 8] 7  and rax, qword ptr [rsp - 8] 8  mov qword ptr [rsp - 8], rax 9  lea rax, [rsp - 8] 10 ret </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Listing 5: Vergleich zwischen Rust und Assembly Quellcode bei der Verwendung des Newtype-Patterns

### 2.5.2 Zobrist Hashing

Die Transpositionstabelle (Abschnitt 5.6) speichert Informationen über bereits besuchte Stellungen, um wiederholte Berechnungen zu vermeiden. Um eine Stellung in der Tabelle nachzuschlagen, wird jedoch ein eindeutiger Schlüssel benötigt, der den gesamten Brettzustand repräsentiert. Diesen Schlüssel liefert der Zobrist-Hash, ein Verfahren, das 1970 von Albert Zobrist für zweidimensionale Spiele vorgeschlagen wurde Abschnitt 2.5.2.

Das grundlegende Prinzip des Zobrist-Hashings beruht auf dem inkrementellen Aufbau eines 64-Bit-Hashwerts durch XOR-Verknüpfung. Jedem elementaren Zustand des Spielbretts wird dabei ein zufälliger 64-Bit-Wert zugewiesen. Der Hash einer Stellung ergibt sich aus der XOR-Summe aller auf dem Brett befindlichen Figuren mit ihren Feldern. Der entscheidende Vorteil der XOR-Verknüpfung ist ihre Selbstinvertierbarkeit: Führt man eine XOR-Operation zweimal mit demselben Wert aus, hebt sie sich auf. Dadurch kann der Hash beim Ausführen eines Zuges inkrementell aktualisiert werden,

ohne die gesamte Stellung neu berechnen zu müssen. Entfernt man eine Figur von ihrem Ursprungsfeld und setzt sie auf ein Zielfeld, genügen zwei XOR-Operationen Abschnitt 2.5.2:

$$H_{\text{neu}} = H_{\text{alt}} \oplus R(\text{Figur, von}) \oplus R(\text{Figur, nach})$$

Neben den Figurenpositionen müssen auch weitere Zustandsinformationen in den Hash einfließen, da beispielsweise zwei Stellungen mit identischer Figurenaufstellung, aber unterschiedlichen Rochaderechten oder En-Passant-Möglichkeiten nicht als identisch gelten. Die deterministische Erzeugung dieser Hashes basiert in Thunfisch auf den Polyglot-Schlüsseln [23], einem standardisierten Satz aus 781 vorberechneten 64-Bit-Zufallszahlen. Der Gesamt-Hash setzt sich aus vier Komponenten zusammen, die über feste Indizes adressiert werden:

- **Figurenpositionen** (768 Schlüssel): Jede der 12 Figurentypen besitzt für jedes der 64 Felder einen eindeutigen Schlüssel ( $12 \cdot 64$ ).
- **Rochaderechte** (4 Schlüssel): Repräsentieren separat die verbleibenden Rochademöglichkeiten (getrennt nach König-/Damenflügel und Weiß/Schwarz).
- **En-Passant-Zielfeld** (8 Schlüssel): Kodieren die jeweilige Spalte, auf der im nächsten Halbzug ein En-Passant-Schlag möglich ist.
- **Seite am Zug** (1 Schlüssel): Ein fester Schlüssel, der den Zustand „Weiß am Zug“ repräsentiert.

Anstatt bei jedem Zug das gesamte Schachbrett vollständig neu zu berechnen, nutzt die Engine die mathematische Eigenschaft der bitweisen XOR-Verknüpfung. Aufgrund der selbstinvertierenden Eigenschaft der Operation kann der Hash-Wert während des Spiels effizient inkrementell aktualisiert werden.

Zusätzlich sind Kollisionen ein inhärentes Problem bei der Verwendung von Zobrist-Hashing, da der verwendete 64-Bit-Hash weitaus weniger Zustände kodieren kann, als es potenziell erreichbare Schachstellungen gibt. Die Wahrscheinlichkeit einer Kollision folgt hierbei dem Geburtstagsparadoxon: Bei einem  $n$ -Bit-Hash ist die erste Kollision nach etwa  $\sqrt{2^n}$  Stellungen zu erwarten. Für einen 32-Bit-Hash tritt eine Kollision bereits nach etwa  $\sqrt{2^{32}} = 65536$  Stellungen auf. Das ist für eine Schach-Engine, die während einer Suche Millionen von Stellungen besucht, ein nicht hinnehmbares Risiko. Mit einem 64-Bit-Hash hingegen ist eine Kollision erst nach etwa  $\sqrt{2^{64}} \approx 4,3 \cdot 10^9$  Stellungen

zu erwarten. Hyatt und Cozzie untersuchten in [24] die Auswirkungen solcher Hash-Kollisionen auf die Spielstärke einer Engine und kamen zu dem Schluss, dass 64 Bit ausreichend sind. Deswegen nutzt auch Thunfisch einen 64-Bit Hash.

### 3 Systemarchitektur der Schach-Engine

Die Implementierung einer Schach-Engine erfordert ein effizientes Zusammenspiel verschiedener Systemkomponenten. Deswegen ist teilt sich die Engine Thunfisch in viele Rust-Module auf. Abbildung 4 zeigt die Architektur der Engine auf Komponentenebene. Die folgenden Kapitel sind nach dem Ablauf einer Suche strukturiert: Zuggenerierung (Abschnitt 4), Suchalgorithmen (Abschnitt 5) und Positionsevaluation (Abschnitt 6).

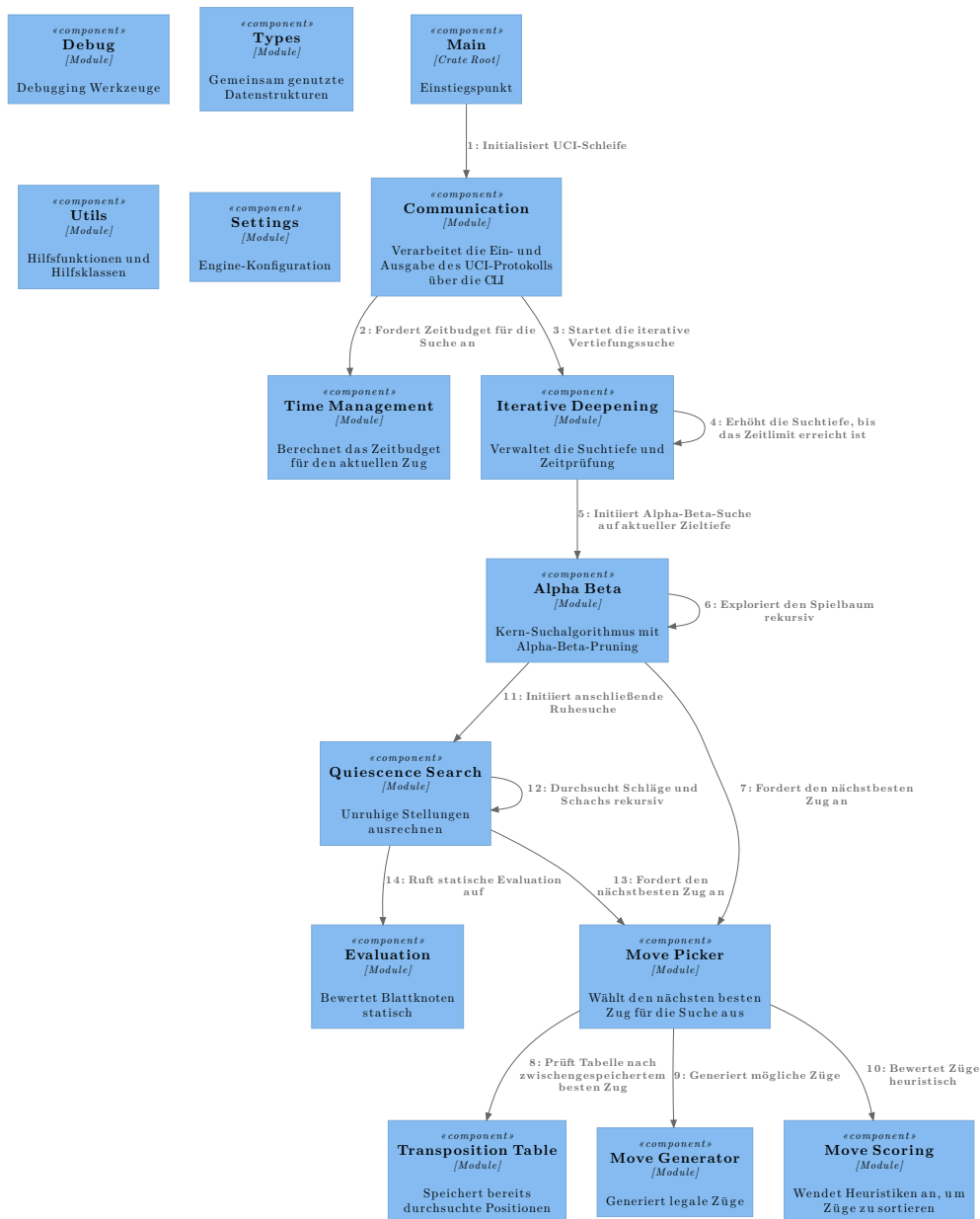


Abbildung 4: Systemarchitektur der Schach-Engine inklusive Datenfluss zwischen den Komponenten

### 3.1 Komponenten

Die Engine gliedert sich in eine Reihe spezialisierter Komponenten (Abbildung 4), die im Quellcode jeweils einem eigenen Rust-Modul entsprechen. Ziel dieser Aufteilung war es, die Trennung der Komponenten im Code und in der Dokumentation konsistent zu halten. Der vollständige Quellcode der Schach-Engine ist öffentlich auf GitHub einsehbar<sup>2</sup>.

- **Main** ist der Einstiegspunkt der Anwendung. Hier wird die Engine initialisiert, und die Kommunikationsschnittstelle wird gestartet.
- **Communication** (Abschnitt 2.1.3) verarbeitet die Ein- und Ausgabe des UCI-Protokolls über die CLI. Empfängt Befehle wie `position` und `go` und gibt Suchergebnisse zurück.
- **Time Management** berechnet das Zeitbudget für den aktuellen Zug, falls keine feste Suchzeit vorgegeben ist. In der aktuellen Version ist die Implementierung relativ einfach und statisch gehalten, weswegen sie kein eigenes Kapitel erhält.
- **Iterative Deepening** (Abschnitt 5.5) steuert die äußere Schleife der Suche. Erhöht schrittweise die Suchtiefe, solange noch Zeit zur Verfügung steht.
- **Alpha-Beta** (Abschnitt 5.2) implementiert den Kern der Baumsuche mit Alpha-Beta-Pruning. Durchläuft rekursiv den Spielbaum, um den besten Zug zu ermitteln.
- **Quiescence Search** (Abschnitt 5.3) führt nach Erreichen der Ziel-Tiefe eine spezialisierte Suche auf taktischen Zügen durch, bis eine ruhige Stellung erreicht ist.
- **Evaluation** (Abschnitt 6) führt die statische Bewertung einer Brettstellung durch und berechnet den numerischen Vorteil einer Seite.
- **Move Picker** (Abschnitt 5.7.4) steuert die on-demand Auswahl des nächsten besten Zuges während der Suche.
- **Transposition Table** (Abschnitt 5.6) ist ein Cache für bereits besuchte Stellungen. Liefert auf Anfrage den zwischengespeicherten besten Zug oder eine Bewertung.
- **Move Generator** (Abschnitt 4) generiert alle legalen Züge für eine gegebene Stellung.
- **Move Scoring** (Abschnitt 5.7) wendet Heuristiken wie MVV-LVA, Killer-Moves und History-Heuristik an, um die generierten Züge zu sortieren.

---

<sup>2</sup><https://github.com/Kaynoux/thunfisch> (11.06.2026)

- **Types** definiert die gemeinsam genutzten Datenstrukturen wie Bitboards (Abschnitt 2.5.1) und das Spielbrett (Listing 4).
- **Utils** stellt allgemeine Hilfsfunktionen und Hilfsklassen bereit.
- **Settings** verwaltet die Engine-Konfiguration und einstellbare Optionen.
- **Debug** stellt Debugging-Werkzeuge und benutzerdefinierte Befehle bereit.

## 3.2 Ablauf der Suche

Das Diagramm in Abbildung 4 stellt den Datenfluss der Engine stark abstrahiert dar. In der Realität handelt es sich bei der Alpha-Beta-Suche um einen rekursiven Algorithmus; das Diagramm bildet daher nicht den tatsächlichen Kontrollfluss ab, sondern zeigt die logische Reihenfolge der Interaktionen auf einer einzelnen Ebene. Die Informationen werden jeweils über die Rekursion wieder nach oben gereicht.

Schritt 1 bis 3 bereiten die Suche vor. **Main** initialisiert die **Communication**-Komponente (1), die daraufhin das Zeitbudget vom **Time Management** anfordert (2). Anschließend startet **Communication** das **Iterative Deepening** (3). Das **Iterative Deepening** erhöht die Suchtiefe schrittweise, bis das Zeitlimit erreicht ist (4). Zu jeder Tiefe wird die **Alpha-Beta**-Suche gestartet (5). Diese durchläuft den Spielbaum rekursiv (6) und fordert für jeden Knoten den nächsten Zug vom **Move Picker** an (7).

Der **Move Picker** wiederum bedient sich dreier Basiskomponenten: Zuerst wird die **Transposition Table** nach einem zwischengespeicherten besten Zug befragt (8). Falls kein Treffer vorliegt, generiert der **Move Generator** alle legalen Züge (9), die anschließend durch das **Move Scoring** heuristisch sortiert werden (10). Wird in der **Alpha-Beta**-Suche die konfigurierte Tiefe erreicht, initiiert sie die **Quiescence Search** (11). Diese durchsucht rekursiv schlagende Züge und Schachzüge, bis keine unruhigen Züge mehr vorhanden sind (12). Auch die **Quiescence Search** fordert Züge über den **Move Picker** an (13) und lässt ruhige Stellungen abschließend von der **Evaluation** bewerten (14).

Die auf diese Weise ermittelten Bewertungen werden rekursiv durch den Spielbaum nach oben gereicht. Das **Iterative Deepening** bricht ab, sobald das Zeitbudget aus Schritt 2 erschöpft ist. Abschließend wird der beste Zug aus der tiefsten vollständig durchgeführten Iteration zurück an die Kommunikationsschnittstelle gegeben und über das UCI-Protokoll ausgegeben.

## 4 Zuggenerierung

Die Zuggenerierung gehört zu den zentralen Komponenten einer Schach-Engine. Ihre Aufgabe besteht darin, für eine gegebene Stellung alle legalen Züge der am Zug befindlichen Seite zu bestimmen. Da diese Operation im Verlauf der Suche sehr häufig ausgeführt wird, hat ihre Effizienz einen unmittelbaren Einfluss auf die gesamte Spielstärke der Engine. Bereits kleine Verbesserungen in der Zuggenerierung können dazu führen, dass in derselben Zeit deutlich mehr Stellungen untersucht werden.

Grundsätzlich lässt sich zwischen pseudo-legalen und legalen Zügen unterscheiden. Pseudo-legale Züge erfüllen zunächst nur die allgemeinen Bewegungsregeln der jeweiligen Figur. Sie ignorieren jedoch Randbedingungen wie Schach, Fesselungen oder Sonderfälle, bei denen ein Zug den eigenen König ungeschützt zurücklassen würde. Legale Züge sind dagegen genau diejenigen Züge, die nach allen Schachregeln tatsächlich ausgeführt werden dürfen.

Ein naheliegender Ansatz besteht darin, zunächst alle pseudo-legalen Züge zu erzeugen und anschließend jeden einzelnen Zug darauf zu prüfen, ob der eigene König nach seiner Ausführung im Schach steht. In diesem Fall wäre der Zug illegal und müsste verworfen werden. Dieser Ansatz ist konzeptionell einfach, verursacht jedoch zusätzlichen Rechenaufwand, weil für viele Kandidaten ein vollständiger Prüfprozess erforderlich ist.

In dieser Arbeit wird daher ein anderer Ansatz verwendet. Der entwickelte Zuggenerator erzeugt möglichst direkt nur legale Züge. Regeln wie Schach, Fesselungen, Königssicherheit sowie Sonderfälle wie Rochade und En Passant werden bereits während der Generierung berücksichtigt. Dadurch steigt zwar die Komplexität der Implementierung, gleichzeitig lässt sich die Anzahl unnötiger Prüfungen jedoch deutlich reduzieren. Gerade in einer Schach-Engine, in der die Zuggenerierung millionenfach aufgerufen wird, überwiegt dieser Leistungsvorteil klar.

Dieses Kapitel beschreibt den Aufbau dieses Zuggenerators im Detail. Die Implementierung basiert auf Bitboards, sowie auf vorberechneten Zieltabellen und verschiedenen Masken, mit denen die Menge potenzieller Zielpositionen schrittweise auf die tatsächlich legalen Züge reduziert wird.

## 4.1 Ablauf

Die Zuggenerierung erzeugt in dieser Engine nicht einfach alle Züge in einem einzigen Schritt, sondern ist in zwei Phasen aufgeteilt. Während der Suche werden zunächst nur schlagende Züge, und danach nur ruhige Züge erzeugt. Dazu wird die Funktion `generate_moves<const QUIET>` mit einem konstanten Parameter aufgerufen. Ist der Boolean `QUIET` falsch, werden nur schlagende Züge und En-Passant-Züge generiert. Ist `QUIET` wahr, werden nur ruhige Züge einschließlich Rochaden erzeugt. Diese Trennung erlaubt es, zuerst schlagende Züge zu behandeln und ruhige Züge nur dann zu berechnen, wenn der aktuelle Pfad der Suche nicht schon während der schlagende Züge verworfen wurde. Dieses Verhalten wird vom Move Picker (s. Abschnitt 5.7.4) gesteuert.

Zu Beginn der Zuggenerierung werden mehrere globale Informationen über die aktuelle Stellung bestimmt. Dazu gehören die horizontal-vertikale Fesselmaske, die diagonale Fesselmaske, die Anzahl der Schachgebote gegen den eigenen König, sowie die Schachmaske. Diese Werte werden auf dem Spielbrett zwischengespeichert und bei Bedarf nur einmal berechnet. Anschließend stehen sie allen Figurenarten zur Verfügung, sodass die eigentliche Zugerzeugung ohne wiederholte globale Analysen auskommt.

Ein Sonderfall ist das Doppelschach. Befindet sich der König gleichzeitig unter Angriff von zwei gegnerischen Figuren, sind ausschließlich Königszüge legal. In diesem Fall bricht der Algorithmus die normale Figureniteration sofort ab und erzeugt direkt nur die möglichen Königszüge. Alle anderen Figuren können ein Doppelschach per Definition weder durch Schlagen, noch durch Dazwischenziehen auflösen.

Liegt kein Doppelschach vor, werden die Figuren in fester Reihenfolge verarbeitet. Zuerst werden Bauernzüge erzeugt, danach Springer-, Läufer-, Turm- und Damenzüge. Anschließend folgen die normalen Königszüge. Rochaden und En Passant werden nicht gemeinsam mit den übrigen Figuren behandelt, sondern als eigene Sonderfälle am Ende ergänzt. Rochaden erscheinen nur in der ruhigen Phase, En-Passant-Züge nur in der Schlagphase.

Für jede Figurenart werden zunächst nur solche Zielpositionen betrachtet, die grundsätzlich zur Bewegungsart der Figur passen. Danach werden diese Ziele schrittweise durch Bitmasken eingeschränkt. Die Schachmaske stellt sicher, dass bei

einem einfachen Schach nur Züge erlaubt bleiben, die das Schach auflösen. Die Fesselmasken sorgen dafür, dass gefesselte Figuren ihre Fessellinie nicht verlassen dürfen. Dadurch erzeugt der Algorithmus nicht erst pseudo-legale Züge, die nachträglich überprüft werden müssten, sondern reduziert die möglichen Ziele bereits während der Generierung direkt auf legale Züge.

Der König bildet eine Ausnahme von diesem allgemeinen Schema. Seine Zielpositionen werden nicht nur gegen freie oder gegnerische Felder geprüft, sondern zusätzlich gegen die gegnerische Angriffsmaske. Dafür wird die Angriffsmaske auf einer Stellung berechnet, in der der eigene König vorübergehend vom Brett entfernt wird. Nur so lässt sich korrekt feststellen, welche Felder der König tatsächlich betreten darf. Auch die Rochade verwendet diese Angriffsmaske, da der König weder aus dem Schach heraus noch durch angegriffene Felder rochieren darf.

Insgesamt basiert der Ablauf damit auf einer Kombination aus vorberechneten Zieltabellen, globalen Stellungsinformationen und maskenbasierten Filteroperationen. Der entscheidende Vorteil dieses Ansatzes besteht darin, dass diese Schachregeln nicht durch nachträgliches Ausführen und Verwerfen von Zügen geprüft werden müssen, sondern bereits beim Erzeugen der Züge in Form von Bitoperationen berücksichtigt wird.

## **4.2 Berechnung der Schachmaske**

Die Schachmaske beschreibt alle Zielfelder, auf die eine eigene Figur trotz einer vorliegenden Schachsituation noch ziehen darf. Befindet sich der König nicht im Schach, ist diese Maske vollständig gesetzt und schränkt die Zuggenerierung nicht weiter ein. Liegt dagegen ein einfaches Schach vor, dürfen nur noch solche Züge erzeugt werden, die das Schach unmittelbar aufheben. Dazu gehören entweder das Schlagen der angreifenden Figur oder, falls es sich um eine gegnerische Gleitfigur handelt, das Besetzen eines Feldes auf der Angriffslinie zwischen Angreifer und König. Bei einem Doppelschach existieren keine solchen Blockade- oder Schlagmöglichkeiten mehr. In diesem Fall sind ausschließlich Königszüge legal.

Die Berechnung dieser Maske erfolgt in der Implementierung effizient direkt aus der Königsperspektive. Anstatt für jede eigene Figur separat zu prüfen, ob ihr Zug ein bestehendes Schach aufhebt, wird zunächst nur die aktuelle Königsposition betrachtet.

Von dort aus wird bestimmt, welche gegnerischen Figuren den König unmittelbar angreifen können. Dafür wird die gleiche Logik wie für die Berechnung der pseudo-legalen, schlagenden Züge der Figuren benutzt. Der einzige Unterschied liegt darin, dass die Perspektive umgekehrt wird: Der Algorithmus behandelt den eigenen König, als wäre dieser die jeweilige gegnerische Figur. Um beispielsweise festzustellen, ob der König von einem gegnerischen Springer angegriffen wird, generiert man von der aktuellen Königsposition aus die pseudo-legalen Ziele eines Springers.

Dadurch lässt sich mit wenigen bitweisen Operationen feststellen, ob sich auf einem dieser Felder tatsächlich ein gegnerischer Bauer oder Springer befindet. Für gegnerische Läufer, Türme und Damen werden die möglichen Angriffe des Königs in diagonal- beziehungsweise horizontal-vertikaler Richtung berechnet und anschließend mit den gegnerischen Gleitfiguren verundet. So erhält man alle gegnerischen Schachgeber, ohne selbst Züge simulieren zu müssen.

Ist kein Angreifer vorhanden, gibt die Funktion sofort eine vollständig gesetzte Maske zurück. Liegt dagegen ein Schach vor, wird die Maske schrittweise aufgebaut. Bei Springer- und Bauernschach genügt es, das Feld des Angreifers in die Maske aufzunehmen, da ein solches Schach nicht blockiert werden kann. Bei einem Schach durch eine Gleitfigur wird zusätzlich das Bitboard der Felder zwischen König und Angreifer berücksichtigt. Dazu verwendet die Implementierung eine vorberechnete In-Between-Tabelle, die für jedes Feldpaar auf dem Brett direkt alle dazwischenliegenden Felder liefert. Das vermeidet jede schrittweise Traversierung entlang der Linie zur Laufzeit. Die endgültige Schachmaske besteht damit aus der Vereinigung des Angreiferfeldes und aller Zwischenfelder.

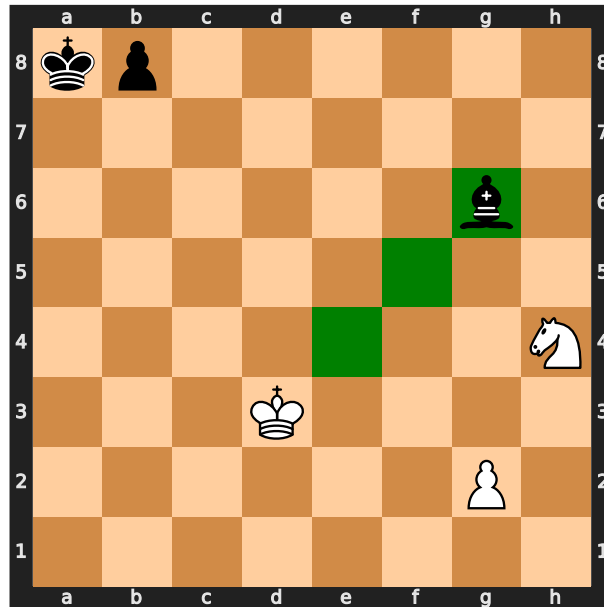


Abbildung 5: Visualisierung einer Schachmaske - gesetzte Felder sind grün markiert

Abbildung 5 zeigt, wie eine Schachmaske für eine bestimmte Spielsituation aussieht. Weiß ist am Zug und der Läufer auf g6 stellt den König ins Schach. Deswegen muss Weiß eine seiner Figuren auf eines der grünen Felder (die gesetzten Felder der Schachmaske) setzen, um das Schach entweder durch Schlagen oder Blockieren des Läufers aufzulösen.

Dieser Ansatz ist deutlich effizienter, als eine nachträgliche Legalitätsprüfung jedes einzelnen Zuges durchzuführen. Würde man zunächst alle pseudo-legalen Züge erzeugen und danach jeden Kandidaten testweise ausführen, müsste für jeden Zug erneut geprüft werden, ob der eigene König im Schach steht. Im vorliegenden Generator wird diese Information stattdessen einmal pro Stellung als Bitmaske berechnet und anschließend von allen Figuren gemeinsam verwendet. Die Kosten der Schachbehandlung werden somit von der Anzahl der möglichen Züge entkoppelt und auf wenige Bitboard-Operationen reduziert.

Zusätzlich wird die Schachmaske im Brettobjekt zwischengespeichert. Sie wird erst dann berechnet, wenn sie tatsächlich benötigt wird, und nach einer Zugausführung wieder ungültig gemacht. Dadurch entstehen innerhalb derselben Stellung keine mehrfachen Neuberechnungen.

### 4.3 Berechnung der Fesselmaske

Die Fesselmaske beschreibt, welche eigenen Figuren absolut gefesselt sind und auf welcher Linie sie sich noch legal bewegen dürfen. Eine absolute Fesselung liegt vor, wenn sich zwischen dem eigenen König und einer gegnerischen Gleitfigur genau eine eigene Figur befindet. Würde diese Figur die Verbindungslinie verlassen, wäre der eigene König unmittelbar im Schach. Für die Zuggenerierung bedeutet dies, dass eine gefesselte Figur nicht frei nach ihren normalen Bewegungsregeln ziehen darf, sondern nur noch entlang der Fessellinie. Im Generator werden dafür zwei getrennte Masken verwendet: eine horizontal-vertikale Fesselmaske für Türme und Damen, sowie eine diagonale Fesselmaske für Läufer und Damen.

Auch diese Masken werden effizient vom König aus berechnet. Die Implementierung untersucht nicht jede eigene Figur einzeln darauf, ob sie eventuell gefesselt ist. Stattdessen wird zunächst von der Position des eigenen Königs aus bestimmt, welche gegnerischen Gleitfiguren den König in einer sogenannten X-Ray-Betrachtung sehen könnten. Dazu werden zunächst die normalen Turm- beziehungsweise Läuferziele des Königs unter aktueller Belegung berechnet. Anschließend werden genau die auf diesen Linien liegenden eigenen Figuren als transparente Blocker behandelt und dieselbe Zielberechnung erneut durchgeführt. Die XOR-Verknüpfung beider Ergebnisse liefert genau diejenigen Felder, die erst dann sichtbar werden, wenn die eigenen Blocker entlang der Linie entfernt werden. Schneidet man diese X-Ray-Ziele mit den gegnerischen Türmen und Damen beziehungsweise Läufern und Damen, bleiben genau die gegnerischen Gleitfiguren übrig, die potenzielle absolute Fesselungen verursachen.

Für jede so gefundene gegnerische Gleitfigur wird anschließend wieder die vorberechnete In-Between-Tabelle verwendet. Sie liefert alle Felder zwischen dem gegnerischen Angreifer und dem eigenen König. Diese Felder werden gemeinsam mit dem Feld des Angreifers in die passende Fesselmaske eingetragen. Auf diese Weise enthält die resultierende Maske genau die jeweilige Fessellinie. Eine gefesselte Figur darf deshalb nur auf Zielfelder ziehen, die zugleich auf ihrer Bewegungsbahn und auf dieser Maske liegen. Ein gefesselter Turm darf also nur auf der horizontalen oder vertikalen Linie bleiben, ein gefesselter Läufer nur auf der Diagonale. Für die Dame entscheidet die Art der Fesselung, welcher Teil ihrer Bewegungsmenge erhalten bleibt. Springer sind ein

Sonderfall: Da sie keine Linienzüge ausführen, können sie eine Fessellinie nicht einhalten und werden bei bestehender Fesselung unmittelbar ausgeschlossen.

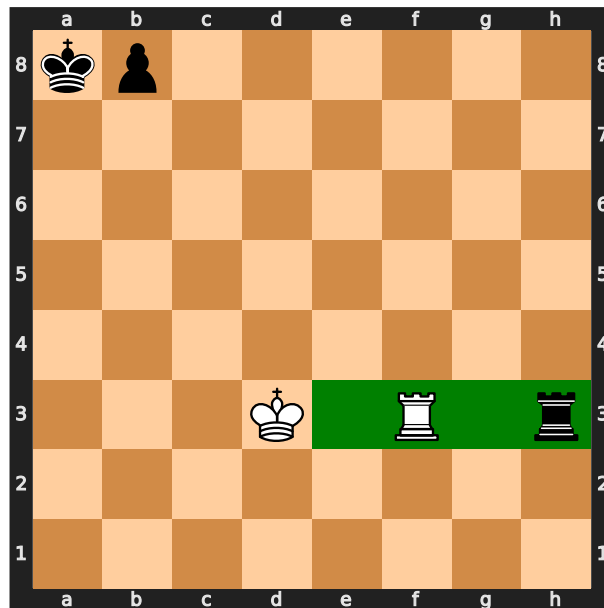


Abbildung 6: Visualisierung einer Fesselmaske - gesetzte Felder sind grün markiert

Abbildung 6 zeigt eine Beispielsituation. Durch den schwarzen Turm auf h3 ist der weiße Turm auf f3 gefesselt und darf sich nur auf grün markierten Feldern zwischen dem weißen König und einschließlich dem gegnerischen Turm bewegen.

Die Effizienz dieses Verfahrens liegt vor allem darin, dass Fesselungen nicht durch Simulation möglicher Züge erkannt werden. Stattdessen werden sie einmal pro Stellung global aus der Königsperspektive bestimmt. Die Zahl der betrachteten Kandidaten wird dabei stark reduziert, weil nur gegnerische Gleitfiguren berücksichtigt werden, die über X-Ray-Angriffe überhaupt in Frage kommen. Zusammen mit den vorberechneten In-Between-Tabellen und den Magic-Bitboard-Angriffsfunktionen entsteht eine sehr kompakte Berechnung, die im Wesentlichen aus wenigen Bitboard-Operationen und Iterationen über tatsächlich relevante Angreifer besteht.

Wie die Schachmaske werden auch die beiden Fesselmasken im Brettobjekt zwischengespeichert und erst bei Bedarf berechnet. Nach jedem ausgeführten Zug werden diese zwischengespeicherten Werte ungültig gemacht, sodass die nächste Stellung ihre Masken erneut korrekt bestimmen kann.

## 4.4 Zuggenerierung für statische Figuren

Unter statischen Figuren versteht man diejenigen Figuren, deren Zielfelder ausschließlich von ihrem Ausgangsfeld, und nicht durch Figuren auf Zwischenfeldern beeinflusst werden kann. Konkret zählen hierzu Bauern, Springer, und Könige. Die Zuggenerierung statischer Figuren lässt sich exemplarisch am Springer erläutern. Bauern und König besitzen zusätzliche Sonderregeln, werden deshalb in der Implementierung separat behandelt.

Zuerst wird das passende Bitboard aus der Board-Struktur geladen, welches die aktuellen Springerpositionen der aktuellen Seite enthält. Hier wird beispielsweise die weiße Seite betrachtet.

```
1  pub fn iter_mut(&mut self) -> impl Iterator<Item = Bit> + '_ {  
2      std::iter::from_fn(move || self.pop_lsb_position())  
3  }  
4  pub fn pop_lsb_position(&mut self) -> Option<Bit> {  
5      if *self == Bitboard(0) {  
6          None  
7      } else {  
8          let pos = Bit::from(*self & self.0.wrapping_neg());  
9          *self &= *self - Bitboard(1);  
10         Some(pos)  
11     }  
12 }
```

Listing 6: Implementierung eines effizienten Iterators für ein Bitboard

Über das Bitboard muss iteriert werden, um für jeden Springer seine möglichen Ziele zu berechnen. Dafür wird die Funktion aus Listing 6 benutzt. Diese wird für den Bitboard-Typ implementiert, deswegen handelt es sich bei `self` um das aktuelle Bitboard. Sie stellt eine effiziente Methode bereit, um über alle gesetzten Bits eines Bitboards zu iterieren. Im Zentrum steht die Funktion `pop_lsb_position`: Diese isoliert und entfernt das Least Significant Bit (LSB). Dadurch wird immer der Springer mit dem niedrigsten Index zurückgegeben. Zur Isolierung des Bits wird die Operation

`self.0 & self.0.wrapping_neg()` eingesetzt. Sie macht sich die Eigenschaften des Zweierkomplements zunutze: Da die Negation einer Zahl alle Bits invertiert und anschließend den Wert eins addiert, bleibt bei einer bitweisen UND-Verknüpfung mit dem ursprünglichen Wert exakt nur das rechteste gesetzte Bit als Information erhalten [25].

Unmittelbar nach der Isolierung wird das Bit durch die Operation `self.0 &= self.0 - 1` aus dem Bitboard gelöscht. Dieser als Teil des „Kernighan-Algorithmus“ bekannte Schritt ist hochgradig effizient, da er ohne konditionale Sprünge direkt auf den CPU-Registern operiert. Im Vergleich zu einer naiven Iteration über alle 64 Felder des Schachbretts reduziert dieser Ansatz die Zeitkomplexität von  $\mathcal{O}(64)$  auf  $\mathcal{O}(k)$ , wobei  $k$  der Anzahl der tatsächlich auf dem Brett vorhandenen Springer entspricht [25].

Die Funktion `iter_mut` implementiert diese Logik dann in das Typsystem von Rust. Dadurch wird das Bitboard in einen Lazy-Iterator umgewandelt. Dieser ruft bei jeder Iterationsstufe die `pop_lsb_position` Funktion auf, bis das Bitboard leer ist und der Iterator terminiert. Diese Architektur ermöglicht es, die Positionen der Figuren speichereffizient und schnell zu durchlaufen. In der Praxis führt dies zu einer signifikanten Leistungssteigerung, da die Engine pro Sekunde Millionen solcher Iterationszyklen durchläuft [25].

In dem Beispiel hat die weiße Seite nur einen Springer auf Position h4. Das Bitboard hat also nur ein Bit gesetzt an Index 31 (s. Abbildung 3). Dieses Bit wird durch den Iterator extrahiert, und anschließend werden alle möglichen Ziele dieses Springers berechnet.

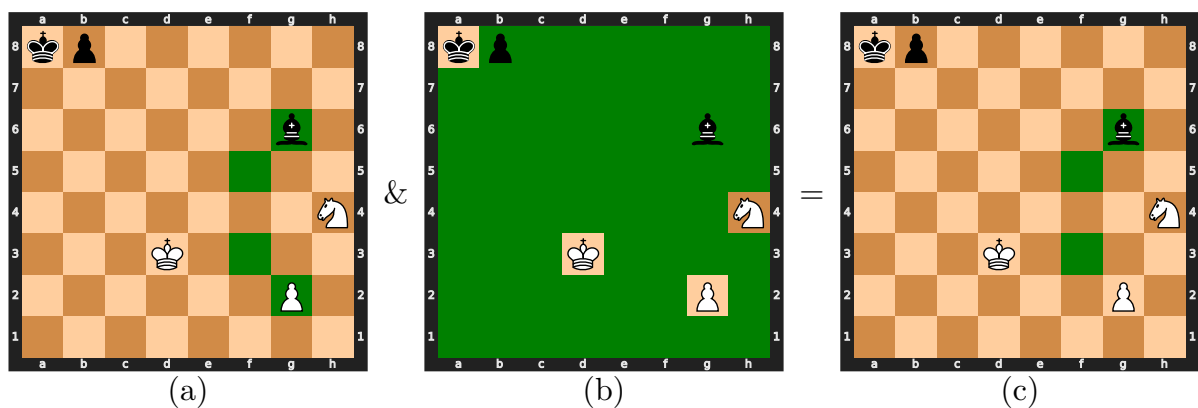


Abbildung 7: Reduktion der potenziellen Ziele eines Springers (a) auf pseudo-legale Ziele (c) mithilfe der Maske aus freien und gegnerischen Feldern (b)

Zuerst werden alle potenziellen Zielfelder des Springers aus einer vorberechneten Lookup-Tabelle geladen. Für jedes der 64 Felder speichert diese Tabelle ein Bitboard aller erreichbaren Zielfelder unter der Annahme, dass keine weiteren Figuren auf dem Brett stehen. Die Tabelle wird als statisches Array zur Kompilierungszeit berechnet (s. Abschnitt 2.4). Dadurch kann die Zielmenge in konstanter Zeit bestimmt werden. Diese Ziele sind in Abbildung 7 (a) sichtbar. Es fällt auf, dass manche dieser Positionen (in diesem Fall der Zug nach g2) im Schach jedoch nicht erlaubt sind. Diese illegalen Ziele müssen als nächstes herausgefiltert werden.

Dafür wird das Bitboard mit einem weiteren Bitboard mit der UND-Operation verknüpft. Es handelt sich bei dem 2. Bitboard um alle leeren Felder und alle gegnerischen Felder (s. Abbildung 7 (b)). Der gegnerische König zählt nicht mit, da ein Zug, der den gegnerischen König schlagen würde, in einer legalen Stellung niemals vorkommen kann.

Durch die UND-Verknüpfung mit der Maske aus leeren und gegnerisch besetzten Feldern werden alle Ziele entfernt, die durch eigene Figuren blockiert sind. Das Resultat ist eine vorläufige zulässige Zielmenge des Springers, die jedoch im aktuellen Generator zusätzlich noch mit Schach- und Fesselregeln eingeschränkt werden muss. Vorerst handelt es sich also um pseudo-legale Ziele des Springers (s. Abbildung 7 (c)).

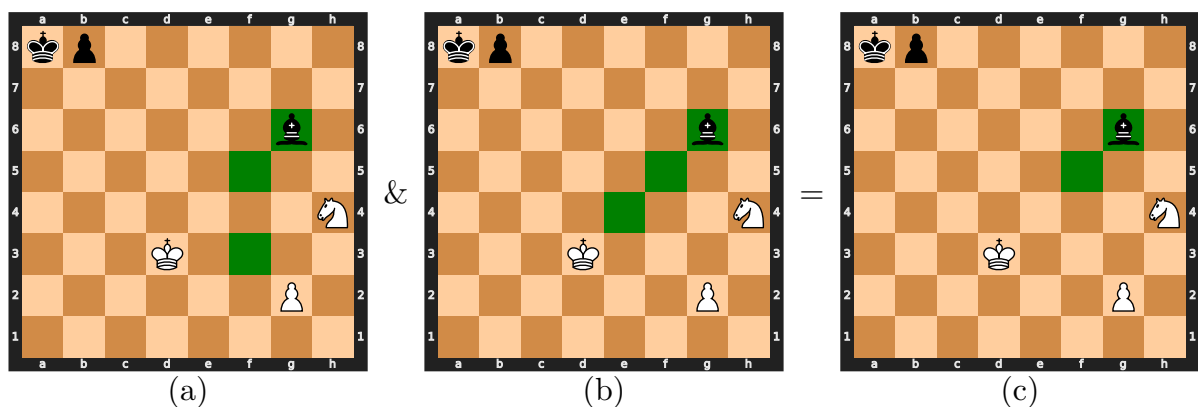


Abbildung 8: Reduktion der pseudo-legalen Ziele (a) eines Springers mithilfe einer Schachmaske (b) auf legale Ziele (c)

Anschließend wird die Zielmenge mit der Schachmaske verundet. Befindet sich die eigene Seite nicht im Schach, ist diese Maske vollständig gesetzt und verändert die Zielmenge nicht weiter. Liegt jedoch ein einfaches Schach vor, bleiben nur noch solche

Zielfelder erhalten, die die angreifenden Figur schlagen oder die Angriffslinie zum König unterbrechen. Im Fall eines Doppelschachs werden im Generator überhaupt keine Springerzüge mehr erzeugt, da ausschließlich Königszüge legal sind.

In diesem Fall befindet sich der weiße König im Schach, da der Läufer ihn angreift (s. Abbildung 8 a). Dadurch sind nur noch Ziele erlaubt, welche entweder den Läufer schlagen oder seinen Weg zum König kreuzen. Diese möglichen Ziele werden durch eine Schachmaske dargestellt (s. Abbildung 8 b). Abschnitt 4.2 behandelt die Berechnung dieser Maske. Mithilfe einer weiteren UND-Operation der pseudo-legalen Ziele mit der Schachmaske erhält man die beiden legalen Ziele (s. Abbildung 8 c).

Zusätzlich muss die Fesselung berücksichtigt werden. Eine absolut gefesselte Figur darf die Verbindungslinie zwischen eigenem König und gegnerischer Gleitfigur nicht verlassen. Für Springer ist diese Einschränkung besonders einfach umzusetzen: Da ein Springer keine horizontalen oder diagonalen Linienzüge ausführen kann, darf ein gefesselter Springer überhaupt keinen Zug ausführen und wird in der Implementierung direkt übersprungen. Liegt keine Fesselung vor, ist der Springer nicht in der Fesselmасke enthalten. Die Zielmenge bleibt dann unverändert. Die Fesselmасken selbst sind in diesem Fall nicht vollständig gesetzt, sondern leer.

Allgemein basiert dieser maskenbasierte Ansatz auf dem Zuggenerator „Gigantua“ [26]. Der entscheidende Vorteil liegt in der drastischen Reduktion von bedingten Sprüngen im Maschinencode. Es wird also zu großen Teilen Branchless-Programming benutzt (s. Abschnitt 2.3.3). Anstatt für jedes potenzielle Zielfeld einzeln mittels if-Abfragen zu prüfen, ob es besetzt ist, ob dort eine eigene Figur steht oder ob der König im Schach steht, werden alle diese Regeln durch eine Kette von bitweisen Operationen gleichzeitig für alle 64 Felder angewendet. Zusätzlich muss die Zuggenerierung noch die Fesselung beachten.

## 4.5 Zuggenerierung für Gleitfiguren

Als Gleitfiguren zählen der Läufer, Turm und Dame. Diese stellen eine besondere Herausforderung dar, da sich diese Figuren unbegrenzt weit in eine Richtung bewegen können. Die Bewegung wird jedoch durch Figuren im Weg beschränkt. Deshalb wird eine effiziente Methode benötigt, welche speziell Züge für diese Art der Bewegung berechnet.

Der einfachste Ansatz hierfür ist Raycasting. Für jede Figur wird in alle Richtungen, in die sich die Figur bewegen kann, ein Strahl (engl. Ray) geschossen. Dadurch werden alle Positionen erfasst, bis eine andere Figur getroffen wird. Der Vorteil bei diesem Ansatz ist, dass keine Lookup-Tabellen gespeichert werden müssen. Dafür ist der Rechenaufwand jedoch relativ hoch, da man für jeden Strahl jeweils eine Schleife laufen lassen muss. Dadurch wird eine Zeitkomplexität von  $\mathcal{O}(n)$ , wobei  $n$  der Anzahl der erfassten Positionen entspricht, erreicht.

Eine Zeitkomplexität von  $\mathcal{O}(1)$  wäre in der Theorie mithilfe einer Lookup-Tabelle einfach zu erreichen. Die trivialste Lösung wäre, für jeden möglichen Brettzustand das exakte Angriffs-Bitboard zu speichern. Das Problem ist jedoch, dass ein Schachbrett insgesamt 64 Felder hat und jedes dieser Felder entweder belegt oder frei ist. Dadurch entstehen  $2^{64}$  mögliche Kombinationen. Man müsste also für jedes der 64 Felder, auf dem eine Gleitfigur stehen könnte, alle  $2^{64}$  Kombinationen speichern. Jeder Eintrag würde ein Bitboard mit 8 Bytes speichern. Gleichung 3 zeigt den enormen Speicherbedarf der Lookup-Tabelle.

$$64 \text{ Felder} \cdot 2^{64} \text{ Zustände} \cdot 8 \text{ B} = 2^{73} \text{ B} \approx 10^{10} \text{ TB} \quad (1)$$

Gleichung 3: Speicherbedarf einer Lookup-Tabelle für alle Angriffs-Bitboards einer Gleitfigur

Mit der heutigen Technik ist es nicht möglich, diese Tabelle im Arbeitsspeicher zu halten. Hier entsteht ein Bedarf für einen Kompromiss zwischen Speicher- und Rechenaufwand. Als Lösung wird Hashing mit sogenannten „Magic Bitboards“ verwendet. Sie sind die Standardlösung aktueller Engines, wie zum Beispiel auch bei Stockfish [27]. Es handelt sich um einen Hash-Algorithmus, welcher die aktuelle Besetzung des Spielbretts mit einer magischen Zahl multipliziert. Diese Zahl dient als Index für eine Hashmap, welche direkt alle Zielpositionen enthält. Dadurch wird eine Zeitkomplexität von  $\mathcal{O}(1)$  mit deutlich geringerem Speicherbedarf für die Tabelle erreicht. Aufgrund dieses Leistungsvorteils wurden sie für diese Arbeit ausgewählt und werden anschließend näher erläutert [27], [28].

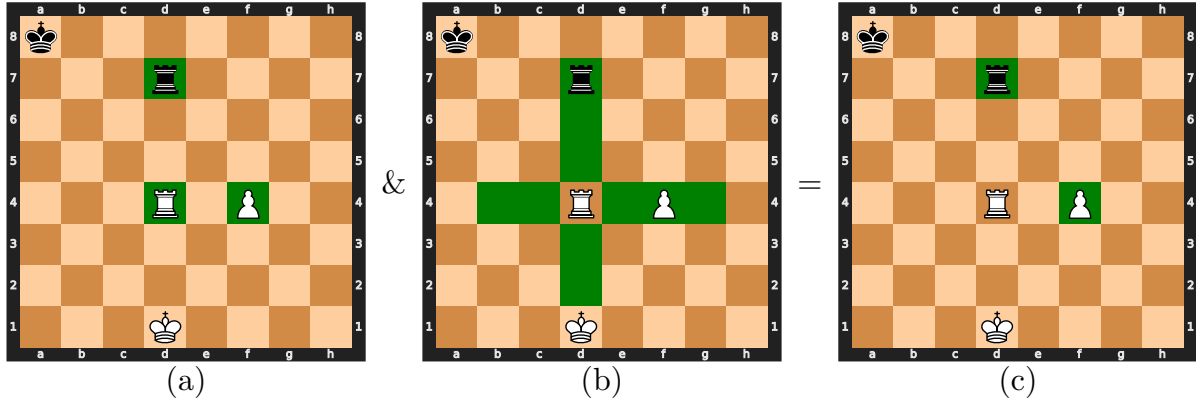


Abbildung 9: Visualisierung der Blocker Maske (c) durch UND-Verknüpfung der aktuell belegten Felder (a) und der Magic-Maske (b)

#### 4.5.1 Magic-Bitboards

Im ersten Schritt ermittelt der Algorithmus, welche Figuren den Turm überhaupt blockieren könnten. Abbildung 9 zeigt ein Beispiel, wo sich ein Turm aktuell auf Position D4 befindet. Zuerst wird eine Blocker-Maske berechnet, welche die aktuell belegten Felder mit der Magic-Maske durch die UND-Operation verknüpft. Die Magic-Maske ist eine vordefinierte Maske, welche für jedes der 64 Felder die gesamte Reihe und Zeile, bis auf die Position selber, enthält. Durch diese Berechnung erhalten wir alle Figuren, welche die Bewegung des Turms blockieren (s. Abbildung 9 (c)). Auffällig ist, dass die Randzeilen (1, 8) und Randlinien (a, h) nicht in der Magic-Maske gesetzt sind. Der Grund dafür ist, dass der Strahl ohnehin am Rand des Spielbretts aufhören würde. Es macht deshalb keinen Unterschied für den Algorithmus, ob diese Position besetzt ist. Dadurch können maximal 12 Bits in einer Maske für einen Turm gesetzt sein, anstatt von 14 Bits [27], [28].

$$\text{Index} = (\text{BlockerMaske} \cdot \text{MagicFaktor}) \gg (64 - \text{BlockerBits}) + \text{MagicOffset}$$

Gleichung 4: Index-Berechnung in der Lookup-Tabelle für Gleitfiguren mit Magic Bitboards

Gleichung 4 zeigt die Transformation der isolierten Bits aus der Blocker-Maske in einen dichten Index, der einen Zugriff auf die Lookup-Tabelle ermöglicht, welche alle Zugmöglichkeiten enthält. Die Bits der Blocker-Maske enthalten zwar bereits alle Informationen für den Lookup, aber diese Bits sind zu weit verstreut über das 64-Bit

Bitboard. Um daraus einen Index zu generieren, wird die Maske mit einem Magic-Faktor multipliziert [28].

Diese Multiplikation fungiert als eine Art „Bit-Sammler“. Die gesetzten Bits des Blocker Bitboards werden in den höchstwertigen Bits des 64-Bit-Ergebnisses verdichtet. Danach ist das Ergebnis jedoch weiterhin ein großer 64-Bit-Wert. Deswegen folgt ein Rechts-Shift um die maximale Anzahl der Bits in der Blockermaske. Beim Turm sind es 12 Bits, da wie oben gezeigt, maximal 12 Bits in der Blocker-Maske gesetzt sein können. Dadurch entsteht bei Türmen ein kompakter Index im Bereich von 0 bis 4095 ( $2^{12} - 1$ ). Bei Läufern werden nur maximal 9 Bits gebraucht. Der restliche Ablauf ist jedoch identisch [27], [28].

Schließlich wird zu diesem Index noch ein feldspezifischer Offset addiert. Dadurch kann die Lookup-Tabelle weiter komprimiert werden. Dieser Prozess nutzt gezielt konstruktive Kollisionen. Da aus unterschiedlichen Blocker-Masken oft das identische Angriffs-Bitboard resultiert, sorgt das perfekte Hashing dafür, dass diese redundanten Einträge auf den gleichen Tabellenplatz abgebildet werden. Dadurch wird die Lookup-Tabelle weiter komprimiert. Der finale Index wird dann als Schlüssel in dieser Lookup-Tabelle verwendet. Das Ergebnis ist ein Array von Angriffs-Bitboards, welches durch intelligente Indizierung alle potenziellen Ziele für jede mögliche Blocker-Maske enthält [27].

Die Ermittlung des Magic-Faktors erfolgt durch ein stochastisches Suchverfahren, bei dem für jedes Feld so lange zufällige 64-Bit-Werte getestet werden, bis eine kollisionsfreie Abbildung gefunden wird. Ein Faktor gilt als erfolgreich, wenn er jedem Zustand der Blocker-Maske einen eindeutigen Index zuweist oder ausschließlich konstruktive Kollisionen erzeugt [27], [28].

Die Magic-Offsets werden anschließend mit einem Packing-Algorithmus bestimmt, der die indizierten Bereiche der Felder in der Tabelle anordnet. Ziel dabei ist es, ungenutzte Lücken zwischen Einträgen zu minimieren, indem verschiedene Felder überlappen [27]. In dieser Arbeit wurden diese Faktoren und Offsets nicht selbst berechnet. Verwendet werden die Werte von Volker Anuss [29]. Diese Werte ermöglichen es, eine hochkomprimierte Tabelle zu bauen, welche mit 88772 Einträgen alle Angriffsbitboards für Läufer und Türme speichert. Die konkrete Implementierung wird von ihm als „Fixed

Shift White Magic Bitboards“ bezeichnet [29]. Generell gibt es viele Magic-Bitboard-Varianten, welche den Tabellenindex alle leicht anders berechnen. „Fixed Shift“ bezieht sich darauf, dass der Algorithmus einen festen Rechts-Shift um  $64 - 12$  durchführt. „Fancy“ bedeutet, dass es pro Feld einen anderen Magic-Faktor und Magic-Offset gibt und „White Magic“ heißt, dass man die Blocker-Maske nicht invertiert. Alternativ existieren nämlich noch „Black Magic“ Bitboards, welche die Blocker-Maske invertieren, um die Tabelle weiter zu komprimieren. Am Ende wurden in dieser Arbeit aber die „White Magics“ gewählt, da diese zwar eine weniger kompakte, aber dafür cache-optimiertere Tabellen erstellen [21], [29].

#### 4.5.2 Reduktion auf legale Züge

Nach der Benutzung des Magic-Bitboard-Algorithmus erhält man die potenziellen Ziele für den Turm. Die Ziele müssen nun jedoch genau wie beim Springer zuerst auf pseudo-legale Ziele reduziert werden, um dann anschließend die legalen Züge zu berechnen.

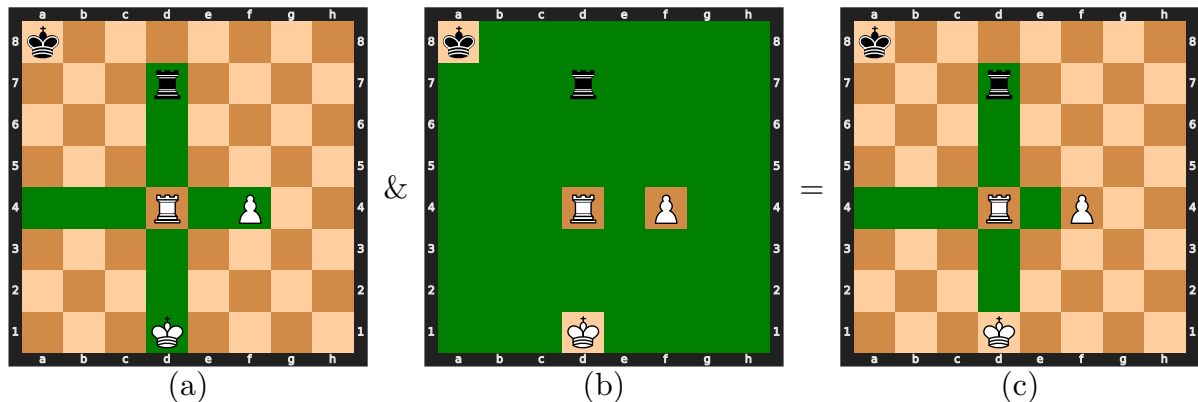


Abbildung 10: Reduktion der potenziellen Ziele eines Turms (a) auf pseudo-legale Ziele (c) mithilfe der Maske aus freien und gegnerischen Feldern (b)

Die pseudo-legalen Ziele werden, genau wie beim Springer, durch eine UND-Verknüpfung der potenziellen Ziele (s. Abbildung 10 a) mit einer Maske aus gegnerischen und leeren Feldern (s. Abbildung 10 b) berechnet.

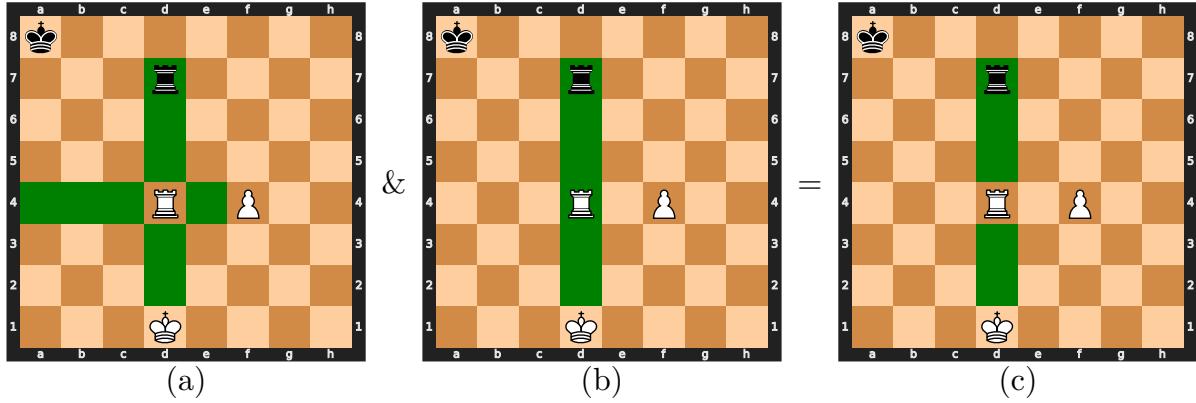


Abbildung 11: Reduktion der pseudo-legalen Ziele (a) eines Turms mithilfe einer Fesselmaske (b) auf legale Ziele (c)

Anschließend müssen die pseudo-legalen Züge auf legale Züge reduziert werden. Das Vorgehen unterscheidet sich nicht von den statischen Figuren. Nun befindet man sich aber in einer anderen Spielsituation. Es liegt kein Schach vor, also ist die Schachmaske vollständig gesetzt. Jedoch liegt nun eine Fesselung vor, da der weiße Turm einen möglichen Schachangriff des schwarzen Turms blockiert. Die Berechnung dieser Maske wurde in Abschnitt 4.3 behandelt. Deswegen darf der weiße Turm sich nur auf der Fessellinie vom weißen König bis zum schwarzen Turm bewegen. Diese Situation wird, wie beim Schach, durch eine weitere Maske abgebildet (s. Abbildung 11 (b)). Die Berechnung der legalen Züge erfolgt also durch die UND-Verknüpfung der pseudo-legalen Züge mit der Fesselmaske. Die finalen legalen Züge des weißen Turms werden durch Abbildung 11 (c) dargestellt.

Trotz der Vielzahl an Reduktionsschritten bleibt die Zuggenerierung insgesamt effizient, da jeder Schritt aus wenigen Bitboard-Operationen besteht. Branchless Bitweise UND-Verknüpfungen, einfache Masken und Lookup-Tabellen ersetzen aufwendige Fallunterscheidungen und Schleifen.

Das Ergebnis dieser Effizienz lässt sich durch einen sogenannten Perft-Test belegen. Ein Perft-Test zählt die Anzahl der legalen Züge einer Stellung bis zu einer bestimmten Tiefe. Tabelle 1 zeigt die Ergebnisse der Startposition bis zur Tiefe 8. Die Anzahl der Knoten und deren Aufschlüsselung bestätigt die Korrektheit der Zuggenerierung. Die Erwartungswerte wurden mit den bekannten korrekten Werten [30] abgeglichen und stimmen exakt überein.

| <b>Tiefe</b> | <b>Knoten</b>  | <b>Schlagende<br/>Züge</b> | <b>Schachmatts</b> | <b>Dauer (ms)</b> | <b>Knoten/s</b> |
|--------------|----------------|----------------------------|--------------------|-------------------|-----------------|
| 1            | 20             | 0                          | 0                  | 0 04              | 465 100         |
| 2            | 400            | 0                          | 0                  | 0 06              | 6 600 000       |
| 3            | 8 902          | 34                         | 0                  | 0 27              | 33 400 000      |
| 4            | 197 281        | 1 576                      | 8                  | 4 61              | 42 700 000      |
| 5            | 4 865 609      | 82 719                     | 347                | 27 39             | 177 600 000     |
| 6            | 119 060 324    | 2 812 008                  | 10 828             | 561 30            | 212 100 000     |
| 7            | 3 195 901 860  | 108 329 926                | 435 767            | 13 660            | 233 800 000     |
| 8            | 84 998 978 956 | 3 523 740 106              | 9 852 036          | 368 800           | 230 400 000     |

Tabelle 1: Perft-Test der Engine für die Startposition bis Tiefe 8

Tabelle 1 zeigt, dass die Engine selbst noch in Tiefe 8 über 84 Milliarden Knoten korrekt verarbeitet. Jeder einzelne dieser Knoten durchläuft die beschriebenen Reduktionsschritte aus Schachmaske, Fesselmasken und Bitboard-Operationen. Der Test wurde auf einem einzelnen Kern auf einem Intel i5-13600KF Prozessor durchgeführt. Dies bestätigt, dass die Zuggenerierung korrekt arbeitet und alle Sonderfälle wie Rochaden, En Passant, Bauernumwandlungen sowie Schach- und Fesselregeln vollständig abdeckt.

Bereits ab Tiefe 5 wird eine Geschwindigkeit von über 177 Millionen Knoten pro Sekunde erreicht, die in den folgenden Tiefen auf über 230 Millionen Knoten pro Sekunde ansteigt. Trotz dieser hohen Geschwindigkeit zeigt sich, dass eine schnelle Zuggenerierung allein nicht ausreicht. Während Tiefe 6 nur etwa 0,56 Sekunden benötigt, steigt die Zeit in Tiefe 7 auf 13,66 Sekunden und in Tiefe 8 auf über sechs Minuten. Grund dafür ist die exponentiell wachsende Anzahl an Knoten, die mit jeder zusätzlichen Tiefe untersucht werden müssen. Die Zuggenerierung ist daher nur ein Teil einer leistungsfähigen Engine. Erst die in den folgenden Kapiteln beschriebenen Suchalgorithmen und Bewertungsfunktionen ermöglichen eine noch tiefergehende Analyse des Spielbaums in einer limitierten Zeit.

## 5 Suchalgorithmen

Das Fundament jeder Schach-Engine bildet die heuristische Suche. Ihr Ziel ist es, den objektiv besten Zug einer Stellung mithilfe einer Tiefensuche durch den simulierten Spielbaum zu ermitteln. Hierbei wird sogenanntes „perfektes Spiel“ angenommen: Dies bedeutet, dass jeder Spieler in jeder Position stets den (objektiv) besten Zug spielt.

In diesem Abschnitt wollen wir eine detaillierte Übersicht über die am weitesten verbreitete Familie dieser Suchalgorithmen geben: Minimax mit der Alpha-Beta-Optimierung. Anschließend werden einige zusätzliche Optimierungen dieser Algorithmen vorgestellt sowie Erweiterungen, die die Performanz der Suche weiter verbessern und einen naiven Brute-Force-Algorithmus in eine intelligente heuristische Suche transformieren.

### 5.1 Minimax

Der Minimax-Algorithmus ist ein simpler Algorithmus, der in einem Nullsummenspiel die Züge errechnet, die für einen Spieler zum bestmöglichen Ergebnis führen. Dies geschieht durch eine Brute-Force-Simulation des gesamten Spielbaums. Der Algorithmus liefert die Grundlage für rein algorithmische Schachprogrammierung, und wurde bereits 1949 von Claude Shannon in [1] motiviert.

Der Minimax Algorithmus simuliert zwei Spieler: den maximierenden Spieler MAX (mit dem Ziel, die größtmögliche Bewertung zu erzielen), und den minimierenden Spieler MIN (mit dem Ziel, die kleinstmögliche Bewertung zu erzielen). Ausgangspunkt des Algorithmus ist eine Startposition  $k_0$ , bei der der maximierende Spieler am Zug ist. Basierend auf dieser Position wird dann rekursiv ein Baum mit festgelegter Tiefe  $d$  aufgebaut: Für jede Position  $k_{ij}$  werden durch Zuggenerierung alle  $n$  Kindpositionen  $k_{(i+1)n}$  identifiziert. Eine Kindposition definiert sich hierbei als eine Position, die durch genau einen legalen Zug aus der aktuellen Position  $k_i$  erreicht werden kann. Dies wird für jede Position wiederholt, bis die Baumtiefe  $d$  erreicht ist, also das Spiel  $d$  Ply in die Zukunft simuliert wurde. Im Spezialfall Schach wechseln sich MIN und MAX in ihren Zügen ab. Haben sowohl MIN als auch MAX jeweils eine Figur bewegt, spricht man von einem vollen Zug. Spricht man von dem durch entweder MIN oder MAX getätigten

Teilzug, redet man von einem Ply. Die Blatt-Positionen des Suchbaums werden dann mithilfe einer statischen Evaluationsfunktion  $\text{eval} : S \rightarrow \mathbb{N}$  numerisch bewertet. Abbildung 12 zeigt einen exemplarischen Suchbaum auf, bei dem die Evaluationen der Blattpositionen willkürlich gewählt sind.

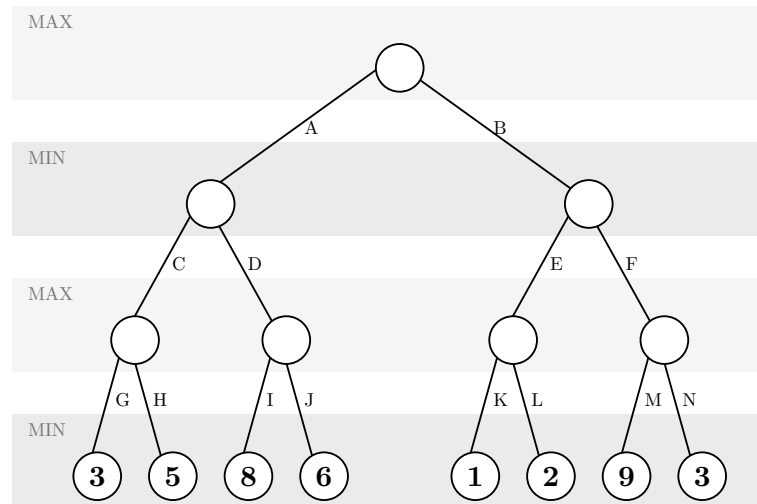


Abbildung 12: Exemplarischer Minimax-Baum mit Blattbewertungen

Um herauszufinden, welcher Zug in der Ausgangsposition bei perfektem Spiel zur höchsten Evaluation führt, also für MIN das bestmögliche Ergebnis darstellt, wird der Baum von unten nach oben mit Bewertungen gefüllt. Die Bewertung einer Position  $k_i$  ist stets das Minimum oder Maximum der Bewertungen ihrer Kinder, abhängig davon, welcher Spieler in  $k_i$  am Zug ist. Schlussendlich wird  $k_0$  durch das Maximum ihrer Kindpositionen errechnet. Der beste Zug ist derjenige, der zu der Kindposition mit dieser maximalen Bewertung führt. Abbildung 13 zeigt für den obigen Baum, wie dieser gefüllt wird. Die PV ist rot markiert.

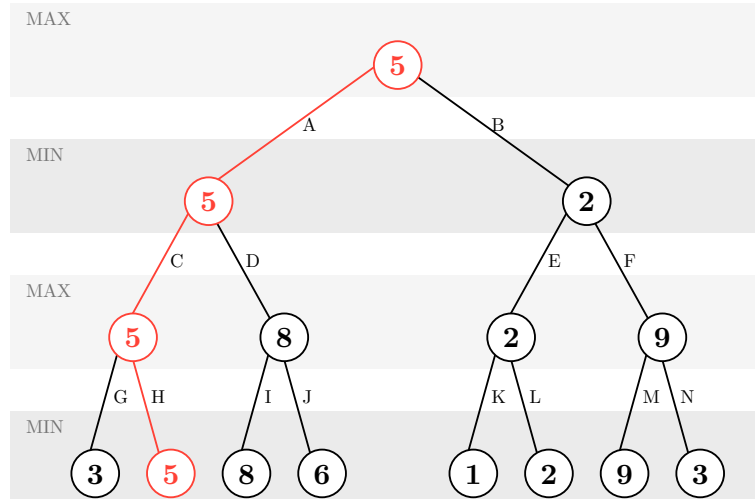


Abbildung 13: Minimax-Baum mit propagierten Bewertungen und markierter PV

## 5.2 Alpha-Beta-Pruning

Minimax garantiert die Findung des bestmöglichen Zuges dadurch, dass alle möglichen Positionen bis hin zur Tiefe  $d$  evaluiert werden. Die Anzahl an besuchten Positionen wächst allerdings exponentiell in  $d$ ; durch die vielen Zugmöglichkeiten im Schach wird der Suchbaum deswegen sehr schnell sehr breit. Ausgehend von der Startposition beispielsweise gibt es bereits nach 10 Ply über  $69 \cdot 10^{15}$  mögliche Positionen [30]. Aufgrund dieses exponentiellen Wachstums stößt der naive Minimax-Algorithmus schnell an seine Grenzen.

Alpha-Beta-Pruning, erstmals motiviert in 1961 von Edwards und Hart [31], ist eine Optimierung des Minimax-Algorithmus, die es ermöglicht, unnötige Pfade des Suchbaums zu entfernen. Der Grundgedanke hierbei ist, dass bei perfektem Spiel nur die PV betrachtet werden muss. Sobald für eine Position gezeigt werden kann, dass sie nicht auf der PV liegt, kann sie verworfen werden und bislang unbetrachtete Kindpositionen ignoriert werden. Dies wird als „Cutoff“ bezeichnet. Die Kombination aus der Minimax-Suche mit der Alpha-Beta-Optimierung wird im Folgenden „Alpha-Beta-Suche“ genannt.

Es gibt zwei Möglichkeiten, wie eine Position als abseits der PV liegen kann [31]:

1. MAX findet einen Zug, der zu einer höheren Evaluation führt als ein anderer Zug von MIN in der Elternposition (genannt: „failing high“)

2. MIN findet einen Zug, der zu einer niedrigeren Evaluation führt als ein anderer Zug von MAX in der Elternposition (genannt: „failing low“)

Im ersten Fall wird MIN bei perfektem Spiel den anderen Zug spielen, um zu vermeiden, dass MIN eine für ihn bessere Position erzwingt; im zweiten Fall vice versa.

Programmatisch wird dieses Prinzip realisiert durch zwei Variablen, die durch den gesamten Suchbaum gereicht werden:  $\alpha$  und  $\beta$ .  $\alpha$  repräsentiert hierbei das bisher bestmögliche Ergebnis für MAX, und  $\beta$  das bestmögliche Ergebnis für MIN; jeweils bei perfektem Spiel. Dadurch fungieren  $\alpha$  und  $\beta$  als untere und obere Schranken für die von der Suche ermittelte Evaluation von  $k_0$ , und bewegen sich im Verlauf einer Suche stetig aufeinander zu, wenn bessere Positionen für die jeweiligen Spieler gefunden werden. Fällt der gesuchte Evaluationswert  $v_k$  eines Knotens außerhalb dieses Intervalls, kann der Pfad verworfen werden.  $v_k > \beta$  ist der Fall „failing high“, und  $v_k < \alpha$  ist der Fall „failing low“. Dieses von  $\alpha$  und  $\beta$  aufgespannte Intervall  $[\alpha, \beta]$  wird als **Suchfenster** bezeichnet. Angemerkt sei, dass im folgenden Verlauf dieser Arbeit stets eine Simplifizierung vorgenommen wird, um die Fallunterscheidung zwischen  $\alpha$  und  $\beta$  basierend darauf, wer gerade am Zug ist, zu vermeiden. Alle Vergleiche mit den Schranken werden ausschließlich aus dem Blickwinkel von MAX betrachtet.<sup>3</sup>

Um das Konzept von Alpha-Beta Pruning zu illustrieren, soll der bereits eingeführte Minimax-Baum als Beispiel dienen. Abbildung 14 zeigt in diesem Baum die gekürzten Pfade ausgegraut. In der ersten Kürzung ist MAX am Zug, und findet Zug *I* mit Evaluation 8. Da MIN in der Elternposition einen Zug (*C*) hat, der zur Evaluation 5 führt, wird MIN diesen Zug spielen. Die zweite Kürzung ist ein Fail-Low-Knoten, nach Zug *B*. Hier findet MIN den Zug *E* mit Evaluation 2, allerdings kann MAX sich in der vorherigen Position durch Zug *A* bereits eine Evaluation von 5 garantieren. Deshalb wird MAX Zug *B* nicht spielen, da er sich dadurch eine schlechtere Evaluation verschafft.

---

<sup>3</sup>Diese Simplifizierung ist bekannt unter dem Namen Negamax; in der Implementierung wird hierbei in jedem Schritt der Wert von  $\alpha$  und  $\beta$  je zuerst negiert und anschließend vertauscht, sodass der aktuell ziehende Spieler stets die Bewertung maximieren will. Dies verhindert doppelte Logik im Code.

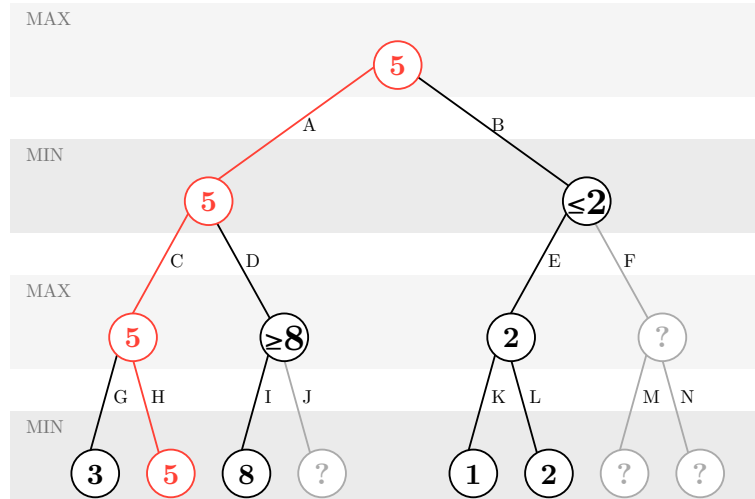


Abbildung 14: Minimax-Baum mit abgeschnittenen Pfaden

Angemerkt sei hier, dass Alpha-Beta-Pruning als Tiefensuche stark abhängig ist von der Reihenfolge der Kindpositionen. Wäre in Abbildung 14 beispielsweise in  $k_0$  Zug  $B$  vor Zug  $A$  simuliert worden, hätten Zug  $F$  und die resultierenden Züge auch berechnet werden müssen. Diese Tatsache ist der Grund, warum Zugsortierung von essenzieller Wichtigkeit für die Performanz einer Alpha-Beta-Suche ist [32, p. 172].

### 5.3 Quiescence Search

Minimax garantiert, den bestmöglichen Zug bei einer Spieltiefe von  $d$  Ply zu finden. Durch das harte Limit bei Tiefe  $d$  des Suchbaums kann aber nicht garantiert werden, dass der Zug  $d + 1$  eine gravierende Änderung an der Evaluation vornimmt. Dies ist als „Horizon Effect“ bekannt [32, p. 36]. Als illustrierendes Beispiel soll folgende Position dienen:

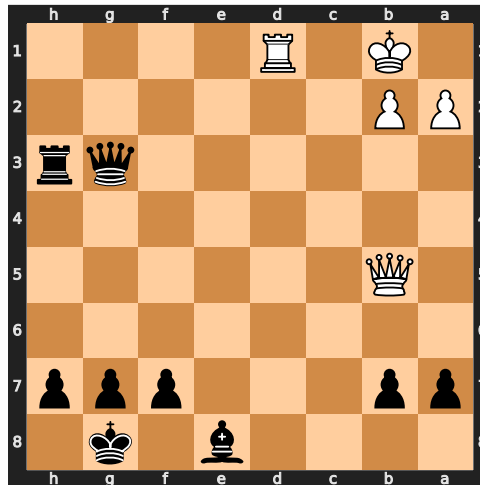


Abbildung 15: Beispielposition für Horizon Effect, Weiß am Zug

Hier ist Schwarz am Zug. Mit einer Suchbaumtiefe von 3 Ply ist der beste Zug **Bxb5**, da Schwarz hierdurch erheblichen Materialgewinn erzielt. Die scheinbare PV in 3 Ply lautet: 1. **Bxb5** 2. **Rd8+** 3. **Be8**, wonach sich folgende Position ergibt:

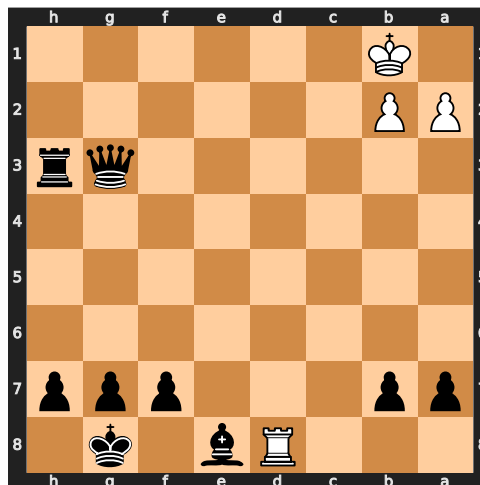


Abbildung 16: Beispielposition für Horizon Effect, nach 3 Ply

Jetzt ist Weiß am Zug, und kann mit 4. **Rxe8#** Schwarz Matt setzen. Da die Suche aber nur 3 Ply umfasst, liegt dieser Zug über dem Horizont des Algorithmus und wird deswegen nicht berücksichtigt.

Die Lösung für dieses Problem liefert das Konzept der QS. Hierbei wird nach Erreichen der Tiefe *d* eine weitere Alpha-Beta-Suche auf *zwingenden Zügen* durchgeführt. Ein zwingender Zug kann vage definiert werden als ein Zug, der eine taktische Sequenz einleitet, die kurzfristig gravierende Änderungen an der Evaluation der Position

vornimmt; was genau als ein zwingender Zug gilt, ist allerdings nicht klar definiert. Als kleiner gemeinsamer Nenner können schlagende Züge genannt werden, da diese eine sofort merkbare Auswirkung auf die Evaluation haben [1, ch. 6]. Schach setzende Züge sind ebenfalls lohnenswert zu betrachten, wie das gegebene Beispiel aufzeigt. Thunfisch betrachtet ausschließlich schlagende Züge - mit der Ausnahme von Positionen, in denen ein Schach existiert. Hier müssen alle Züge betrachtet werden, um ggf. ein falsches Matt zu verhindern. Zusätzlich betrachtet Thunfisch ruhige Züge, die ein Schach geben, für die ersten zwei Ply der QS.

## 5.4 Principal Variation Search

Eine weitere Optimierung der Alpha-Beta-Suche ist die Principal Variation Search (PVS). PVS klassifiziert für jeden Knoten  $k$ , ob dieser auf der PV liegt:  $k \in PV$  oder  $k \in \overline{PV}$ . In welche der beiden Kategorien  $k$  fällt, wird vom Elternknoten zunächst heuristisch auf Basis der Zugsortierung entschieden: Es wird davon ausgegangen, dass der an erster Stelle sortierte Zug der beste Zug in der Position ist. Somit ist  $k \in PV$  gdw.  $k$  sich aus dem ersten Zug in  $k_{-1}$  ergibt. Basierend auf dieser Klassifikation wird das Suchfenster angepasst. PV-Knoten werden mit dem vollen Fenster  $[\alpha, \beta]$  gesucht. Für alle anderen Knoten wird das Suchfenster auf ein sogenanntes Zero Window beschränkt:  $[\alpha, \alpha + 1]$ . Dieses Fenster hat die spezielle Eigenschaft, dass Pfade sehr schnell durch fail-low Cutoffs verworfen werden können. Wird  $\alpha$  jedoch verbessert, führt dies sofort zu einem fail-high Cutoff, und die gesamte Teilsuche wird abgebrochen [33].

Durch diese schnelle Entscheidungsfähigkeit ist eine Suche mit Zero Window sehr performant. Der Nachteil ergibt sich daraus, dass das Ergebnis der Suche keine exakte Bewertung zurückgeben kann; Die gewonnene Information beschränkt sich auf das Beantworten der Frage: „Kann der aktuelle Spieler sich in dieser Position verbessern“? Falls dies der Fall ist (gezeigt durch einen fail-high Cutoff), heißt dies, dass die Kategorisierung des aktuellen Knotens als  $k \in \overline{PV}$  falsch war. Als Resultat muss der Knoten mit vollem Suchfenster erneut durchsucht werden, da er jetzt Teil der Principal Variation ist [34, ch. 3.1].

Angemerkt sei, dass PVS durch die initiale Kategorisierung der Kindknoten stark abhängig ist von der Stärke der Zugsortierung. Je häufiger die Zugsortierung den tatsächlich besten Zug an erste Stelle sortiert, desto weniger Knoten muss PVS auf

vollständiger Tiefe durchsuchen. Aus diesem Grund bildet PVS eine starke Synergie mit der Transpositionstabelle (s. Abschnitt 5.6), wenn diese zu Zugsortierungszwecken genutzt wird.

## 5.5 Iterative Deepening

Eine Einschränkung der Alpha-Beta-Suche ist, dass die Suchtiefe (abseits der QS) inhärent durch ein festes  $d$  festgelegt ist. Einem realen Spiel mit Zeitkontrolle wird eine fixe Suchtiefe allerdings nicht gerecht. Um die besten Züge zu finden, sollte eine Engine möglichst all ihre verfügbare Zeit nutzen, um die Anzahl der betrachteten Züge zu maximieren. Allerdings darf die Engine nicht zu tief suchen, da sie sonst womöglich die verfügbare Zeit überschreitet, und somit das Spiel verliert [20, art. 6.9]

Als Lösung für dieses Problem hat sich Iterative Deepening etabliert [32, p. 185]. Dabei wird die Suchtiefe iterativ erhöht. Begonnen wird mit einer Tiefe von  $d = 1$ , und die Suche wird inklusive QS vollständig durchgeführt. Ist noch Zeit übrig, wird eine neue Suche mit  $d = 2$  durchgeführt, anschließend mit  $d = 3$  usw. Ist die allokierte Zeit für die Suche ausgelaufen, wird die aktuelle Suche abgebrochen und das Ergebnis der tiefsten vollständig durchgeführten Suche genommen.

## 5.6 Transpositionstabelle

Für eine Schach-Engine ist die Effizienz der Suche der limitierende Faktor für die Spielstärke. Dabei spielen sogenannte Transpositionen eine große Rolle. Eine Transposition bezeichnet das Erreichen einer identischen Brettstellung über unterschiedliche Zugreihenfolgen. So führt die Eröffnung 1. e4 e5 2. d4 d5 zur gleichen Stellung wie 1. d4 d5 2. e4, e5. Um zu verhindern, dass diese beiden Teilbäume mit gleichem Ergebnis mehrfach unabhängig voneinander durchsucht werden, wird eine Transpositionstabelle (TT) eingesetzt. Dabei handelt es sich um eine Hash-Tabelle, welche als globales Gedächtnis der Suche agiert und Informationen über bereits besuchte Positionen abspeichert [35].

### 5.6.1 Optimierte Datenstruktur der Einträge

Die meisten Schach-Engines limitieren die Größe der TT. Die genaue Größe lässt sich mit dem UCI-Protokoll über den Befehl `setoption name Hash <Hashgröße>` festlegen,

wobei `Hashgröße` die Größe der TT in Megabyte definiert. Bei Computer Chess Rating Lists (CCRL) ist die Größe je nach Turnierformat entweder auf 256 MB oder 512 MB festgelegt. CCRL ist der größte Verein, welcher sich auf Turniere zwischen den besten Chessengines fokussiert. Anhand der Ergebnisse werden die allgemein anerkannten Elo-Zahlen der besten Schachprogramme bestimmt [13].

Aufgrund diesen Limitierungen ist eine speicher- und cacheeffiziente Ausrichtung der TT von kritischer Bedeutung. Jedes zusätzliche Byte pro Eintrag hat große Auswirkungen auf die speicherbare Gesamtzahl der Stellungen. In Thunfisch wird deswegen Bit-Packing benutzt, um mehrere Variablen in nur einem 64-Bit Integer pro Eintrag zu speichern. Listing 7 zeigt komprimierte Darstellung eines Eintrags (`EncodedHashEntry`) und dekomprimierte Darstellung (`DecodedTTEntry`). Dadurch, dass beide Strukturen 64-Bit groß sind und durch `repr(C)` die Reihenfolge der Variablen im dekomprimierten Zustand auf die Definitionsreihenfolge festgelegt ist, kann der Eintrag mittels `std::mem::transmute` ohne Kosten als komprimiert interpretiert werden [35].

```
1  #[derive(Default)]
2  pub struct EncodedHashEntry {
3      data: u64,
4  }
5  #[repr(C)]
6  pub struct DecodedTTEntry {
7      key: u16,
8      best_move: EncodedMove, // u16
9      score: i16,
10     depth: i8,
11     info: TTInfo, // u8
12 }
```

 Rust

Listing 7: Definition der Datenstrukturen für komprimierte und dekomprimierte Tabelleneinträge in der Transpositionstabelle

Jeder Eintrag ist so kompakt wie möglich, um eine möglichst große Anzahl zu ermöglichen. Das 64-Bit-Layout ist inspiriert von der Schach-Engine Akimbo und Viridithas [15], [36]. Es teilt sich wie folgt auf:

- **Schlüssel** (key): Um Kollisionen zu vermeiden, werden die oberen 16-Bit des Board-Hashes gespeichert.
- **Bester Zug** (bestmove): Dieses Feld speichert den besten Zug, der für diese Brettstellung ermittelt wurde.
- **Bewertung** (score): In diesem Feld wird bisherige Evaluation der Brettstellung gespeichert.
- **Suchtiefe** (depth): Die verbleibende Tiefe des Suchbaums aus dem die Bewertung hervorging.
- **Weitere Informationen** (info): In den verbleibenden 8-Bit werden in einer Struktur zusätzliche Information gespeichert.
  - **Alpha-Beta-Grenzen** (Bits 0 bis 1): Die ersten beiden Bits werden genutzt, um die Bewertung auf Basis der Alpha-Beta-Grenzen (Abschnitt 5.2) näher zu beschreiben. Insgesamt gibt es vier Fälle:
    - **Exakt**: Die Bewertung ist der exakte Minimax-Wert.
    - **Niedriger**: Die echte Bewertung ist mindestens so hoch wie gespeichert.
    - **Höher**: Die echte Bewertung ist höchstens so hoch wie gespeichert.
  - **PV-Boolean** (Bit 2): Das dritte Bit ist gesetzt, wenn sich der gesuchte Knoten auf der bisherigen PV befindet. In diesem Fall bekommt der Eintrag eine höhere Priorität und wird so seltener überschrieben.
  - **Alter des Eintrags** (Bits 3 bis 7): Jeder Eintrag wird mit einem Alter gespeichert. Dadurch werden alte Einträge beim Speichern eher überschrieben als neue Werte, da diese tendenziell nicht mehr aktuelle Informationen enthalten.

### 5.6.2 Speichern von Einträgen

Die Transpositionstabelle wird am Ende der Suche eines Knotens in der Alpha-Beta-Suche beschrieben. Zu diesem Zeitpunkt wurden alle Kindpositionen des Knotens bereits besucht und man weiß die Bewertung sowie den besten nächsten Zug für diese Position. Diese Informationen werden nun für die Zukunft gespeichert. Dadurch kann eine erneute Berechnung des Knotens in der Zukunft eingespart werden.

Der Zielindex in der Tabelle wird über die unteren Bits des Zobrist-Hashes (Abschnitt 2.5.2) bestimmt:  $\text{idx} = \text{hash} \bmod n$ , wobei  $n$  die Anzahl der Einträge in der Tabelle ist. Diese ist stets eine Zweierpotenz. Dadurch kann die Modulooperation effizient mit einer bitweisen Und-Verknüpfung umgesetzt werden. Zur

Kollisionserkennung werden die oberen 16 Bit des Zobrist-Hashes als Schlüssel (key) im Eintrag abgelegt. So wird verhindert, dass eine Position mit denselben unteren Bits einen Eintrag einer komplett anderen Position aus der Tabelle liest. Kollisionen werden so zwar nicht vollständig verhindert, aber auf ein Minimum reduziert [35], [37].

Da die Transpositionstabelle beschränkt ist und mehrere Stellungen auf denselben Index abgebildet werden können, muss eine Ersetzungsstrategie festgelegt werden, wann ein alter Eintrag überschrieben wird. Thunfisch verwendet hierbei eine Prioritätsstrategie, die verschiedene Faktoren gewichtet und auf der Implementierung von Viridithas basiert [15].

$$p_{\text{neu}} = 3 \cdot \left( d_{\text{neu}} + s_{\text{neu}} + \left\lfloor \frac{(\Delta A)^2}{4} \right\rfloor \right)$$

$$p_{\text{alt}} = 2 \cdot (d_{\text{alt}} + s_{\text{alt}})$$

Gleichung 5: Berechnung der Priorität beim Speichern eines neuen Eintrags in der Transpositionstabelle

- $d$  beschreibt die Suchtiefe, aus der die Bewertung kommt, dadurch werden tiefere Bewertungen belohnt
- $s$  beschreibt die Art der Schranke aus der Alpha-Beta-Suche<sup>4</sup> (Exakt = 3, Unten = 2, Oben = 1)
- $\Delta A$  beschreibt den Altersunterschied zwischen der alten bisherigen Funktion und der neuen Position in Ply. Die quadratische Skalierung stellt sicher, dass sehr alte Einträge auch dann überschrieben werden, wenn sie eine hohe Tiefe besitzen.

Die Priorität  $p_{\text{neu}}$  wird für den neuen Eintrag, welcher gespeichert werden soll und für  $p_{\text{alt}}$ , den bisherigen Eintrag an der Stelle berechnet. Wenn die  $p_{\text{neu}} \geq p_{\text{alt}}$  gilt, wird der alte Eintrag überschrieben. Außerdem wird der alte Eintrag immer überschrieben, wenn  $s_{\text{neu}}$  exakt ist und  $s_{\text{alt}}$  nicht exakt ist. Diese Bevorzugung exakter Einträge ist dadurch motiviert, dass diese den tatsächlichen Minimax-Wert der Stellung repräsentieren, während untere und obere Schranken nur approximative Informationen liefern [35].

Gespeichert werden im neuen Eintrag die Informationen aus Listing 7. Zusätzlich werden Bewertungen, die einem Matt in  $n$  Ply entsprechen, als  $\text{MATE} - n$  (für Matt durch den

---

<sup>4</sup>Ist der aus der Alpha-Beta-Suche zurückgegebene Wert  $v_k > \beta$ , ist  $v_k$  eine untere Schranke, da die Suche den exakten Wert aufgrund eines Cutoffs nicht ermitteln konnte. Analog ist  $v_k < \alpha$  eine obere Schranke (s. Abschnitt 5.2).

suchenden Spieler) beziehungsweise  $-\text{MATE} + n$  (für Matt durch den Gegner) kodiert (s. Abschnitt 6). Matt-Bewertungen sind nämlich positionsabhängig: Ein Matt in 5 Ply, das auf Tiefe 2 des Suchbaums gefunden wurde, ist für einen Knoten auf Tiefe 4 nur noch ein Matt in 3 Ply. Um die Bewertung von der Tiefe zu entkoppeln, werden Matt-Scores bei der Speicherung normalisiert, indem der Ply-Wert des speichernden Knotens abgezogen wird. Bei der Abfrage wird die Ply-Verschiebung entsprechend zurückgerechnet, sodass stets die korrekte Matt-Distanz für den abfragenden Knoten angezeigt wird [35].

### 5.6.3 Abfragen von Einträgen

Die Abfrage der Transpositionstabelle erfolgt zu Beginn eines jeden Knotens, noch vor der Zuggenerierung. Über die Methode `probe` wird der Hash der aktuellen Stellung in der Tabelle nachgeschlagen. Dabei werden die folgenden Schritte durchlaufen [37]:

1. **Indexberechnung:** Der Zielindex wird analog zur Speicherung über die unteren Bits des Hashes bestimmt.
2. **Kollisionsprüfung:** Die oberen 16 Bit des Hashes werden mit dem gespeicherten Schlüssel verglichen. Stimmen diese nicht überein, handelt es sich um eine Kollision; die Abfrage gibt `None` zurück.
3. **Dekomprimierung:** Bei einem Treffer wird der 64-Bit-Eintrag über `std::mem::transmute` in die dekomprimierte Struktur `DecodedTTEntry` überführt (s. Listing 7).
4. **Rückrechnung von Matt-Bewertungen:** Bei einer Matt-Bewertung wird die vorherige Normalisierung nun, wie vorher beschrieben, wieder rückgängig gemacht.

### 5.6.4 Einsatzzweck in der Suche

Die Transpositionstabelle erfüllt in der Alpha-Beta-Suche zwei zentrale Aufgaben: Sie ermöglicht das vorzeitige Verwerfen von Knoten durch gespeicherte Bewertungen (sogenannte TT-Cutoffs) und liefert einen bevorzugten Zug für die Zugsortierung.

Wird ein Tabellentreffer erzielt, wird zunächst geprüft, ob der gespeicherte Eintrag für einen sofortigen Abbruch des aktuellen Knotens genügt. Hierfür müssen zwei Bedingungen erfüllt sein [36]:

1. Die Suchtiefe des gespeicherten Eintrags muss mindestens der aktuellen Suchtiefe entsprechen, damit wird eine ausreichende Qualität des Eintrags für die aktuelle Tiefe gewährleistet
2. Die gespeicherte gesuchte Bewertung  $v_k$  muss in Abhängigkeit von der gespeicherten Schranke einen Abbruch rechtfertigen:
  - Bei einer **unteren Schranke**:  $v_k > \beta$  — der Knoten kann verworfen werden, da der aktuelle Pfad für MIN nicht optimal ist.
  - Bei einer **oberen Schranke**:  $v_k < \alpha$  — der Knoten kann verworfen werden, da der aktuelle Pfad für MAX nicht optimal ist.
  - Bei einer **exakten Bewertung**: Die Bewertung kann in jedem Fall übernommen werden.

Thunfisch wendet TT-Cutoffs nur auf Knoten an, die nicht von der PVS als PV-Knoten klassifiziert wurden. Für PV-Knoten wäre ein vorzeitiger Abbruch zu riskant, da die Bewertung der Principal Variation möglichst exakt bestimmt werden muss [36].

Unabhängig davon, ob ein TT-Cutoff durchgeführt wird, wird der in der Transpositionstabelle gespeicherte beste Zug als Startwert für den Move Picker (s. Abschnitt 5.7.4) verwendet. Dadurch profitiert die Zugsortierung von den Ergebnissen vorheriger Iterationen des Iterative Deepening (s. Abschnitt 5.5). Da es sich um den besten Zug aus einer vorherigen, flacheren Suche handelt, ist er mit hoher Wahrscheinlichkeit auch in der aktuellen, tieferen Suche der beste Zug. Dies verbessert die Effektivität der Alpha-Beta-Suche erheblich, da ein frühes Finden des besten Zuges zu mehr Alpha-Beta-Cutoffs führt [36].

## 5.7 Zugsortierung

Die Zugsortierung bildet die Brücke zwischen Zuggenerierung und Alpha-Beta-Suche. Wie in Abschnitt 5.2 motiviert, ist die Zugreihenfolge elementar wichtig für den Effekt der Alpha-Beta-Optimierung. Edwards und Hart behaupten in ihrer Präsentation der Optimierung, dass eine perfekt sortierte Liste das exponentielle Wachstum des Suchbaums halbieren kann [31, p. 3]. In diesem Abschnitt werden wir einige häufig eingesetzten Heuristiken präsentieren, um generierte Züge zu sortieren, um die Anzahl der von der Alpha-Beta-Suche betrachteten Knoten so weit wie möglich zu minimieren.

### 5.7.1 MVV-LVA

Die Most Valuable Victim - Least Valuable Aggressor (MVV-LVA) Heuristik befasst sich spezifisch mit schlagenden Zügen. Sie folgt dem simplen Prinzip des maximalen Materialgewinns: Eine Dame mit einem Bauern zu schlagen ist wahrscheinlich ein besserer Zug, als einen Bauern mit einer Dame zu schlagen (vgl. Tabelle 1). Die Züge werden sortiert nach der Differenz des Wertes des Aggressors (die schlagende Figur) und des Opfers (die geschlagene Figur) - Der nach dieser Differenz beste Zug ist demnach der Zug mit dem am wenigsten wertvollen Aggressor und dem wertvollsten Opfer [38, p. 42]. Daher rührt der Name der Heuristik. Implizit enthält MVV-LVA auch die heuristische Annahme, dass es meistens besser ist, eine Figur zu schlagen, als dies nicht zu tun, da Material der ausschlaggebendste Faktor in der Evaluation ist. Jeder nicht-schlagende Zug wird deswegen mit einem Score versehen, der kleiner ist als der schlechteste schlagende Zug, sodass stets schlagende Züge vor nicht-schlagenden betrachtet werden. Zu beachten ist, dass die Heuristik eingeschränkt ist für den Fall, dass die geschlagene Figur geschützt ist. Einen Bauern mit einer Dame zu schlagen, wenn der Bauer geschützt ist, ist beinahe nie ein guter Zug; da die Heuristik diesen dennoch vor ruhigen Zügen wertet, kann dies Rechenzeit kosten.

### 5.7.2 Killer Heuristics

Die Killer-Heuristik beruht auf einem einfach zu beschreibenden Prinzip: Ein Zug, der in einer Position sehr gut ist, ist wahrscheinlich auch in ähnlichen Positionen gut [39, p. 95]. Als Beispiel kann folgende konstruierte Position dienen:

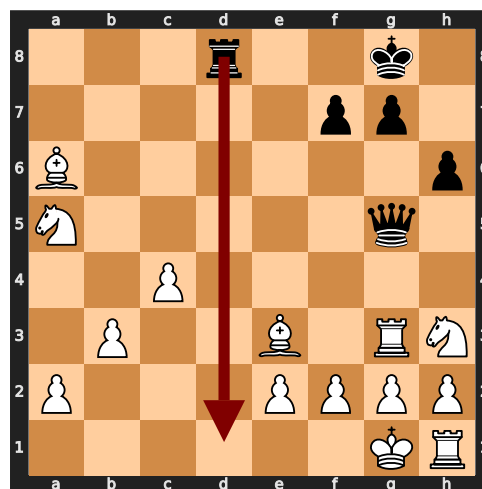


Abbildung 17: Beispielposition für Killer moves, Weiß am Zug

Weiß hat in dieser Position 25 Züge<sup>5</sup>. Allerdings stellt Schwarz eine essentielle Bedrohung: **Rd1#** (in Abbildung 17 rot eingezeichnet). Von den 25 möglichen Zügen für Weiß gibt es genau drei Züge, die dieses Matt durch **Rd1** verhindern: **f4**, **Bd2** und **f3**<sup>6</sup>. Da **Rd1#** 22 weiße Züge sofort widerlegt (gewissermaßen „tötet“), ist **Rd1#** ein Killer Move. Solche Züge werden durch die Killer-Heuristik durch eine Art Cache erkannt. Der Ablauf hierbei ist folgendermaßen:

1. Weiß macht einen schlechten Zug (bspw. **Bxg5**)
2. Auf der nächsten Suchtiefe werden alle schwarzen Züge in unsortierter Reihenfolge betrachtet.
3. **Rd1#** wird gefunden. Die resultierende Position wird mit dem Matt-Score von  $-\text{MATE}$  bewertet.
4. Die Suche wird durch diese extreme Bewertung nach dem Prinzip des Alpha-Beta-Pruning abgebrochen, da  $-\text{MATE} < \alpha$ . Der Zug **Rd1**, der zu dieser Evaluation geführt hat, wird als Killer Move im Cache gemerkt.
5. Weiß sucht in der Ausgangsposition einen weiteren schlechten Zug (bspw. **Nxg5**)
6. Auf der nächsten Ebene wird der gespeicherte Killer Move **Rd1** (sofern er auch in dieser Position legal ist) als erstes betrachtet. Wieder wird die Position mit  $-\text{MATE}$  bewertet. Alle weiteren schwarzen Züge werden nicht durchsucht.

Technisch lässt sich die Killer-Heuristik effizient durch ein Array mit fester Größe gleich der maximalen Suchtiefe implementieren, sodass Killer Moves in konstanter Zeit geschrieben und gelesen werden können. Dadurch kann auf jeder Suchtiefe genau ein Killer Move gespeichert werden - alte Killer Moves werden überschrieben.

Im Zuge der Studienarbeit wurde experimentell getestet, ob es eine Verbesserung darstellt, jeder Suchtiefe Kapazität für zwei Killer Moves zu geben, da sich dies in verschiedener Literatur findet [34, ch. 3.3], [39, p. 95]. Allerdings konnte für diesen Ansatz keine statistisch signifikante Verbesserung in Spielstärke festgestellt werden.

### 5.7.3 History Heuristics

Auf einem sehr ähnlichen Prinzip wie die Killer-Heuristik basiert auch die History Heuristic, erstmals präsentiert in 1988 von Jonathan Schaeffer [40]. Anders als bei

---

<sup>5</sup>a2a3, f2f3, b3b4, c4c5, a2a4, f2f4, h3g5, h3f4, a5c6, a5b7, e3g5, e3c1, e3d2, e3d4, e3f4, e3c5, e3b6, e3a7, a6b5, a6b7, a6c8, g3g5, g3f3, g3g4, g1f1

<sup>6</sup>Angemerkt sei, dass **f3** dennoch eine sichere Niederlage ist: 1. **f3 Qxe3+** 2. **Kf1 Rd1#**

der Killer-Heuristik wird hier aber nicht ein besonderer Zug pro Tiefe betrachtet, sondern alle möglichen Züge werden über ein gesamtes Spiel hinweg bewertet. Der Grundgedanke ist folgender: Je öfter ein Zug als der beste Zug in einer Position identifiziert wird, desto besser ist der Zug prinzipiell. Hierfür werden zwei Tabellen mit  $64 \times 64$  Einträgen verwendet, die den History-Wert jedes Zuges speichern. Zwei Tabellen werden verwendet, da die Werte für Weiß und Schwarz getrennt behandelt werden. Ein Feld der Tabelle repräsentiert einen möglichen, potentiell pseudo-legalen<sup>7</sup> Zug.

Die History Tabellen werden, ähnlich wie die Transpositionstabelle, im Laufe eines Spiels aufgebaut und persistieren über einzelne Suchen hinweg, um das Wissen aus vorherigen Suchen nicht zu verlieren. Allerdings werden auch hier die Einträge gealtert [42], in diesem Fall vor jeder neuen Suche. Aktualisiert werden die Werte in jedem besuchten Knoten. Hierbei gilt folgendes Prinzip:

1. Ist der beste Zug ein nicht-schlagender Zug: Inkrementiere den History-Wert nach Gravity Formula (Gleichung 6)
2. Für alle ruhigen Züge, die vor dem besten Zug angeschaut werden: Bestrafe den Zug für „Maluse“ nach Gravity Formula

Die „Gravity Formula“ ist eine Formel zum aktualisieren von History-Werten, die sich in modernen Engines etabliert hat. Folgendes Exemplar, welches auch in Thunfisch verwendet wird, ist aus Viridithas [15] übernommen:

$$h_{\text{neu}} = h_{\text{alt}} + b - h_{\text{alt}} \cdot \frac{|b|}{\text{MAX}} \quad ; \text{ mit } b = \begin{cases} 230+355d & ; \text{ if bonus} \\ -515-110d & ; \text{ if maluse} \end{cases}$$

Gleichung 6: Gravity Formula zur Aktualisierung von History-Werten

Zu erkennen ist, dass der History Bonus linear abhängig ist von der Suchtiefe. Da  $d$  in der Alpha-Beta-Suche eine dekrementierende Variable ist, ist der Bonus hierbei stärker, je früher in der Suche der Zug betrachtet wird. Die Formel gewichtet außerdem den aktuellen History-Wert des Zuges: hat dieser einen bereits hohen History-Wert, fällt der Bonus kleiner aus. Es werden unterschiedliche Boni verwendet für Inkrement

---

<sup>7</sup>Durch die Anatomie der Tabellen, die das gesamte Schachbrett abbilden, wird zwangsläufig Platz für illegale Züge allokiert. Beispielsweise hat ein Zug von **a1** nach **b8** einen Eintrag in dieser Tabelle, allerdings kann dieser Zug in keiner Position legal sein. [41] zeigt, dass nur 44% aller möglichen Züge in einer solchen Tabelle tatsächlich pseudo-legal sein können; dennoch ist generell akzeptiert, dass der zusätzliche Aufwand bei der Indexierung, der mit einer Platzreduktion einherginge, nicht lohnenswert ist.

und Dekrement für Maluse, wobei beide auf ein hier der Übersichtlichkeit halber nicht erwähntes Maximum eingeschränkt werden.

Schaeffer beobachtet, dass die Heuristik nur merkliche Auswirkungen bei der Sortierung von ruhigen Zügen hat. Allerdings liefert sie für diese ruhigen Züge eine kosteneffiziente Heuristik zur Sortierung, die laut Schaeffer selbst ausführliche Domänenwissen-basierte Sortierungen schlägt [40].

#### **5.7.4 Move Picker**

Durch das intelligente Sortieren von Zügen ergibt sich eine Problematik: Gerade durch beispielsweise die Killer-Heuristik, die Transpositionstabelle oder die Tatsache, dass schlagende Züge häufig besser sind als ruhige Züge, befindet sich der beste Zug sehr häufig bereits unter den ersten 5 Zügen<sup>8</sup>. In einer durchschnittlichen Position mit 30 Zügen bedeutet dies, dass über 80% der Züge zwar generiert, aber aufgrund eines Alpha-Beta-Cutoffs durch den Besten Zug nie betrachtet werden. Trotz der Effizienz durch die Nutzung von Bitboards ist die Zuggenerierung teuer, und unnötige Generierungen zu verhindern kann die Suchgeschwindigkeit erheblich erhöhen.

Der Move Picker verfolgt den Ansatz, die Auswahl und Generierung von Zügen in Schritten zu staffeln. Thunfisch greift diese Idee von Viridithas auf [15] und implementiert sie analog. Allerdings findet sich der Ansatz auch in anderen bekannten Engines [12], [43], [44]. Der Grundgedanke des Move Pickers ist es, nicht alle legalen Züge auf einmal zu generieren und die resultierende Liste nach den genannten Heuristiken zu sortieren, sondern die Auswahl von Zügen in Phasen aufzuteilen. Abbildung 18 zeigt in einem Zustandsdiagramm die einzelnen Phasen des Move Pickers auf:

---

<sup>8</sup>Diese Aussage ergibt sich anekdotisch aus Debug-Ausgaben von Thunfisch, nachdem die beschriebenen Sortierungsverfahren implementiert wurden

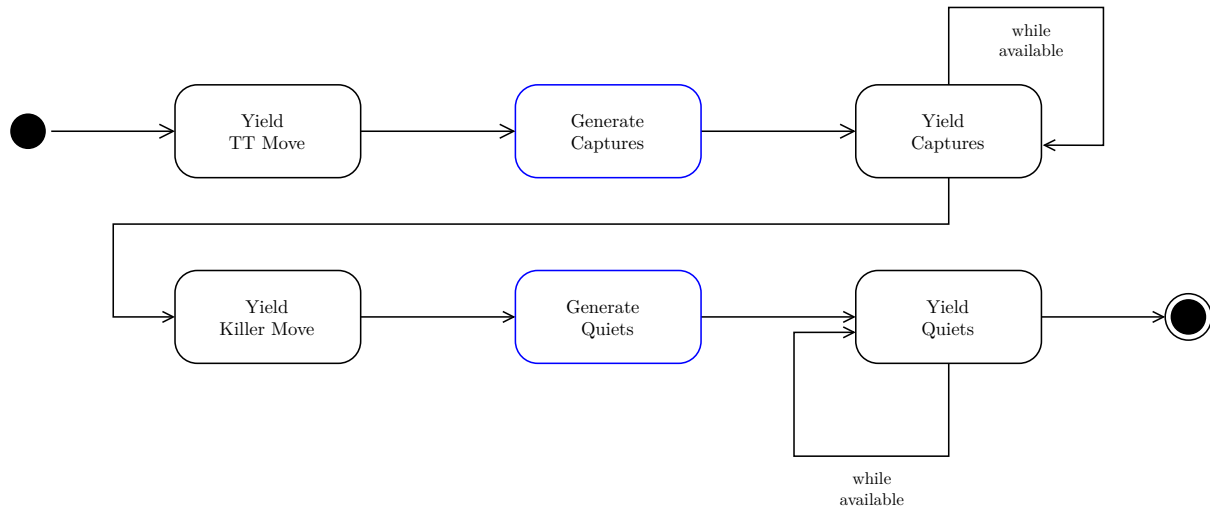


Abbildung 18: Zustandsdiagramm des Move Pickers

Wie zu sehen ist, wird die Generation von schlagenden (Captures) und ruhigen Zügen (Quiets) aufgeteilt. Durch diese Trennung wird verhindert, dass ruhige Züge generiert werden, wenn ein schlagender Zug bereits einen Alpha-Beta-Cutoff verursacht. Diese Phasen sind die einzigen, in denen die Zuggenerierung abläuft, und zur Akzentuierung blau hervorgehoben. In allen anderen Zuständen ist die Aufgabe des Move Pickers, den nächstbesten Zug an die Suche zu übergeben. Dies geschieht durch eine Funktion `MovePicker::next() -> Option<EncodedMove>`, die als standardisiertes Interface agiert, und als Nebeneffekt den Zustand des Move Pickers steuert. Im Falle des Zuges aus der Transpositionstabelle und des Killer Moves ist dies einfach: Der Picker überprüft, ob diese Züge jeweils hinterlegt sind, und falls ja, ob dieser in der aktuellen Position legal ist. Ist eines davon nicht der Fall, geht der Move Picker leise in den nächsten Zustand über.

Für die Zustände „Yield Captures“ und „Yield Quiets“, deren Aufgabe es ist, die generierten schlagenden und ruhigen Züge an die Suche zu übergeben, ist mehr Logik erforderlich. Da das `MovePicker::next()` Interface so aufgebaut ist, dass der Move Picker stets den nächsten Zug zurückgibt, wird in diesen Zuständen aus der Liste an generierten Zügen der nächstbeste Zug gesucht. Hierfür wird ein Insertion Sort verwendet, bei dem für jeden Zug eine Iteration durchgeführt wird. Um zu wissen, welche Züge der Liste nicht mehr zu betrachten sind, verwendet der Move Picker eine Zustandsvariable `move_index`, die den Start der unsortierten restlichen Züge markiert. Listing 8 zeigt die Implementierung hiervon in Thunfisch, welche stark an die Implementierung in Viridithas [15] angelehnt ist.

```

1  pub fn yield_next_best_move(&mut self) -> Option<EncodedMove> { Rust
2      loop {
3          let remaining = &mut self.move_list.list[self.move_index..];
4
5          if remaining.is_empty() {
6              return None;
7          }
8          // Der beste Zug wird durch eine Lineare Suche gefunden.
           // Durch das Tauschen mit Index 0 wird diese Suche zum Teil
           // eines In-Place Insertion Sorts
9          if let Some((best_idx, _)) =
            remaining.iter().enumerate().max_by_key(|(_, m)| m.score) {
10             remaining.swap(0, best_idx);
11         }
12         // Killer Move und Zug aus der TT werden übersprungen
13         let best_move = remaining[0].mv;
14         self.move_index += 1;
15         if Some(best_move) == self.tt_move
16             || Some(best_move) == self.killer_mv
17         {
18             continue;
19         }
20         // Bester Zug wird zurückgegeben
21         return Some(best_move);
22     }
23 }

```

Listing 8: Schrittweiser Insertion Sort zum Identifizieren des nächstbesten Zuges

## 5.8 Forward Pruning

Unter Forward Pruning versteht man den Prozess, Pfade des Suchbaums auf Basis heuristischer Annahmen zu kürzen. Anders als bei dem Verwurf eines Pfades in der Alpha-Beta-Suche ist hierbei keine beweisbare Garantie vorhanden, dass der verworfene Pfad tatsächlich der aktuellen PV unterlegen ist; deshalb wird mit dem Verwerfen

des Pfades stets ein Risiko eingegangen, dass der verworfene Pfad doch ein Ergebnis innerhalb des Intervalls  $[\alpha, \beta]$  liefert [34]. Ist die Heuristik allerdings stark genug, kann argumentiert werden, dass ein gelegentliches Übersehen des besten Zuges einer Position in der Spielstärke vernachlässigbar ist relativ zu dem Anstieg in Suchgeschwindigkeit, der aus der Verkleinerung des Suchbaums hervorgeht [34, ch. 3.2].

Thunfisch implementiert zwei konkrete Instanzen von Forward Pruning, in erster Linie aus Gründen der Einfachheit und Intuitivität. Bei den beiden Verfahren handelt es sich um Null Move Pruning (NMP) und Reverse Futility Pruning (RFP), die im Folgenden näher erläutert werden. Um die Auswirkungen eines Übersehens des besten Zuges klein zu halten, wendet Thunfisch diese Techniken nur auf Knoten an, die von der Principal Variation Search als abseits der Principal Variation kategorisiert werden.

**Null Move Pruning** basiert auf der Annahme, dass es in der Regel von Nachteil ist, den Gegner zweimal direkt hintereinander ziehen zu lassen. Wenn man also in einer Position seinen Zug abgeben kann, und der Gegner davon nicht profitieren kann, ist dies gewissermaßen „zu schön, um wahr zu sein“, und der aktuelle Suchpfad kann verworfen werden. Dieses Abgeben eines Zuges wird programmatisch repräsentiert durch einen Zug wie `a1a1`, also einen Zug, der das Brett nicht verändert, genannt **Null Move**. Ob „der Gegner profitieren“ kann, wird programmatisch durch das  $[\alpha, \beta]$  Intervall festgelegt: Produziert der Teilbaum, der nach dem Abgeben des Zuges entsteht, einen gesuchten Evaluationswert  $v_{k+1} > \beta$ , bedeutet dies nach den Regeln der Alpha-Beta-Suche, dass es eine bessere Variation für den Gegner gibt, als eine Variation in der er zweimal hintereinander ziehen darf. Dies berechtigt das Verwerfen des Pfades. Ist  $v_{k+1} < \beta$ , muss der Knoten hingegen vollständig mit allen Zügen durchsucht werden. Aus diesem Grund wird die Null Move Suche kleiner gehalten, um sie so effizient wie möglich zu gestalten. Gesucht wird mit Zero Window und reduzierter Tiefe [45].

**Reverse Futility Pruning** manifestiert Vertrauen in die Evaluationsfunktion der Engine: Wenn die statische Evaluation eines Knotens gut genug ist, wird nicht weiter gesucht. Hintergrund hierfür ist die Annahme, dass sich die statische Evaluation zwischen Positionen von Zug zu Zug bei perfektem Spiel kaum verändert. RFP prüft, ob die statische Evaluation der aktuellen Position  $\text{eval}(k)$  höher ist als  $\beta + m$ , wobei  $m$  eine Sicherheitsmarge ist. Ist  $\text{eval}(k)$  ausreichend hoch, nimmt RFP an, dass sich die statische Evaluation nicht weit genug nach unten bewegen kann, um innerhalb des

Suchfensters zu landen, und verwirft den Pfad. Die Marge verringert sich linear mit der Suchtiefe, sodass die Hürde bei frühen Positionen höher ist als bei Positionen, die tiefer im Baum liegen [15].

## 5.9 Late Move Reductions

Late Move Reductions (LMR) kürzen den Suchbaum durch das Variieren der Suchtiefe jeder Kindposition. Der motivierende Gedanke ist derselbe wie für die Principal Variation Search: Es wird davon ausgegangen, dass eine hinreichend gute Zugsortierung den besten Zug in einer Position weit nach vorne sortiert, sowie schlechtere Züge weit hinten sortiert werden. Anstatt auch die wahrscheinlich schlechteren Züge auf vollständiger Tiefe zu durchsuchen, wird die Suchtiefe für diese hinteren Züge reduziert [45, ch. 2.3]. Stellt einer dieser reduzierten Suchen eine Verbesserung der Evaluation über  $\alpha$  dar, muss der Zug noch einmal mit vollständiger Tiefe durchsucht werden.

### 5.9.1 Festlegen der Reduzierung

Um wie viel die Suche eines Zuges reduziert wird, hat starke Einflüsse auf die Performanz von LMR. Grundsätzlich ist der Ansatz etabliert, eine Basisreduktion abhängig von Tiefe  $d$  und Sortierungsposition  $s_{mv}$  des Zuges zu errechnen, und diese Reduktion anschließend anhand weiterer, Domänenwissen-basierter Heuristiken anzupassen. Unter den führenden Engines gibt es verschiedene Funktionen zum Errechnen dieser Basisreduktion ([15], [43], [44]), allerdings teilen alle die Gemeinsamkeit der Nichtlinearität bezüglich sowohl der Suchtiefe  $d$  als auch  $s$ . Thunfisch übernimmt seine Funktion der Engine Weiss [44]<sup>9</sup>:

$$r_d(mv) = \left\lfloor 1.35 + \ln(d) \cdot \frac{\ln(s_{mv})}{2.75} \right\rfloor$$

Gleichung 7: Funktion der Basisreduktion eines Zuges in LMR

Anhand Abbildung 19 können einige Eigenschaften dieser Funktion erkannt werden, die sich mit den zentralen Ideen von LMR decken. Erkennbar ist, dass  $r_d$  stets logarithmisch wächst. In der durchschnittlichen Schachposition gibt es ca. 30 legale Züge.  $r_{d(30)}$  ist bereits hoch genug, dass durch die Reduktion eine signifikante Anzahl an Knoten entfällt. Gleichzeitig wird sichergestellt, dass in Positionen mit sehr vielen Zügen, auch

---

<sup>9</sup>Weiss nutzt verschiedene Formeln für schlagende und ruhige Züge. Der Einfachheit halber generalisiert Thunfisch die letztere.

die hintersten Züge noch ausreichend tief durchsucht werden, um relevante Information zu gewinnen.

Auch ist erkennbar, dass die Funktion höhere Werte für höhere Tiefen liefert. Dies bedeutet, dass früher im Baum stärker, und nahe der Blätter weniger stark reduziert wird.

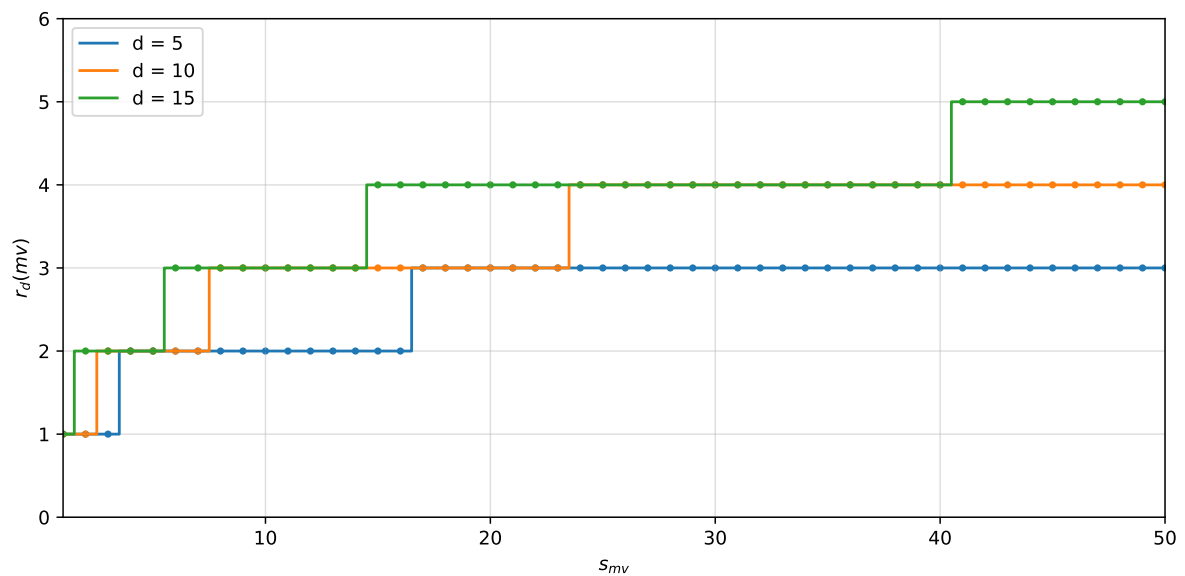


Abbildung 19: Visuelle Darstellung von Gleichung 7

Thunfisch reduziert aktuell nur nach dieser Funktion. Im Laufe der Entwicklung wurden einige Domänenwissen-basierte Reduktionen experimentell angewandt, führten aber zu keinem messbaren Resultat.

### 5.9.2 LMR in Kombination mit PVS

Aufgrund ihrer Ähnlichkeit in Motivation sind PVS und LMR gemeinsam anwendbar, um im Worst Case maximal drei Suchen pro Zug durchzuführen. Hierbei wird auf Positionen abseits der PV folgende Reihenfolge verwendet [15]:

1. Suche mit reduzierter Tiefe und Zero Window
2. Suche mit vollständiger Tiefe und Zero Window
3. Suche mit vollständiger Tiefe und vollständigem Fenster

Für Positionen auf der PV wendet Thunfisch LMR nicht an.

## 6 Positionsevaluation

In Abschnitt 5.1 zur Minimax-Suche wurde bereits eine statische Evaluationsfunktion `eval` motiviert. Ihre Rolle ist es, die Positionen an den Blattpositionen ohne weitere Suche zu bewerten und einen numerischen Wert zurückzugeben. Die im Suchbaum propagierten Werte  $v_k$  entstehen erst durch die Rekursion der Alpha-Beta-Suche aus diesen statischen Blattbewertungen. Da Entscheidungen in der Alpha-Beta-Suche stets auf diesen Evaluationen beruhen, ist die statische Evaluationsfunktion von zentraler Bedeutung für die Güte der Suche.

Die statische Evaluationsfunktion kann gesehen werden als eine heuristische Schätzung über den Ausgang des Spiels ausgehend von der evaluierten Position. Das Vorzeichen ihrer Ausgabe gibt an, welcher der beiden Spieler nach dieser Schätzung das Spiel gewinnen wird. Im professionellen Schachspiel hat sich etabliert, dass eine positive Bewertung einen Vorteil für Weiß bedeutet und eine negative Bewertung einen Vorteil für Schwarz; im Kontext der Alpha-Beta-Suche ist dadurch Weiß der maximierende Spieler und Schwarz der minimierende. Der Betrag der statischen Evaluation gibt an, wie stark der Vorteil ist; eine Position  $k_0$  mit  $|\text{eval}(k_0)| = 10$  beispielsweise ist für den besseren Spieler deutlich einfacher zu gewinnen, als eine Position  $k_1$  mit  $|\text{eval}(k_1)| = 1$ . Es gibt drei besondere Werte, die die statische Evaluationsfunktion annehmen kann [1, ch. 4]:

1.  $\text{eval}(s) = -\text{MATE}$ : Eine minimale statische Evaluation bedeutet, dass Weiß schachmatt ist. Demnach hat Schwarz gewonnen.
2.  $\text{eval}(s) = 0$ : Die Position ist exakt ausgeglichen. Ein Beispiel hierfür ist eine Position, die nur noch Könige enthält - hier ist es theoretisch unmöglich, dass eine der beiden Seiten gewinnt.
3.  $\text{eval}(s) = \text{MATE}$ : Eine maximale statische Evaluation bedeutet, dass Schwarz schachmatt ist. Demnach hat Weiß gewonnen.

Der Wert `MATE`, der eine Schachmattsituation beschreibt, ist abhängig von der Implementierung. Wichtig zu beachten ist hierbei nur, dass der Wert weit über jeder noch so schlechten anderen Position liegen muss, um zu vermeiden, dass die

Engine sichere Niederlagen einer schlechten, aber nicht zwangsläufig verlorenen Position bevorzugt. Thunfisch verwendet  $MATE = 3 \cdot 10^4$ .

## 6.1 Material

Material ist die einfachste Form der Evaluierung und gleichzeitig eine der mächtigsten. Dabei handelt es sich um nichts weiteres als das Abgleichen, wie viele Figuren welcher Art jede Seite noch besitzt. Hierbei werden Figuren nach ihrer Stärke gewichtet; es ist trivial, dass eine Dame eine wertvollere Figur ist als ein Bauer. Schachanfängern wird häufig folgende Richtlinie für den Wert der Figuren relativ zueinander gegeben [1, ch. 3]:

| Figur | Bauer | Springer | Läufer | Turm | Dame | König    |
|-------|-------|----------|--------|------|------|----------|
| Wert  | 1     | 3        | 3      | 5    | 9    | $\infty$ |

Tabelle 1: Wert von Schachfiguren relativ zueinander

Die einfachste aller Evaluierungsfunktionen besteht demnach daraus, das Material beider Spieler gewichtet aufzusummieren und die Summe des Materials des Schwarzen Spielers von dem des Weißen Spielers abzuziehen. Anzumerken hierbei ist, dass die Gewichtung der Figuren meist nicht exakt dieser simplen Tabelle entspricht. IM Larry Kaufman zeigte 1999 experimentell, dass sich in realem Spiel der Wert von Figuren dynamisch unterscheidet [46]. Viele der bekannten Schach-Engines benutzen andere Werte [12], [15]. Thunfisch benutzt aktuell Werte, die ebenso wie die im Folgenden erläuterten Piece Square Tables von Ronald Friedrich [47] übernommen wurden.

## 6.2 Piece Square Tables

### 6.2.1 Theoretischer Hintergrund

Material alleine ist allerdings noch lange nicht ausschlaggebend. Oft ändert sich der relative Wert einer Figur radikal basierend darauf, wo auf dem Brett sie sich befindet. Ein intuitives Beispiel ist der Bauer: Bauern können zu einer beliebigen anderen Figur umgewandelt werden, wenn sie die gegnerische Grundlinie erreichen. Aus diesem Grund kann ein Bauer, der nur noch einen Zug von der Promotion entfernt ist, als bedeutend wertvoller betrachtet werden als ein Bauer, der noch auf seiner Startposition steht.

Ähnliche, wenn auch weniger extreme Fälle, lassen sich für jede Figur finden. Ein bekanntes Sprichwort unter Schachspielern, „Ein Springer am Rande bringt Kummer und Schande“, illustriert beispielsweise das Konzept, dass ein Springer im Zentrum des Bretts mehr Kontrolle ausübt als am Rand und demnach wertvoller ist.

Zusätzlich ergibt sich eine Herausforderung dadurch, dass sich im Laufe des Spiels die optimale Position einer Figur verändert. Als Beispiel soll der König dienen: Je mehr Figuren auf dem Brett sind, desto größer ist die Gefahr für ein Matt. Aus diesem Grund sollte der König während Eröffnung und Mittelspiel gut geschützt werden.

### 6.2.2 Umsetzung

In der Praxis werden diese strategischen Aspekte einer Position in Engines umgesetzt durch Masken, die auf ein Schachbrett gelegt werden. Dabei wird jedem Feld auf dem Brett ein Koeffizient gegeben, der mit dem Wert einer Figur verrechnet wird, wenn sie sich auf diesem Feld befindet. In 1968 motivierte Jack Good einen solchen Ansatz [39, ch. 3.2]. In modernen Engines ist es üblich, eigene Piece Square Tables (PSQTs) für jede Art von Figur zu haben. Abbildung 20 soll als visuelles Beispiel einer PSQT dienen. Es handelt sich um eine simple PSQT (entnommen aus [48]) für die Figur des Bauern. In ihr ist das oben genannte Konzept der Werte der Bauern wiederzufinden: Die siebte Reihe wird komplett mit einem Koeffizienten von 7 belegt, die zweite jedoch nur mit dem Koeffizienten 2.

|   | a | b | c | d | e | f | g | h |   |
|---|---|---|---|---|---|---|---|---|---|
| 8 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 8 |
| 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 | 7 |
| 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 | 6 |
| 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 | 5 |
| 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 | 4 |
| 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 | 3 |
| 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 | 2 |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
|   | a | b | c | d | e | f | g | h |   |

Abbildung 20: Beispiel für eine simple PSQT für Bauern

Bei der Evaluation wird für jede Figur auf ihren Materialwert der Koeffizient addiert für das Feld, auf dem sie sich befindet. Diese modifizierten Materialwerte werden dann wie in der in Abschnitt 6.1 beschriebenen Evaluationsfunktion verrechnet, um die finale Evaluation zu ergeben. Um der zuvor beschriebenen Herausforderung durch sich ändernde Rollen von Figuren abhängig von der Spielphase gerecht zu werden, werden allerdings nicht nur eine, sondern zwei Masken verwendet: eine für das Mittelspiel und eine für das Endspiel. Um einen fließenden Übergang zu gewährleisten, wird zwischen den beiden Masken interpoliert. Diese Interpolation erfolgt linear abhängig davon, wie viel Material (ausgenommen Bauern) sich insgesamt noch im Spiel befindet [47]. Abbildung 21 zeigt diese Interpolation visuell aufbereitet für den König. Zu sehen sind die Mittel- und Endspielmasken, in denen sich die verschiedenen Rollen des Königs wiederfinden lassen: Die Mittelspielmaske bestraft einen zentralen König, während die Endspielmaske einen solchen positiv bewertet. Die Interpolation zeigt den Fall, dass sich noch genau die Hälfte des ursprünglichen Materials im Spiel befindet.

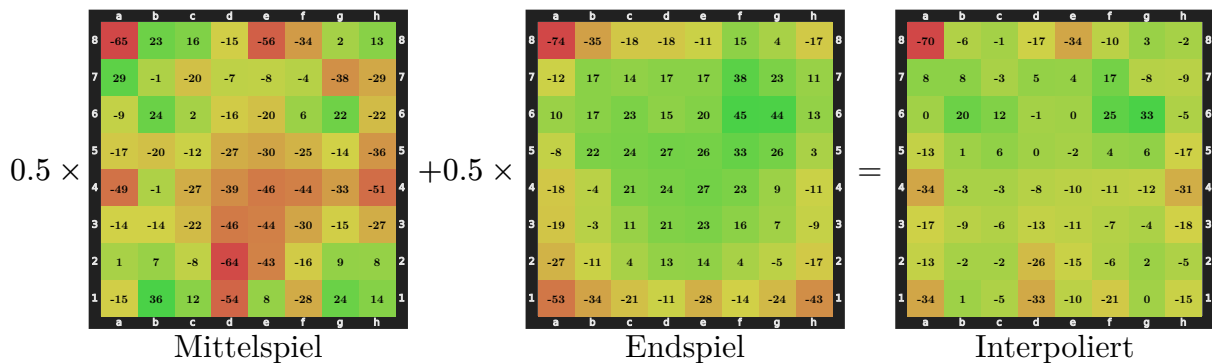


Abbildung 21: Interpolation zwischen Mittel- und Endspielmasken

Als Optimierung werden zur Kompilierzeit alle Masken initialisiert und in einer statischen Tabelle gespeichert, sodass der Wert für jede Figur mit einem  $\mathcal{O}(1)$  Lookup erhalten werden kann.

Als Ausgangspunkt übernimmt Thunfisch seine PSQTs von Ronald Friedrichs Engine Rofchade und PeSTO [47]. Diese Tabellen gelten in der Community als guter Ausgangspunkt für eine Evaluation, die ausschließlich auf PSQTs basiert und noch keine weiteren Terme einführt. Aufgrund ihrer Herkunft sind diese Tabellen als PeSTO-Tabellen bekannt. Thunfisch entfernt sich von PeSTO-Tabellen durch späteres autonomes Tuning (s. Abschnitt 6.4), um die PSQTs besser in Einklang mit den eingeführten fortgeschrittenen Termen zu bringen.

## 6.3 Weitere Terme der Evaluation

Abseits der simplen Piece Square Tables gibt es weitere Aspekte, die in die statische Evaluation einer Position einfließen können. Das Ziel hierbei ist, der Engine ein besseres Verständnis für strategische Aspekte einer Position zu geben, analog dazu, wie die besten menschlichen Schachspieler dies tun. Die Entscheidung, welche Aspekte in Thunfisch implementiert wurden, wurde anhand manueller Überprüfung von Spielen getroffen, die von Thunfisch autonom auf der Plattform Lichess gegen andere Engines gespielt wurden. Uns sind dabei drei prominente strategische Schwachstellen in Thunfischs Spiel aufgefallen:

1. Eine Tendenz, gegnerische Angriffe auf den eigenen König zu spät als Bedrohung wahrzunehmen
2. Eine Tendenz, dem Gegner Freibauern zu überlassen, und Endspiele wegen dieser Bauern zu verlieren
3. Eine Tendenz, eigene Bauern zu doppeln

In diesem Kapitel wollen wir darauf eingehen, wie diesen Schwachstellen durch neue Terme in der Evaluationsfunktion entgegengewirkt werden kann.

### 6.3.1 Freibauern

Als Freibauer wird ein Bauer bezeichnet, wenn er von keinem gegnerischen Bauern daran gehindert werden kann, die Grundreihe zu erreichen. Freibauern sind im Endspiel besonders potent, da ihre Umwandlung schwer zu stoppen ist; nicht selten muss bei perfektem Spiel eine Leichtfigur für einen Freibauern geopfert werden.

Ob ein Bauer gestoppt werden kann, ist mithilfe von Bitboards einfach festzustellen: Für jeden Bauern muss eine Maske generiert werden, die alle Felder vor dem Bauern auf derselben Linie sowie auf der Linie links und rechts daneben beinhaltet. Die Grundreihe selbst muss nicht beinhaltet werden, da hier keine Bauern stehen können. In Abbildung 22 ist jeweils die Maske für den weißen Bauern auf e4 (grün) und den schwarzen Bauern auf d4 (rot) beispielhaft aufgeführt:

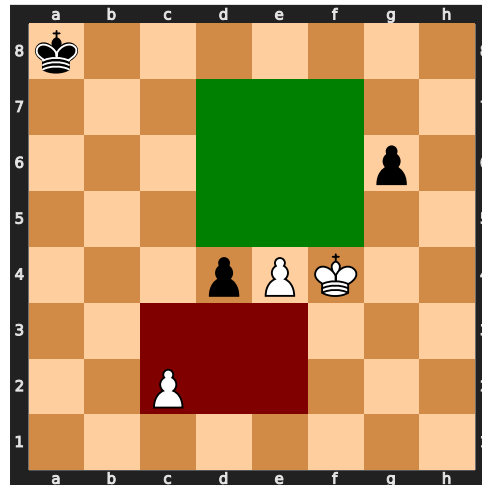


Abbildung 22: Exemplarische Masken zur Erkennung von Freibauern

Diese Maske wird mit dem Bitboard der gegnerischen Bauern verundet. Der Bauer ist ein Freibauer gdw. das resultierende Bitboard leer ist. In Abbildung 22 ist dies für den Bauern auf e4 der Fall; der Bauer auf d4 wird durch den Bauern auf c2 blockiert.

Als Inspiration für die Implementierung der Maske dient Sebastian Lagues Schach-Engine [49].

### 6.3.2 Doppelbauern

Unter einem Doppelbauern versteht man eigentlich zwei Bauern: Zwei Bauern derselben Farbe sind genau dann gedoppelt, wenn sie auf derselben Linie stehen. Doppelbauern werden prinzipiell als Schwäche gesehen, da sie sich gegenseitig in ihrer Mobilität einschränken sowie sich gegenseitig nicht unterstützen können. Diese Kombination macht Doppelbauern häufig zu einfachen Zielen für den Gegner, um einen Materialgewinn zu erzielen.

Auch Doppelbauern sind durch Bitboards einfach zu erkennen. Das Bitboard der eigenen Bauern wird für jede Linie mit einer Maske verundet, die nur die aktuelle Linie repräsentiert. Auf dem Ergebnis  $b$  kann durch die Operation  $b \& (b - 1)$  das niedrigste Bit umgekehrt werden. Ist die resultierende Maske leer, befindet sich maximal ein Bauer auf der Linie. Falls nicht, befinden sich mindestens zwei Bauern auf der Linie. Die Anzahl der gesetzten Bits kann gezählt und mit einem Strafwert multipliziert werden, um gedoppelte Bauern zu bestrafen.

### 6.3.3 Königssicherheit

Königssicherheit ist abstrakter als Frei- und Doppelbauern und nicht formal zu definieren. Professionelle Schachspieler wissen oft intuitiv, ob der eigene König unter signifikanter Gefahr ist oder nicht. Für die algorithmische Erkennung verwendet Thunfisch drei Heuristiken:

1. Bestrafe einen König auf einer offenen Linie
2. Vergib Boni für sogenannte „Bauernschilder“: Bauern, die sich auf den 3 Feldern vor dem König befinden, zu denen er sich bewegen kann
3. Vergib Boni für Figuren, die nahe an den gegnerischen König ziehen können.

Punkt 1 kann analog zu Doppelbauern festgestellt werden: Eine Linie gilt als offen, wenn sich keine Bauern darauf befinden. Punkt 2 ist ebenso durch eine triviale Maske zu erkennen. Für Punkt 3 wird ebenfalls eine Maske definiert, genannt „Königszone“. Diese beinhaltet die Felder, die der König erreichen kann, sowie weitere drei Felder auf der Reihe in Richtung der gegnerischen Seite.

Die Masken für Punkt 2 und 3 sind exemplarisch in Abbildung 23 für Weiß und Schwarz aufgezeigt:

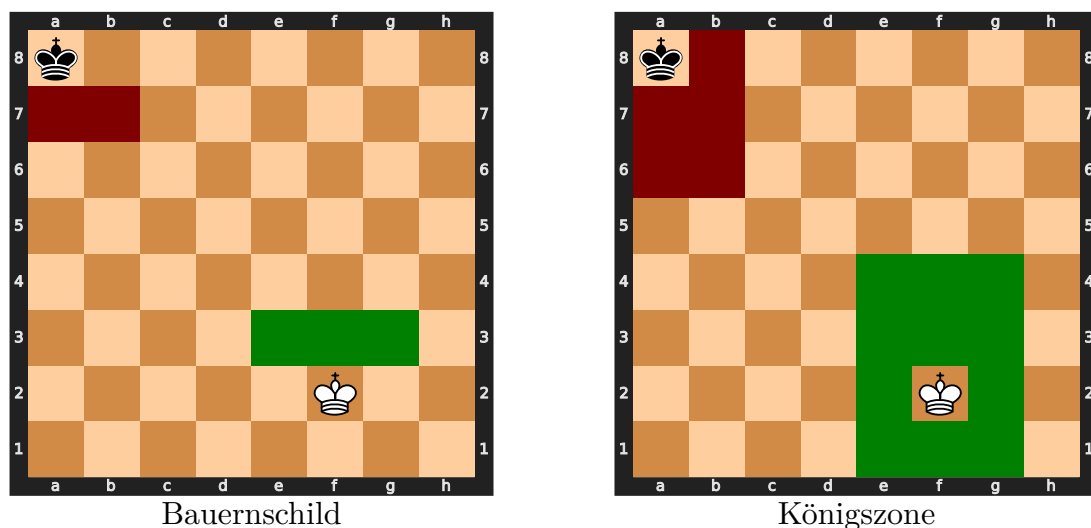


Abbildung 23: Masken für Königssicherheit

Um eine finale Bewertung der Königssicherheit zu erlangen, wird für alle gegnerischen Figuren, gezählt, wie viele Felder innerhalb dieser Zone die Figur erreichen kann. Für jedes Feld, in das die Figur ziehen kann, wird ein Punkt zu einem Safety Score addiert. Die Punkte werden hierbei nach der Art der Figur gewichtet, da eine Dame

beispielsweise eine größere Bedrohung darstellt als ein Läufer. Schlussendlich wird der Safety Score als Index in eine Tabelle genutzt, welche den Safety Score in eine Bewertung nach Centipawns umwandelt. Die hierfür verwendete Tabelle ist aus einer alten Version von Stockfish übernommen [50].

## 6.4 Texel Tuning

Unter „Texel Tuning“ versteht man den Prozess, die Gewichte der Evaluationsfunktion automatisiert zu optimieren. Unter „Gewicht“ versteht man hierbei einen einzelnen Wert, der in irgendeiner Form den Ausgabewert der Evaluationsfunktion beeinflusst. Der Ansatz wurde erstmals motiviert von Peter Österlund in [51], der die Evaluation seiner Engine Texel damit trainierte, was der Methode den Namen gibt. Die Methodik wurde später formalisiert und optimiert durch Andrew Grant in seiner Engine Ethereal [52].

Texel Tuning ist ein überwachtes Lernverfahren, welches auf Basis der Evaluationsfunktion eine Regression durchführt auf der Vorhersage, wie das Spiel ausgehend der gegebenen Position ausgehen wird. Es gibt drei mögliche Ausgänge:

1.  $\omega_1 = \text{Sieg für Schwarz} = 0$
2.  $\omega_2 = \text{Unentschieden} = 0.5$
3.  $\omega_3 = \text{Sieg für Weiß} = 1$

Die entscheidenden Ausgänge grenzen das reale Intervall  $[0, 1]$  ein. Eine Ausgabe  $E$  der Evaluationsfunktion kann auf dieses Intervall normiert werden, um semantisch einer Siegeswahrscheinlichkeit für Weiß zu entsprechen. Peter Österlund definiert hierfür folgenden Sigmoid:

$$\sigma_K(E) = \frac{1}{1 + 10^{\frac{1}{400} K \cdot E}}$$

Gleichung 8: Sigmoid zum Normieren der Evaluation

Dieser Sigmoid wird parametrisiert durch einen Hyperparameter  $K$ : Dieser wird initial aus dem Trainingsdatensatz  $D_{\text{train}}$  geschätzt. Dies geschieht durch eine Grid Search über  $\sigma$ , wobei  $E$  errechnet wird durch die unoptimierte Evaluation.  $K$  wird optimiert, sodass der Mean Square Error ( $MSE$ ) der geschätzten Evaluation gegenüber den gelabelten Daten minimal wird.

Nachdem  $K$  optimiert ist, beginnt das Training der Evaluationsfunktion. Auch hier ist das Ziel,  $MSE(\sigma_K(\text{eval}(D_{\text{train}})))$  zu minimieren. Der Algorithmus zur Optimierung ist für das Resultat nicht ausschlaggebend; Thunfisch nutzt nach Vorbild von Fatalii [53] die Gradient Descent-Variante Adaptive Moment Estimation (ADAM). Hier zeigt sich eine zentrale Herausforderung: Die Berechnung des Gradienten der Loss-Funktion  $MSE$  bezüglich der Evaluationsgewichte,  $\frac{\partial MSE}{\partial w}$ . Grant berechnet in [52] analytische Gradienten und geht auf die Schwierigkeit ein, die durch nichtlineare Komponenten der Evaluationsfunktion entsteht. Auch geht er darauf ein, dass es häufig die Evaluation der Königssicherheit ist, die eine Nichtlinearität der Evaluation verursacht. Fatalii umgeht dieses Problem, indem Königssicherheit in der Evaluation schlichtweg nicht behandelt wird; im Falle Thunfischs ist dies aber Teil der Evaluation und ebenfalls nichtlinear. Aus Gründen der limitierten Zeit und des Ziels, die Königssicherheit ebenfalls zu optimieren, wurde die Entscheidung getroffen, die Gradienten numerisch durch den Differenzenquotienten zu berechnen:

$$\frac{\partial MSE}{\partial w} = \frac{MSE(\text{eval}_{w+1}(D_{\text{train}})) - MSE(\text{eval}_{w-1}(D_{\text{train}}))}{2}$$

Gleichung 9: Differenzenquotient zur numerischen Gradientenberechnung einer Loss-Funktion

Hierbei ist  $\text{eval}_{w\pm 1}$  die Evaluationsfunktion, mit allen Werten um  $\pm 1$  abgeändert.

Der zentrale Nachteil der numerischen Gradientenberechnung zeigt sich in dem Verhalten der Laufzeit: Der Trainingssatz, mit dem Thunfischs Evaluation trainiert wurde, umfasst 1.4 Millionen Positionen. Um alle dieser Positionen einmal zu bewerten, benötigt die Evaluationsfunktion auf einem Intel Xeon Platinum 8176 Chip mit 224 Kernen Parallelität 50 ms. Durch die numerische Gradientenberechnung muss der gesamte Datensatz pro Gewicht 2-mal evaluiert werden, was der Gradientenberechnung die Komplexität  $\mathcal{O}(|w| \cdot |D_{\text{train}}|)$  gibt ( $w$  ist der Vektor aller änderbaren Parameter). Thunfischs Evaluation umfasst zum Zeitpunkt dieser Arbeit 1156 Parameter. Daraus ergibt sich, überschlagen, folgende Dauer pro trainierte Epoche:

$$1156 \cdot 2 \cdot 100 \cdot 10^{-3} s = 231.2 s \approx 3m \ 51s$$

Gleichung 10: Hochrechnung zur Trainingsdauer einer Epoche

Um die Evaluation hiermit für 300 Epochen zu trainieren, werden dadurch beinahe 20 Stunden benötigt. Nach 300 Epochen Training konnte der  $MSE$  auf den Testdaten von  $\approx 0.640$  auf  $\approx 0.615$  reduziert werden. Dies korrespondiert zu einer Verbesserung um ca. 20 Elo verglichen mit der untrainierten Variante.

Durch eine analytische Berechnung der Gradienten lässt sich die Anzahl der benötigten Durchläufe über den Trainingsdatensatz auf einmal pro Epoche reduzieren: Anstatt für jedes Gewicht zwei Iterationen über den Datensatz zu benötigen, muss für jede Epoche nur einmal über den Datensatz iteriert werden, um die Bewertung der einzelnen Evaluationsterme zu extrahieren [53]. Ein Term ist hierbei ein isolierter Aspekt der Evaluation; z.B. die Bewertung der Freibauern. Durch diesen Ansatz kann die Gradientenberechnung analytisch erfolgen, was eine Anpassung der Gewichte dann in  $\mathcal{O}(|w|)$  ermöglicht. Rückblickend hätte dieser Ansatz durch seine immense Zeiteinsparung eine enorme Steigerung in Flexibilität in der Tuning-Phase ermöglicht. Das Tuning auf diesen Ansatz umzubauen sollte höchste Priorität in zukünftiger Arbeit an der Engine haben.

## 7 Evaluation und Ergebnisse

In diesem Kapitel wollen wir die vorgestellten Algorithmen und Konzepte hinsichtlich ihrer Auswirkung auf Thunfischs Spielstärke evaluieren. Hierfür präsentieren wir mit SPRT eine etablierte statistische Methode, um die Spielstärke zweier Engines zu vergleichen. Mit Hilfe dieses Verfahren werden wir sukzessive präsentieren, wie viel Elo-Wertung einzelne Features gewinnen. Schlussendlich werden wir Thunfischs finale Elo-Wertung anhand der Referenz-Engine Stash festlegen.

### 7.1 Testmethodik

#### 7.1.1 Sequential Probability Ratio Test

Der Sequential Probability Ratio Test (SPRT) ist ein statistischer Hypothesentest zum Prüfen von Hypothesen auf einer variabel großen Stichprobe. Erstmals motiviert wurde das Verfahren in 1945 von Abraham Wald [54]. Ein nicht-sequentieller Hypothesentest mit den Hypothesen  $H_0$  und  $H_1$  trifft auf einer Stichprobe eine von zwei Entscheidungen:

1. Akzeptiere  $H_0$
2. Akzeptiere  $H_1$

SPRT kann eine zusätzliche entscheidung treffen [54]:

3. Vergrößere die Stichprobe um weitere Observationen

Durch diese dritte Auswahl ist es möglich, die Stichprobe so lange zu erweitern, bis eine Hypothese mit ausreichender Konfidenz akzeptiert werden kann. SPRT errechnet hierfür eine Log Likelihood Ratio LLR, die sich aus den geschätzten Wahrscheinlichkeiten  $P(\text{Probe} | H_0)$  und  $P(\text{Probe} | H_1)$  ergibt. In jedem Schritt wird LLR auf der bisher gesammelten Probe errechnet, und auf Basis eines Konfidenzintervalls  $[\alpha, \beta]$  eine Entscheidung getroffen:

- wenn  $\text{LLR} < \alpha$ : Akzeptiere  $H_0$
- wenn  $\text{LLR} > \beta$ : Akzeptiere  $H_1$
- wenn  $\alpha < \text{LLR} < \beta$ : Tätige weitere Observationen und gehe in die nächste Iteration

Das Konfidenzintervall ist hierbei abhängig von dem gewünschten statistischen Fehler, welcher parameterisierbar ist.

### 7.1.2 Anwendung in der Engine-Entwicklung

SPRT eignet sich durch seine sequentielle Methodik hervorragend, um statistisch signifikante Aussagen über die Stärke zweier Schach-Engines zu treffen. Aus diesem Grund ist das Verfahren der Standard Ansatz in der Engine-Programmierung, um zu verifizieren, ob eine Änderung an der Engine diese besser macht als zuvor.

Um solch eine Verifikation vorzunehmen, werden Hypothesen wie folgt definiert:

1.  $H_0$ : Das Feature gewinnt maximal 0 Elo
2.  $H_1$ : Das Feature gewinnt mindestens 10 Elo

Die Grenzen an Elo-Bewertung hierbei sind variabel. Prinzipiell gilt: je stärker die Engine, desto kleiner sollte das Intervall gewählt werden. Die Grenzen von 0 und 10 Elo haben sich in der Community für schwächere Engines wie Thunfisch etabliert [55].

Anschließend wird der SPRT durchgeführt, indem schrittweise Spiele zwischen der Engine mit der Änderung (NEW), und der Engine ohne Änderung (OLD) gespielt werden. Die Wahrscheinlichkeiten  $P(\text{Probe} \mid H_n)$  ergibt sich durch Gleichung 1. Um ein aussagekräftiges Resultat zu erhalten, kann allerdings nicht jedes Spiel von der Startposition beginnen. Durch den Determinismus der Alpha-Beta-Suche würde deshalb jedes Spiel genau gleich verlaufen, was effektiv einer konstanten Stichprobengröße von 1 entspricht. Um diesem Problem entgegen zu wirken, werden den beiden Engines die ersten  $n$  Züge des Spiels durch sogenannte Opening Books vorgegeben. Thunfisch nutzt hierfür den Datensatz „8moves\_v3.pgn“, welcher auch von Stockfish verwendet wird [56]. Der Datensatz enthält rund 35 000 Spiele mit je 8 Zügen pro Seite, also 16 Ply.

Die Verwendung von Opening Books führt allerdings ein weiteres Problem ein: Bias von Eröffnungen. Da die ersten 8 Züge des Spiels vorgeben sind, kann es sein, dass die resultierende Position bereits einen sehr starken Vorteil für eine der beiden Seiten darstellt. Mit einer ausreichend vorteilhaften Position kann eine schwächere Engine selbst eine deutlich stärkere Engine besiegen. Dieser Bias führt zu viel statistischem Rauschen, was die Konvergenz eines SPRT erschwert. Die Lösung hierfür nennt sich Pentanomiale Distribution: Anstatt jede Eröffnung nur einmal zu spielen, wird sie zwei mal gespielt; Einmal spielt NEW die weiße Seite, einmal die schwarze. Als Observation für den SPRT wird dann nicht mehr ein einzelnes Spiel genommen, sondern das Ergebnis

des Spielpaares einer Eröffnung. Dadurch ergeben sich fünf mögliche Ausgänge jeder Observation, was eine pentanomiale Verteilung bildet [57].

Ist die Eröffnung stark für eine Seite biased, gewinnt jede Engine diese Eröffnung einmal, was äquivalent ist dazu, als hätten beide Engines beide male unentschieden gespielt [8]. Dies entschärft das statistische Rauschen durch Bias in Eröffnungen erheblich.

### **7.1.3 Methodik während der Entwicklung**

Um während der Entwicklung Thunfischs statistisch fundierte, empirische Beweise zu haben, dass eine Änderung Thunfischs Performanz tatsächlich verbessert, wurde ein rigoroses Framework implementiert. Jede Änderung muss auf einem eigenen Branch implementiert werden, und anschließend durch eine Pull Request in den main branch aufgenommen werden. Als Parameter für den SPRT werden die oben genannten Elo Schranken von 0 und 10 Elo verwendet, sowie ein Fehler von 10% akzeptiert. Als Zeitkontrolle für jedes Spiel wird `8+0.08` Hyperbullet verwendet; Jede Engine beginnt mit 8 Sekunden auf der Uhr, und bekommt nach jedem Zug 0.8 Sekunden Zeitbonus. Auch arbeitet Thunfisch hier mit Pentanomialen Verteilungen. Um den SPRT durchzuführen, wird das Programm Fastchess [8] verwendet. Mithilfe von GitHub Actions [58] wurde eine CI/CD Pipeline aufgesetzt, die für offene Pull Requests automatisch einen SPRT durchführt, wenn diese mit dem Label `run-benchmark` versehen ist. Ein Auszug dieser CI/CD Pipeline ist in Listing 9 zu sehen.

```

1  name: Elo Benchmark between PR and base branch
2  on:
3    pull_request:
4      types: [opened, reopened, labeled, synchronize]
5
6  jobs:
7    #... Bauen der Engines base (OLD) und current (NEW)
8    benchmark:
9      if: contains(github.event.pull_request.labels.*.name, 'run-
      benchmark')
10     runs-on: self-hosted
11     container: heofthetea/thunfischsdk:latest-arm
12     timeout-minutes: 720 # 12 hours
13     # irrelevante Konfiguration
14     steps:
15       #... Vorbereitungen
16       - name: Run SPRT
17         run: |
18           fastchess \
19             -engine cmd=./testing/current name=dev \
20             -engine cmd=./testing/previous name=base \
21             -each proto=uci tc=8+0.08 \
22             -concurrency 2 \
23             -rounds 5000 \
24             -repeat \
25             -openings file=./testing/8moves_v3.pgn format=pgn
             order=random \
26             -sprt elo0=0 elo1=10 alpha=0.05 beta=0.05

```

Listing 9: Ausschnitt einer CI/CD Pipeline zum automatischen Durchführen eines SPRTs auf Pull Requests

In dem Listing ist in Zeilen 4 und 9 zu sehen, dass der Job nur ausgeführt wird, wenn die Pull Request das entsprechende Label besitzt. Auch zu sehen ist, dass der Job auf selbst gehosteten Job Runners läuft (Z. 10). Dies liegt daran, dass ein SPRT durchaus

zeitintensiv ist. Zeile 12 zeigt, dass der Job nach 12 Stunden abgebrochen wird. Die Erfahrung zeigt, dass in diese Zeit ca. 3500 Spiele gespielt werden können, was nicht selten nicht genug ist, um eine finale Entscheidung zu treffen. Für Thunfisch wird dies akzeptiert mit der Begründung, dass nach solch einer langen Zeit der Gewinn so minimal wäre, dass er auf der aktuellen Stärke der Engine nicht ausschlaggebend ist. Da Jobs selbst bei einem schnell konvergierenden SPRT häufig zwei bis drei Stunden laufen, wurde die Entscheidung getroffen, Runner selbst zu hosten, um Kosten an GitHub zu vermeiden. Benutzt wird hierfür das kostenlose Angebot eines etablierten Cloud-Anbieters.

## 7.2 Verbesserungen in Spielstärke einzelner Features

Ein Ziel dieser Studienarbeit ist es, herauszufinden, wie stark sich einzelne eingeführte Features auf die Spielstärke der Engine auswirken. Um eine Idee hierfür zu bekommen, wurden akkumulativ Wettkämpfe zwischen zwei Versionen der Engines mit jeweils einem Feature Unterschied gespielt<sup>10</sup>. Die Basis-Engine besitzt nur eine Minimax-Suche und die fertige Evaluation. Die zweite Version nimmt Alpha-Beta-Pruning hinzu, die dritte QS, usw. Jedes Spiel wurde mit der Zeitkontrolle  $40+0.4$  gespielt. Dies gibt den Engines mehr Zeit als der  $8+0.08$  Hyperbullet, der für die SPRT verwendet wurde, und verhindert dadurch Instabilitäten in der Suche. Anstatt eines SPRT wurden jedoch Wettkämpfe mit einer fixen Anzahl an Spielen (2300, mit Pentanomialem Modell macht dies 4600 einzelne Spiele) gespielt. Dies liegt daran, dass ein SPRT abbricht, sobald eine Hypothese angenommen wurde. Dies verhindert eine akkurate Schätzung der Elo-Wertung: Beispielsweise ist eine Elo-Differenz  $D = 500 \pm 200$  Elo trotz des großen Fehlers weit über der Schranke  $H_1 = 10$  Elo, weshalb die Elo-Differenz nicht genauer errechnet wird.

Die Ergebnisse sind grafisch in Abbildung 24, und tabellarisch inklusive Schätzungsfehlern in Tabelle 2 aufgeführt. Abbildung 24 fokussiert sich auf die absoluten Elo-Werte, die jeder Version zuzuordnen sind. Diese absoluten Werte ergeben sich aus Rückrechnungen von der Version mit allen Features (s. Abschnitt 7.4). Die Elo-Differenzen der einzelnen Versionen wurden dann sukzessive von dieser

---

<sup>10</sup>Unter „Feature“ ist jeweils eine isoliert zu betrachtende Optimierung der simplen Minimax-Suche zu verstehen. Meist korrespondiert ein Feature zu einem Kapitel in Abschnitt 5. „Eval“ umfasst alle Erweiterungen der Evaluation über die ursprünglichen PeSTO-Tabellen, inklusive Tuning.

Version abgezogen. Die Schätzfehler der einzelnen Wettkämpfe wurden hierbei der Übersichtlichkeit halber nicht mit einberechnet. Der kumulierte Fehler nach Tabelle 2 errechnet sich auf 110,89, woraus sich für die Wertung der Basis-Version das Konfidenz-Intervall  $\approx [565, 787]$  ergibt.

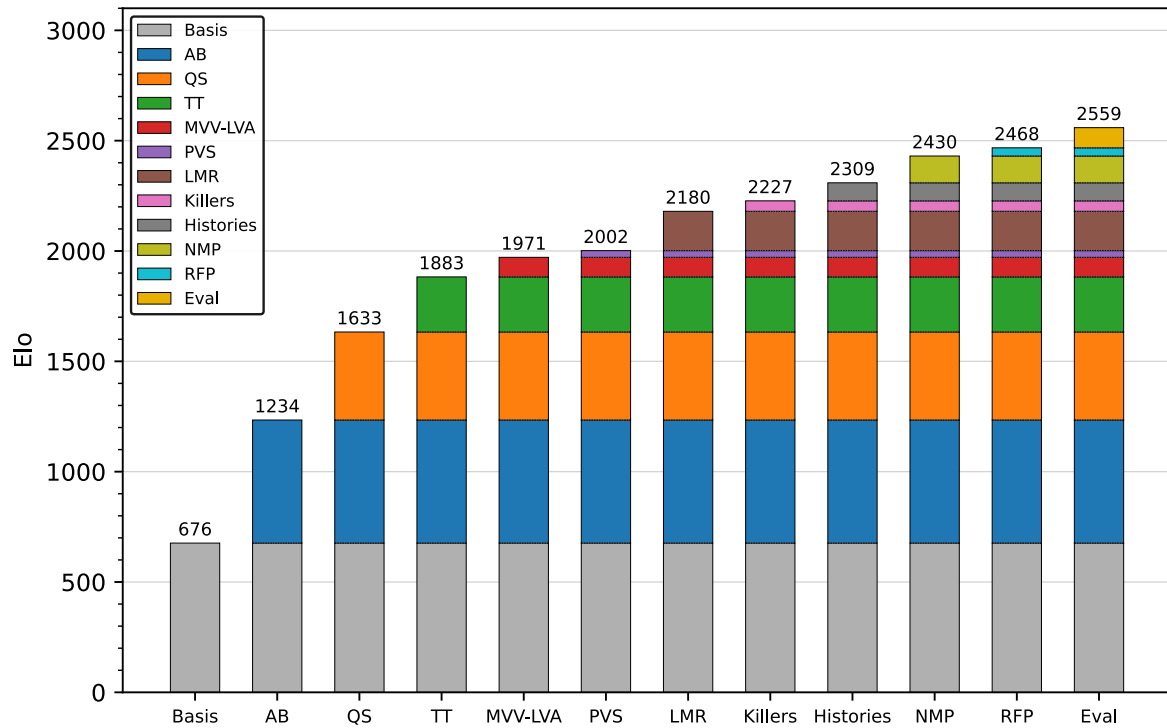


Abbildung 24: Elo-Gewinn jedes Features

Auffallend sind vor allem der scheinbar exponentielle Abfall der Gewinne jedes Features. Dies zeigt, dass Alpha-Beta-Pruning und Quiescence Search alleine bereits in der Lage sind, die Engine auf eine Ebene mit durchschnittlichen menschlichen Schachspielern zu bringen. Danach haben den größten Einfluss auf die Spielstärke der Engine die Transpositionstabelle, und Late Move Reductions. Ersteres lässt sich erklären durch die Vielseitigkeit der Transpositionstabelle, welche sowohl bei der Zugsortierung, als auch als Forward Pruning Mechanismus zentrale Aufgaben übernimmt. Der Elo-Gewinn von LMR kann erklärt werden dadurch, dass durch die Reduktion der tieferen Ebenen des Suchbaums eine Tiefe bedeutend schneller gesucht werden kann, und die Iterative Deepening Schleife dadurch deutlich tiefer kommt (vgl. Abbildung 25). Ebenfalls auffallend ist die Tatsache, dass PVS alleinstehend beinahe kaum an Stärke gewinnt. Dies wird allerdings ausgeglichen durch die vielen Synergien, die PVS mit Forward Pruning Mechanismen bildet. Da NMP und RFP nur auf Knoten abseits der PV

durchgeführt werden, können diese radikaler vorgehen, ohne bei den wichtigsten Knoten auf der PV zu riskieren, einen guten Zug zu übersehen.

| <b>Feature</b>         | <b>Elo Gewinn</b> | <b><math>\pm</math> Error</b> |
|------------------------|-------------------|-------------------------------|
| Alpha-Beta             | +546              | 16,94                         |
| QS                     | +402              | 13,81                         |
| TT Move Ordering       | +230              | 10,02                         |
| TT Cutoffs             | +22               | 7,67                          |
| MVV-LVA                | +82               | 8,43                          |
| PVS                    | +29               | 7,80                          |
| LMR                    | +172              | 8,79                          |
| Killer Moves           | +46               | 7,28                          |
| History Heuristic      | +85               | 7,38                          |
| NMP                    | +126              | 7,54                          |
| RFP                    | +37               | 6,97                          |
| Verbesserte Evaluation | +91               | 8,26                          |

Tabelle 2: Elo-Gewinn mit Schätzungsfehler jedes Features

Hervorzuheben ist außerdem die Verbesserung in Elo, die die Änderungen an der Evaluation mit sich bringen. Dies zeigt, dass die zusätzlichen Terme Erfolg hatten dabei, die beobachteten Schwächen in Thunfischs Spiel zu beheben.

### 7.3 Verbesserungen in Suchtiefe

Die primäre Evaluation der Stärke einer Engine sollte durch die Elo-Wertung erfolgen. Andere Metriken wie besuchte Knoten pro Sekunde, Verzweigungsfaktor des Suchbaums oder zeitliche Metriken sind keine optimalen Indikatoren der Stärke einer Engine. Ein Beispiel illustriert dies gut: Es lässt sich behaupten, dass je kleiner der Verzweigungsfaktor, desto tiefer kann die Engine suchen, was für das Spiel der Engine besser ist. Diese pauschale Behauptung kann einfach widerlegt werden durch eine Engine, die auf jeder Tiefe genau einen Zug sucht. Dies reduziert den Verzweigungsfaktor auf ein Minimum, allerdings ist der gesuchte Zug nicht immer der

beste, und eine solche Engine würde vermutlich selbst gegen eine Engine mit nichts weiter als einer unoptimierten Minimax-Suche verlieren.

Dennoch wollen wir eine dieser alternativen Metriken präsentieren: Die Zeit, die die Engine auf jeder Suchtiefe des Iterative Deepening aufbringen muss. Obwohl alleine auf Basis dieser Metrik keine direkten Rückschlüsse auf die Stärke der Engine zu schließen sind, kann sie Erklärungen dafür liefern, wieso einzelne Features die Elo-Wertung der Engine verbessern.

Abbildung 25 zeigt dies für die einzelnen Features, die in Thunfisch implementiert wurden. Hierfür wurde mit jeder Version<sup>11</sup> aus Abbildung 24 eine Suche ausgehend von der Startposition gestartet. Jeder Engine wurden 10 Sekunden Zeit gegeben. Für jede Suchtiefe wurde die kumulierte Zeit erfasst. Die Ergebnisse sind in Abbildung 25 zu sehen.

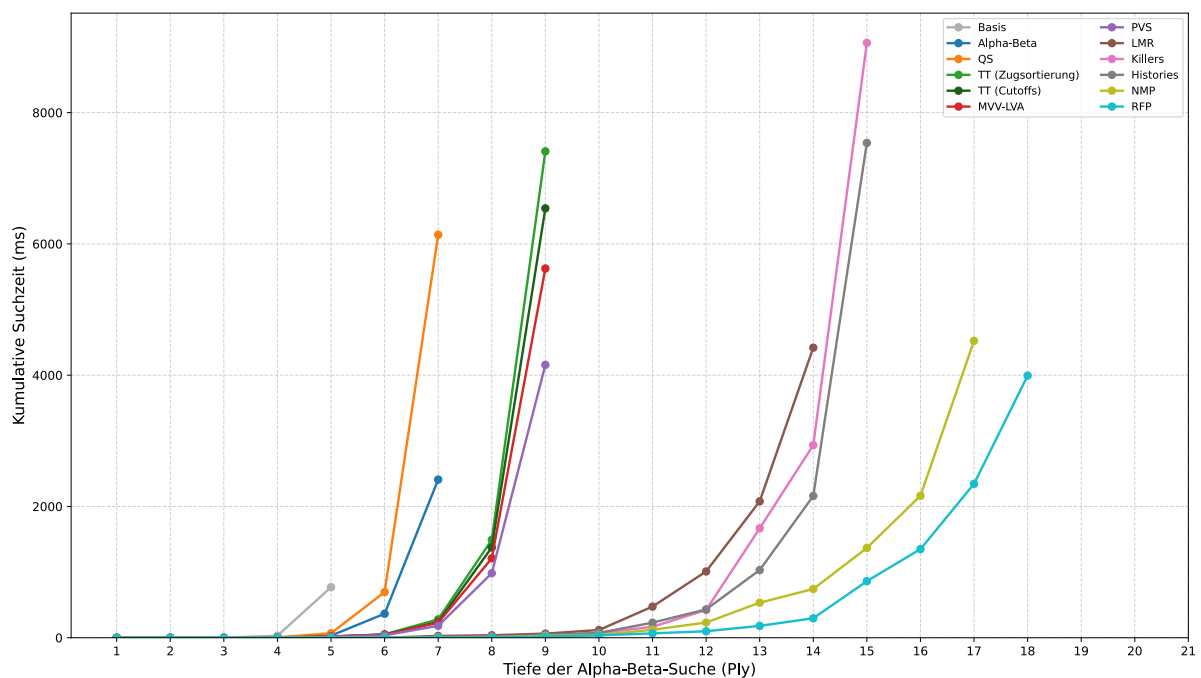


Abbildung 25: Auswirkung von Features auf die Suchtiefe

Zu erkennen ist, dass jedes Feature die benötigte Zeit auf einer Tiefe reduziert. Die einzige Ausnahme hierbei ist QS. Dies liegt daran, dass durch die Einführung der Quiescence Search nach Erreichen der gewollten Tiefe noch eine weitere Suche

<sup>11</sup>Ausgenommen der zusätzlichen Evaluation, da diese keine signifikante Auswirkung auf die Suchtiefe hat.

stattfindet. Diese Suche benötigt Zeit, weshalb die Engine mit nur Alpha-Beta-Pruning schneller ist, als wenn QS eingeführt wird. Da QS dennoch signifikant an Elo gewinnt, zeigt sich hier wieder, dass die benötigte Zeit pro Suchtiefe keine eindeutige Metrik ist, um die Stärke der Engine festzustellen.

Sehr bemerkenswert ist die zeitliche Verbesserung, die durch Late Move Reductions zustande kommt. Während die meisten Features maximal eine zusätzliche Tiefe in derselben Zeit wie eine vorherige Version erreichen, bringt LMR einen Sprung von 4-5 zusätzlichen Tiefen in derselben Zeit wie die vorherige Version. Ebenfalls auffällig ist die Tatsache, dass PVS beinahe keine Beschleunigung der Suche mit sich bringt. Ähnlich wie bei der Betrachtung der Spielstärke kann auch hier wieder argumentiert werden, dass PVS die Forward Pruning Mechanismen NMP und RFP unterstützt, welche ihrerseits wiederum sichtbare Verbesserungen einbringen.

## 7.4 Bestimmung der Elo-Wertung

Um die absolute Spielstärke Thunfischs zu ermitteln, wurde ein Turnier gegen zwei Versionen der Referenz-Engine Stash durchgeführt. Stash wurde gewählt, da die Engine in der Szene als verlässlicher Maßstab für Elo-Bewertung gilt und ihre Elo-Wertungen von der CCRL allgemein anerkannt sind. Die verwendeten Versionen waren Stash 21 mit 2713 Elo und Stash 20 mit 2509 Elo. Die Werte stammen aus der Kategorie Blitz. Gespielt wurden jeweils 10000 Eröffnungspaare gegen jede der beiden Stash-Versionen. Die Partien wurden auf einem Intel Xeon Platinum Prozessor ausgetragen mit einer Zeitkontrolle von  $40 + 0.4$  und der bereits erwähnten Eröffnungsdatenbank `8moves_v3.pgn` mit pentanomalem Modell. Tabelle 3 zeigt die Ergebnisse des Turniers.

| Engine    | Elo  | $\pm$ | LL   | LD   | DD/WL | WD   | WW   |
|-----------|------|-------|------|------|-------|------|------|
| Stash 21  | +127 | 4,24  | 341  | 1044 | 2763  | 3001 | 2851 |
| Thunfisch | -25  | 3,05  | 3561 | 4341 | 6119  | 3409 | 2570 |
| Stash 20  | -72  | 4,17  | 2229 | 2365 | 3356  | 1340 | 710  |

Tabelle 3: Elo-Ergebnisse des Gauntlet-Turniers aus Sicht der Stash-Referenz-Engines

Die Spalten LL, LD, DD/WL, WD und WW kodieren die pentanomalen Ergebnisse: LL steht für zwei Niederlagen, LD für eine Niederlage und ein Remis, DD/WL für zwei

Remis oder je einen Sieg und eine Niederlage, WD für einen Sieg und ein Remis, WW für zwei Siege (jeweils aus Sicht der referenzierten Engine in der ersten Spalte). Die Summe aller fünf Spalten ergibt die Anzahl der Eröffnungspaare, die Gesamtzahl der Partien ist entsprechend doppelt so hoch.

Da das Turnier als Gauntlet ausgetragen wurde, spielte ausschließlich Thunfisch gegen beide Gegner, während die Stash-Versionen nicht gegeneinander antraten. Dadurch lässt sich die absolute Spielstärke von Thunfisch direkt über die bekannten Fixpunkte der Referenz-Engines kalibrieren.

Die beiden Berechnungen des Turnier-Nullpunkts liegen mit 2586 Elo (abgeleitet aus Stash 21) und 2581 Elo (abgeleitet aus Stash 20) dicht beieinander. Der Mittelwert dieses Nullpunkts beträgt 2584 Elo. Verrechnet man diesen Nullpunkt mit der relativen Turnierleistung von Thunfisch ( $-25$  Elo), ergibt sich eine kalibrierte absolute Spielstärke von 2559 Elo.

Die Auswertung der Pentanomial-Verteilung gibt zusätzlich Aufschluss über die statistische Sicherheit des Ergebnisses. Aus dem im Turnier ermittelten Schätzfehler von  $\pm 3,05$  Elo für Thunfisch resultiert ein Konfidenzintervall von gerundet  $[2556, 2562]$  Elo innerhalb dieser Testumgebung. Damit wird das definierte Ziel von 2500 Elo nicht nur erreicht, sondern deutlich übertroffen.

Hervorzuheben ist, dass Thunfisch selbst gegen die stärkere Engine Stash 21 (2713 Elo) in 341 von 10000 Eröffnungspaaren beide Partien für sich entscheiden konnte und in 1044 der Paare einen Sieg mit einem Remis kombinierte. Gegen die schwächere Engine Stash 20 (2509 Elo) fiel der Vorsprung mit 2229 vollständigen Siegpaaren und 2365 Sieg-Remis-Paaren deutlich aus. Zwar verlor Thunfisch auch hier 710 Paare komplett und in 1340 Paaren gelang nur ein Remis, dennoch konnte Thunfisch Stash 20 klar schlagen, was die berechnete Spielstärke von über 2550 Elo im Turnierkontext bestätigt.

## 8 Zusammenfassung und Ausblick

Im Zuge dieser Studienarbeit wurde die Schach-Engine Thunfisch von einem rudimentären Prototypen in eine ernstzunehmende Engine mit einer Spielstärke von etwa 2560 Elo entwickelt. Ausgehend von einer naiven Minimax-Suche und einer simplen materialbasierten Evaluation wurden über mehrere Entwicklungsiterationen hinweg sukzessive Optimierungen implementiert und deren Einfluss auf die Spielstärke mittels Sequential Probability Ratio Tests statistisch validiert.

Die Basis der Engine bildet eine Bitboard-Repräsentation des Spielbretts, die sämtliche Board-Operationen auf effiziente bitweise Instruktionen abbildet. Darauf aufbauend wurde ein legaler Zuggenerator entwickelt, der durch maskenbasierte Filterung direkt legale Züge erzeugt, ohne nachträgliche Legalitätsprüfungen durchzuführen. Der Generator folgt dabei weitgehend dem Prinzip des Branchless Programming, um Pipeline-Stalls durch fehlklassifizierte Sprungvorhersagen zu vermeiden.

Den Kern der Engine stellt eine Alpha-Beta-Suche dar, die durch mehrere Erweiterungen optimiert wurde. Die Quiescence Search (QS) verhindert den Horizon Effect, indem sie Blattpositionen in taktischen Sequenzen bis zu einer ruhigen Stellung weiterverfolgt. Die Principal Variation Search (PVS) optimiert die Suche durch Zero Window für Knoten abseits der PV. Eine Transpositionstabelle (TT) cached bereits berechnete Stellungen und liefert bevorzugte Züge für die Sortierung. Iterative Deepening erlaubt die dynamische Ausnutzung verfügbarer Bedenkzeit. Für die Sortierung der Züge wurden MVV-LVA, Killer-Heuristik, History-Heuristik und ein Move Picker implementiert, der die Generierung staffelt, um unnötige Berechnungen zu vermeiden. Forward Pruning durch Null Move Pruning (NMP) und Reverse Futility Pruning (RFP) verwirft aussichtslose Pfade frühzeitig. Late Move Reductions (LMR) reduzieren die Suchtiefe für vermeintlich schwächere Züge und erzielen den größten Effekt auf die Suchtiefe, mit einem Sprung von vier bis fünf zusätzlichen Tiefen in derselben Bedenkzeit.

Viele dieser Optimierungen entfalten ihre Wirkung erst im Zusammenspiel. PVS allein bringt mit 29 Elo kaum Gewinn, ermöglicht aber ein aggressiveres Vorgehen von NMP und RFP (126 bzw. 37 Elo), da diese nur auf Knoten abseits der PV angewendet werden.

Ebenso liegt der eigentliche Wert der TT nicht im Cachen von Bewertungen (22 Elo), sondern in der Sortierung des besten Zuges (230 Elo).

Die statische Evaluationsfunktion basiert auf Piece Square Table (PSQT) nach PeSTO, die zwischen Mittel- und Endspiel interpolieren. Darüber hinaus wurden Terme für Freibauern, Doppelbauern und Königssicherheit implementiert, um beobachtete strategische Schwächen in Thunfischs Spiel zu adressieren. Mittels Texel Tuning mit numerisch berechneten Gradienten wurden die Gewichte der Evaluation über 300 Epochen auf einem Datensatz von 1,4 Millionen Positionen optimiert, was den Mean Square Error von 0,640 auf 0,615 senkte und einer Verbesserung von etwa 20 Elo entspricht.

In einer abschließenden Evaluation wurde die absolute Spielstärke Thunfischs durch ein Gauntlet-Turnier gegen die Referenz-Engine Stash (Versionen 20 und 21) über insgesamt 40000 Partien bestimmt. Die relative Turnierleistung in Höhe von  $-25$  Elo bei einem Nullpunkt von 2584 Elo ergibt eine kalibrierte Spielstärke von 2559 Elo mit einem Konfidenzintervall von etwa 2556 bis 2562 Elo innerhalb der Testumgebung. Unter Berücksichtigung von Hardware-Differenzen und abweichender Zeitkontrollen zur CCRL-Skala wird die reale Spielstärke auf 2550 bis 2570 Elo geschätzt. Damit wird das eingangs definierte Ziel von 2500 Elo nicht nur erreicht, sondern übertroffen.

Auf der Plattform Lichess, auf der Thunfisch autonom kontinuierlich gegen andere Engines spielt, pendelte sich die Wertung in den Zeitkontrollen Blitz und Bullet bei etwa 2350 Elo ein. Dieser Unterschied zur CCRL-skalierten Wertung ist spekulativ unter anderem der Tatsache zuzuordnen, dass Lichess an Stelle des konventionellen Elo-Systems das alternative Glicko-2-System verwendet, und Bewertungen deshalb nicht exakt ineinander überführbar sind [59].

## 8.1 Limitationen

Trotz der signifikanten erzielten Ergebnisse weist die aktuelle Implementierung Thunfischs nach wie vor Limitierungen auf. Die implementierten heuristischen Verbesserungen der Alpha-Beta-Suche schöpfen noch nicht das vollständige Potenzial aus, das die aktuell besten Engines aus ihnen gewinnen. Beispielsweise wurde bei der Implementierung von Late Move Reductions komplett auf Domänenwissen-basierte Reduktionen und Erweiterungen der Suchtiefe verzichtet. Ein Blick in führende Engines

wie Stockfish oder Viridithas zeigt jedoch, dass diese die Performanz von LMR erheblich verbessern können.

Eine weitere Limitation zeigt sich in der aktuellen Form des Tunings der Evaluationsfunktion. Durch die Entscheidung, Gradienten numerisch zu berechnen, ist das Tuning stark limitiert in der Anzahl der Epochen, die durchgeführt werden können, sowie in der möglichen Größe der Trainingsdaten. Als Resultat führt das Training zwar zu Verbesserungen in der Evaluation, allerdings beschränken sich diese auf 20 Elo in durchgeführten Tests. Österlund erwähnt einen Gewinn von 99,6 Elo-Punkten durch seine Methode [51], ähnliche Margen finden sich in diversen Berichten der Community. Dies lässt vermuten, dass durch eine größere Datenmenge und schnelleres Tuning durchaus mehr Elo durch den Tuning-Prozess gewonnen werden kann.

## 8.2 Ausblick

Obwohl Thunfischs Leistung durch die implementierten Erweiterungen erheblich gesteigert werden konnte, implementiert diese Studienarbeit lediglich einen kleinen Anteil an möglichen Optimierungen. Durch Jahrzehnte an Forschung und kontinuierliche Aufwände der Community in populären Open Source Projekten ist ein breites Sammelsurium an Wissen entstanden, wie die Performanz einer Alpha-Beta-Suche in vielerlei Hinsicht verbessert werden kann. Überfliegt man die aktuell stärksten Open-Source Engines, findet man beinahe überall weitere Techniken, Optimierungen und Ansätze, die diese Engines weit über das aktuelle Niveau von Thunfisch hinaus heben. Folgende Liste soll einen Überblick geben über einige der am weitesten verbreiteten Optimierungen und Techniken, die in Thunfisch aktuell noch nicht implementiert sind. Diese Liste ist aus Durchsicht anderer Open Source Engines [12], [15], [43] und Artikeln des Chess Programming Wiki [60] zusammengetragen:

- Aspiration Windows (Verbesserung des Iterative Deepenings)
- Bessere Zeitkontrolle mit weichen und harten Grenzen
- Razoring (Forward Pruning basierend auf QS)
- Futility Pruning (ähnlich zu RFP)
- Multi-Cut Pruning (Forward Pruning basierend auf der Anzahl an Zügen, die einen fail-high Cutoff erzeugen)
- Countermove Heuristics (Erweiterung von History Heuristics)

- Static Exchange Evaluation (Verbesserte Sortierung für schlagende Züge, sowie Kriterium für Forward Pruning)
- Parallelisierung der Suche nach Lazy SMP oder Young Brother's Wait Concept

Auch die statische Evaluation kann bedeutend verbessert werden. Ein Punkt ist die Verbesserung des Tuning Mechanismus von numerischen auf analytische Gradienten. Dies würde es erlauben, der Evaluation weitere Terme hinzuzufügen und die hinzugefügten Gewichte sehr schnell in die bereits existierende Evaluation zu integrieren. Abseits dessen allerdings haben die führenden Engines bereits seit einiger Zeit den Bereich der PSQT-basierten Evaluation hinter sich gelassen. Mit den rapiden Fortschritten von Neuronalen Netzwerken haben diese auch ihren Weg in die Engine-Programmierung gefunden: Efficiently Updatable Neural Networks (NNUE) sind in führenden Engines der Standard für die statische Positionsevaluierung, da sie erhebliche Verbesserungen in Spielstärke gegenüber dem PSQT-Ansatz liefern. Wenn Thunfisch eines Tags an führende Engines herankommen will, wird die Evaluation zwangsläufig auf NNUE umgestellt werden müssen.

Abseits der schieren Verbesserung der Spielstärke kann ein weiteres Ziel sein, Thunfisch in verschiedene anerkannte Ranglisten aufzunehmen. Das prominenteste Beispiel hierfür sind die bereits erwähnten CCRL. In der Zukunft ist es geplant, sich dort gegen die besten Engines der Welt zu messen.

Abschließend lässt sich sagen, dass die Entwicklung einer Schach-Engine niemals abgeschlossen ist. Diese Studienarbeit hat aufgezeigt, welches Potential in der Optimierung einer simplen Alpha-Beta-Suche steckt, sowie dass die Sprache Rust eine hervorragende Grundlage liefert für maximale Performanz. Wir sind entschlossen, das Projekt Thunfisch weiter zu führen mit dem Ziel, die Engine so gut werden zu lassen wie möglich. Als nächstes konkretes Ziel kann die Schranke von 3000 Elo ausgerufen werden.

## Literatur

- [1] C. E. Shannon, „Programming a Computer for Playing Chess“, *Computer Chess Compendium*. Springer, New York, NY, S. 2–13, 1988. doi: 10.1007/978-1-4757-1968-0\_1.
- [2] T. Standage, *The Turk*. Martinsville, VA: Walker Books, 2002.
- [3] IBM, „Deep Blue“. Zugegriffen: 10. Juni 2026. [Online]. Verfügbar unter: <https://www.ibm.com/history/deep-blue>
- [4] D. Silver u. a., „Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm“. Zugegriffen: 13. Februar 2026. [Online]. Verfügbar unter: <http://arxiv.org/abs/1712.01815>
- [5] FIDE, „B. PERMANENT COMMISSIONS / 01. International Title Regulations (Qualification Commission) / FIDE Title Regulations“. Zugegriffen: 7. Juni 2026. [Online]. Verfügbar unter: <https://handbook.fide.com/chapter/B012023>
- [6] S. Meyer-Kahlen, „Universal Chess Interface (UCI) Protocol“. 2000.
- [7] A. E. Elo, *The Rating of Chessplayers, Past and Present*. Arco Pub., 1978.
- [8] Disservin, „Disservin/fastchess“. Zugegriffen: 4. Juni 2026. [Online]. Verfügbar unter: <https://github.com/Disservin/fastchess>
- [9] C. Thompson, „How Rust Went from a Side Project to the World’s Most-Loved Programming Language“. Zugegriffen: 22. Januar 2026. [Online]. Verfügbar unter: <https://www.technologyreview.com/2023/02/14/1067869/rust-worlds-fastest-growing-programming-language/>
- [10] STACKOVERFLOW, „2025 Stack Overflow Developer Survey“. Zugegriffen: 22. Januar 2026. [Online]. Verfügbar unter: <https://survey.stackoverflow.co/2025/technology>
- [11] D. Spracklen und K. Spracklen, *SARGON: A Computer Chess Program*. 1978.
- [12] STOCKFISH COMMUNITY, „Stockfish“. Zugegriffen: 27. Februar 2026. [Online]. Verfügbar unter: <https://stockfishchess.org/>

- [13] G. Banks, „Computer Chess Rating Lists“. Zugegriffen: 7. Juni 2026. [Online]. Verfügbar unter: <https://computerchess.org.uk/4040/about.html>
- [14] A. Surkov, „Reckless“. Zugegriffen: 27. Februar 2026. [Online]. Verfügbar unter: <https://github.com/codedeliveryservice/Reckless>
- [15] C. Bobak, „Cosmobobak/Viridithas“. Zugegriffen: 24. Oktober 2025. [Online]. Verfügbar unter: <https://github.com/cosmobobak/viridithas>
- [16] S. Klabnik, C. Nichols, und C. Kryocho, *The Rust Programming Language, 3rd Edition*. San Francisco, CA: No Starch Press, 2026.
- [17] RUST FOUNDATION, „The Embedded Rust Book“, [Online]. Verfügbar unter: <https://doc.rust-lang.org/beta/embedded-book/>
- [18] S. Slotin, „Branchless Programming - Algorithmica“. Zugegriffen: 3. März 2026. [Online]. Verfügbar unter: <https://en.algorithmica.org/hpc/pipelining/branchless/>
- [19] RUST FOUNDATION, „Constant Evaluation - The Rust Reference“. Zugegriffen: 5. März 2026. [Online]. Verfügbar unter: [https://doc.rust-lang.org/reference/const\\_eval.html#constant-evaluation](https://doc.rust-lang.org/reference/const_eval.html#constant-evaluation)
- [20] FIDE, „Laws of Chess“. [Online]. Verfügbar unter: <https://www.fide.com/FIDE/handbook/LawsOfChess.pdf>
- [21] CHESS PROGRAMMING WIKI, „Bitboards“. Zugegriffen: 27. Februar 2026. [Online]. Verfügbar unter: <https://www.chessprogramming.org/Bitboards>
- [22] CHESS PROGRAMMING WIKI, „Square Mapping Considerations“. Zugegriffen: 27. Februar 2026. [Online]. Verfügbar unter: [https://www.chessprogramming.org/Square\\_Mapping\\_Considerations](https://www.chessprogramming.org/Square_Mapping_Considerations)
- [23] A. Persaud, „Polyglot Book Format“. Zugegriffen: 8. Juni 2026. [Online]. Verfügbar unter: [http://hgm.nubati.net/book\\_format.html](http://hgm.nubati.net/book_format.html)
- [24] R. M. Hyatt und A. Cozzie, „THE EFFECT OF HASH SIGNATURE COLLISIONS IN A CHESS PROGRAM“, *ICGA Journal*, Bd. 28, Nr. 3, S. 131–139, Sep. 2005, doi: 10.3233/ICG-2005-28302.

- [25] CHESS PROGRAMMING WIKI, „Bitboard Serialization“. Zugriffen: 20. Februar 2026. [Online]. Verfügbar unter: [https://www.chessprogramming.org/Bitboard\\_Serialization](https://www.chessprogramming.org/Bitboard_Serialization)
- [26] D. Inführ, „Worlds Fastest Bitboard Chess Movegenerator“. Zugriffen: 24. Oktober 2025. [Online]. Verfügbar unter: <https://www.codeproject.com/articles/Worlds-Fastest-Bitboard-Chess-Movegenerator>
- [27] CHESS PROGRAMMING WIKI, „Magic Bitboards“. Zugriffen: 20. Februar 2026. [Online]. Verfügbar unter: [https://www.chessprogramming.org/Magic\\_Bitboards](https://www.chessprogramming.org/Magic_Bitboards)
- [28] P. Kannan, „Magic Move-Bitboard Generation in Computer Chess“, 2007.
- [29] V. Anuss, „Magic Numbers“. Zugriffen: 9. März 2026. [Online]. Verfügbar unter: <https://www.talkchess.com/forum/viewtopic.php?t=64790>
- [30] „Number of possible chess games at the end of the n-th ply - OEIS“. Zugriffen: 16. Februar 2026. [Online]. Verfügbar unter: <https://oeis.org/A048987>
- [31] D. J. Edwards und T. P. Hart, „The Alpha-Beta Heuristic“, Dez. 1961, Zugriffen: 22. Februar 2026. [Online]. Verfügbar unter: <https://dspace.mit.edu/handle/1721.1/6098>
- [32] D. N. L. Levy und M. Newborn, *How Computers Play Chess*. New York, NY: W.H. Freeman, 1990.
- [33] J. Pearl, „A Simple Game-Searching Algorithm with Proven Optimal Properties“, 1980.
- [34] T. Marsland, „A Review of Game-Tree Pruning“, *ICGA Journal*, Bd. 9, Nr. 1, S. 3–19, März 1986, doi: 10.3233/ICG-1986-9102.
- [35] D. Breuker und J. Uiterwijk, „Transposition Tables in Computer Chess“, Feb. 1970.
- [36] J. Whiting, „Akimbo“. Zugriffen: 7. Juni 2026. [Online]. Verfügbar unter: <https://github.com/jw1912/akimbo>
- [37] D. Breuker, J. Uiterwijk, und H. Herik, „Information in Transposition Tables“, Bd. 8, Feb. 1970.

- [38] M. Boulé und Z. Zilic, „AN FPGA MOVE GENERATOR FOR THE GAME OF CHESS“, *ICGA Journal*, Bd. 25, Nr. 2, S. 85–94, Juni 2002, doi: 10.3233/ICG-2002-25203.
- [39] D. N. L. Levy, *Computer Chess Compendium*. Springer New York, 1988. doi: 10.1007/978-1-4757-1968-0.
- [40] J. Schaeffer, „The History Heuristic“, *ICGA Journal*, Bd. 6, Nr. 3, S. 16–19, Aug. 1983, doi: 10.3233/ICG-1983-6305.
- [41] D. Hartmann, „Butterfly Boards“, *ICGA Journal*, 1988, doi: 10.3233/icg-1988-112-303.
- [42] M. H. M. Winands, E. C. D. Van Der Werf, H. J. Van Den Herik, und J. W. H. M. Uiterwijk, „The Relative History Heuristic“, *Computers and Games*, Bd. 3846. Springer Berlin Heidelberg, Berlin, Heidelberg, S. 262–272, 2006. doi: 10.1007/11674399\_18.
- [43] gab8192, „gab8192/Obsidian“. Zugegriffen: 30. Mai 2026. [Online]. Verfügbar unter: <https://github.com/gab8192/Obsidian>
- [44] T. Kirstihagen, „TerjeKir/weiss“. Zugegriffen: 25. Mai 2026. [Online]. Verfügbar unter: <https://github.com/TerjeKir/weiss>
- [45] K. Hoki und M. Muramatsu, „Efficiency of three forward-pruning techniques in shogi: Futility pruning, null-move pruning, and Late Move Reduction (LMR)“, *Entertainment Computing*, Bd. 3, Nr. 3, S. 51–57, Aug. 2012, doi: 10.1016/j.entcom.2011.11.003.
- [46] L. Kaufman, „The Evaluation of Material Imbalances“, März 1999, Zugegriffen: 11. März 2026. [Online]. Verfügbar unter: <https://pdfcoffee.com/the-evaluation-of-material-imbalances-pdf-free.html>
- [47] R. Friederich, „New uci engine: Rofchade - Page 2 - TalkChess.com“. Zugegriffen: 11. März 2026. [Online]. Verfügbar unter: <https://www.talkchess.com/forum3/viewtopic.php?f=2&t=68311&start=19>
- [48] D. Beal und M. Smith, „Learning Piece-square Values using Temporal Differences“, *ICGA Journal*, Bd. 22, Nr. 4, S. 223–235, Dez. 1999, doi: 10.3233/ICG-1999-22404.

- [49] S. Lague, „SebLague/Chess-Coding-Adventure“. Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://github.com/SebLague/Chess-Coding-Adventure>
- [50] CHESS PROGRAMMING WIKI, „King Safety“. Zugegriffen: 10. Juni 2026. [Online]. Verfügbar unter: [https://www.chessprogramming.org/King\\_Safety](https://www.chessprogramming.org/King_Safety)
- [51] P. Österlund, „The Texel Evaluation Function Optimization Algorithm“. Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://www.talkchess.com/forum/viewtopic.php?t=50823&start=26>
- [52] A. Grant, „Evaluation & Tuning in Chess Engines“, *GitHub*, Okt. 2020, Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://github.com/AndyGrant/Ethereal/blob/master/Tuning.pdf>
- [53] P. Heck, „FitzOReilly/fatalii“. Zugegriffen: 31. Mai 2026. [Online]. Verfügbar unter: <https://github.com/FitzOReilly/fatalii>
- [54] A. Wald, „Sequential Tests of Statistical Hypotheses“, *The Annals of Mathematical Statistics*, Bd. 16, Nr. 2, S. 117–186, 1945, Zugegriffen: 3. Juni 2026. [Online]. Verfügbar unter: <https://www.jstor.org/stable/2235829>
- [55] CHESS PROGRAMMING WIKI, „Sequential Probability Ratio Test“. Zugegriffen: 3. Juni 2026. [Online]. Verfügbar unter: [https://www.chessprogramming.org/Sequential\\_Probability\\_Ratio\\_Test](https://www.chessprogramming.org/Sequential_Probability_Ratio_Test)
- [56] STOCKFISH COMMUNITY, „official-stockfish/books“. Zugegriffen: 3. Juni 2026. [Online]. Verfügbar unter: <https://github.com/official-stockfish/books>
- [57] vdbergh, „vdbergh/pentanomial“. Zugegriffen: 4. Juni 2026. [Online]. Verfügbar unter: <https://github.com/vdbergh/pentanomial>
- [58] GITHUB, „GitHub Actions documentation“. Zugegriffen: 4. Juni 2026. [Online]. Verfügbar unter: <https://docs-internal.github.com/en/actions>
- [59] COMPUTER CHESS INDEX, „CCI Ratings (ELO Ranking)“. Zugegriffen: 8. Juni 2026. [Online]. Verfügbar unter: <https://computer-chess-index.github.io/cci/Lichess-CCI-Bots.html>

- [60] M. Lefler, „Chessprogramming Wiki (CPW)“. Zugegriffen: 9. Januar 2025. [Online].  
Verfügbar unter: <https://www.chessprogramming.org/>

## Anmerkung zur Nutzung von Künstlicher Intelligenz

Im Rahmen dieser Arbeit wurden KI-basierte Werkzeuge zur Unterstützung eingesetzt. Der Einsatz ist nachfolgend tabellarisch aufgeführt.

| Werkzeug                                                                           | Beschreibung der Nutzung                                                       |
|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|
| DeepSeek (DeepSeek V4 Flash, DeepSeek V4 Pro)                                      | Rechtschreibkorrektur und Grammatikkorrektur sowie Formulierungshilfe (gesamt) |
| Google Gemini (Gemini 2.5 Flash, Gemini 2.5 Pro, Gemini 3.1 Pro, Gemini 3.5 Flash) | Formulierungshilfe sowie Unterstützung beim allgemeinen Verständnis von Themen |